

BLOCKCHAIN DEEP LEARNING ROBOTIC PROCESS AUTOMATION

**SUBMITTED IN PARTIAL FULFILLMENT OF THE
REQUIREMENTS FOR THE AWARD OF
THE DEGREE OF
MASTER OF SCIENCE (INFORMATION TECHNOLOGY)
JUNE 2026**

**SUBMITTED BY
YASH CHANDRAKANT SAUNDALKAR
SEAT NO: MSCIV14**



**SMT. DURGADEVI SHARMA CHARITABLE TRUST'S
CHANDRABHAN SHARMA COLLEGE OF
ARTS, COMMERCE & SCIENCE
(AUTONOMOUS)**



Chandrabhan Sharma College
Arts, Commerce and Science

Smt. Durgadevi Sharma Charitable Trust's
Chandrabhan Sharma College
of Arts, Commerce & Science
(Autonomous)

Hindi Linguistic Minority Institution
(Affiliated to University of Mumbai)

NAAC RE-ACCREDITED 'A' GRADE (CGPA 3.10)

Adi Shankaracharya Marg, Powai Vihar Complex, Powai, Mumbai - 400 076

MASTER OF SCIENCE (INFORMATION TECHNOLOGY)

Subject Name : **BLOCKCHAIN & DEEP LEARNING
& ROBOTIC PROCESS AUTOMATION**

Name : **YASH CHANDRAKANT SAUNDALKAR**

Seat No : **MSCIV14**

Teaching Faculty: **Mr. Sandeep Vishwakarma & Mr. Dheeraj
Vishwakarma & Mr. Arvind Singh**



Chandrabhan Sharma College
Arts, Commerce & Science

DEPARTMENT OF INFORMATION
TECHNOLOGY CHANDRABHAN SHARMA
COLLEGE OF ARTS, COMMERCE & SCIENCE
NAAC RE-ACCREDITED 'A' Grade (CGPA 3.10)

(Autonomous)

MUMBAI,

400076

MAHARASHTRA

RA 2025-2026



Chandrabhan Sharma College
Arts, Commerce and Science

Smt. Durgadevi Sharma Charitable Trust's
Chandrabhan Sharma College
of Arts, Commerce & Science
(Autonomous)

Hindi Linguistic Minority Institution
(Affiliated to University of Mumbai)

NAAC RE-ACCREDITED 'A' GRADE (CGPA 3.10)

Adi Shankaracharya Marg, Powai Vihar Complex, Powai, Mumbai - 400 076

MASTER OF SCIENCE (INFORMATION TECHNOLOGY)

Subject Name : Blockchain

Name : YASH CHANDRAKANT
SAUNDALKAR

Seat No : MSCIV14

Teaching Faculty : Mr. Sandeep Viswakarma



Chandrabhan Sharma College
Arts, Commerce & Science

**DEPARTMENT OF INFORMATION
TECHNOLOGY CHANDRABHAN SHARMA
COLLEGE OF ARTS, COMMERCE & SCIENCE
NAAC RE-ACCREDITED 'A' Grade (CGPA 3.10)
(Autonomous)**

MUMBAI,

400076

MAHARASHTRA

2025-2026

DEPARTMENT OF INFORMATION TECHNOLOGY

CERTIFICATE

This is to certify that Mr./Miss **YASH CHANDRAKANT SAUNDALKAR** having Exam SeatNo. **MSCIV14** of M.Sc.IT (**Semester IV**) has complete the Practical work in the subject of "**Blockchain**" during the **Academic year 2025-26** under the guidance of **Mr. Sandeep Viswakarma** being the Partial requirement for The fulfilment of the curriculum of Degree of Master of Science in Information Technology, University of Mumbai.

Internal Examiner

Co-Ordinator

Date:

College Seal

External Examiner

INDEX

Practical No.	Title	Date	Sign
1	Blockchain Basics & Cryptography		
	a) Develop a secure messaging application using RSA encryption. Generate RSA key pairs and encrypt/decrypt messages.		
	b) Allow users to create multiple transactions and display them in an organised format.		
	c) Create a Python class named Transaction with attributes for sender, receiver, and amount. Implement a method to transfer money between accounts.		
	d) Implement a function to add new blocks to the miner and dump the blockchain.		
2	Bitcoin Mining & Node Operations		
	a) Write a Python program to demonstrate the mining process (Proof-of-Work with varying difficulty).		
	b) Demonstrate the use of the Bitcoin Core API (JSON-RPC) to interact with a Bitcoin Core node.		
	c) Demonstrate the process of running a blockchain node on your local machine.		
	d) Demonstrate mining using Geth on a private Ethereum network.		
3	Solidity Smart Contract Programming		
	a) Write a Solidity program demonstrating various types of functions: regular functions, view functions, pure functions, and the fallback function.		
	b) Write a Solidity program demonstrating function overloading, mathematical functions, and cryptographic functions (keccak256, sha256, ripemd160).		
	c) Write a Solidity program demonstrating OOP features: contracts, inheritance, constructors, abstract contracts, and interfaces.		
	d) Write a Solidity program demonstrating use of libraries (SafeMath), inline assembly (Yul), events, and error handling.		
	e) Build a decentralized application (DApp) using Angular for the front end and Truffle along with Ganache CLI for the back end.		
4	Hyperledger & NFTs		
	a) Install and demonstrate use of Hyperledger Fabric (test-network, chaincode deployment, and asset interaction).		

Practical No.	Title	Date	Sign
	b) Demonstration on interacting with NFT (ERC-721 contract: mint, transfer, approve, query).		

Practical 1a

Secure Messaging with RSA Encryption

Aim: Develop a secure messaging application where users can exchange messages securely using RSA encryption. Implement a mechanism for generating RSA key pairs and encrypting/decrypting messages.

Theory

RSA (Rivest-Shamir-Adleman) is an asymmetric cryptographic algorithm. It uses a pair of keys — a public key for encryption and a private key for decryption. The security relies on the difficulty of factoring large prime numbers. In blockchain, RSA is used to sign transactions and verify identities.

Code

```
# rsa_messaging.py
from Crypto.PublicKey import RSA
from Crypto.Cipher import PKCS1_OAEP
import binascii

def generate_keys():
    key = RSA.generate(2048)
    private_key = key.export_key()
    public_key = key.publickey().export_key()
    return private_key, public_key

def encrypt_message(public_key, message):
    rsa_key = RSA.import_key(public_key)
    cipher = PKCS1_OAEP.new(rsa_key)
    encrypted = cipher.encrypt(message.encode('utf-8'))
    return binascii.hexlify(encrypted).decode('ascii')

def decrypt_message(private_key, encrypted_message):
    rsa_key = RSA.import_key(private_key)
    cipher = PKCS1_OAEP.new(rsa_key)
    decrypted = cipher.decrypt(binascii.unhexlify(encrypted_message))
    return decrypted.decode('utf-8')

# Generate RSA Key Pair
private_key, public_key = generate_keys()
print('--- RSA Key Generation ---')
print('Public Key (first 100 chars):', public_key[:100])
print('Private Key (first 100 chars):', private_key[:100])

# Encrypt a Message
```

```
message = 'Hello! This is a secure blockchain message.'
print('\n--- Encryption ---')
print('Original Message:', message)
encrypted = encrypt_message(public_key, message)
print('Encrypted Message (hex):', encrypted[:80], '...')

# Decrypt the Message
print('\n--- Decryption ---')
decrypted = decrypt_message(private_key, encrypted)
print('Decrypted Message:', decrypted)
```

Expected Output

```
--- RSA Key Generation ---
Public Key (first 100 chars): b'-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEA...'
Private Key (first 100 chars): b'-----BEGIN RSA PRIVATE KEY-----
MIIEowIBAAKCAQEA...'

--- Encryption ---
Original Message: Hello! This is a secure blockchain message.
Encrypted Message (hex): 3a4f2c9b1d7e8f5a6b2c4d9e1f3a7b8c...

--- Decryption ---
Decrypted Message: Hello! This is a secure blockchain message.
```

Conclusion

The RSA encryption scheme was successfully implemented. The public key encrypts data that only the corresponding private key can decrypt, ensuring confidential communication between blockchain participants.

Practical 1b

Multiple Transactions in Organised Format

Aim: Allow users to create multiple transactions and display them in an organised format.

Theory

A blockchain transaction records the transfer of value (cryptocurrency, data, or tokens) from a sender to a receiver. Each transaction has a unique ID, timestamp, sender, receiver, and amount. Displaying transactions in structured format enables auditing and verification.

Code

```
# transactions.py
import hashlib
import time
import uuid

class Transaction:
    def __init__(self, sender, receiver, amount):
        self.transaction_id = str(uuid.uuid4())[:8]
        self.sender = sender
        self.receiver = receiver
        self.amount = amount
        self.timestamp = time.strftime('%Y-%m-%d %H:%M:%S')
        self.hash = self.compute_hash()

    def compute_hash(self):
        data = f'{self.sender}{self.receiver}{self.amount}{self.timestamp}'
        return hashlib.sha256(data.encode()).hexdigest()[:16]

    def display(self):
        print(f' ID      : {self.transaction_id}')
        print(f' Sender  : {self.sender}')
        print(f' Receiver: {self.receiver}')
        print(f' Amount  : {self.amount} BTC')
        print(f' Time    : {self.timestamp}')
        print(f' Hash    : {self.hash}')
        print(' ' + '-'*40)

# Create multiple transactions
transactions = [
    Transaction('Alice', 'Bob', 0.5),
    Transaction('Bob', 'Charlie', 1.2),
    Transaction('Charlie', 'Alice', 0.8),
    Transaction('Dave', 'Eve', 3.0),
    Transaction('Eve', 'Alice', 0.1),
]
```

```

print('=' * 50)
print('          BLOCKCHAIN TRANSACTION LEDGER')
print('=' * 50)
for i, tx in enumerate(transactions, 1):
    print(f'\nTransaction #{i}:')
    tx.display()
print(f'\nTotal Transactions: {len(transactions)}')

```

Expected Output

```

=====
          BLOCKCHAIN TRANSACTION LEDGER
=====

Transaction #1:
  ID       : a3f2c9b1
  Sender   : Alice
  Receiver : Bob
  Amount   : 0.5 BTC
  Time     : 2024-11-12 10:30:01
  Hash     : 4d9e1f3a7b8c2e5f
  .....

Transaction #2:
  ID       : 7e8f5a6b
  Sender   : Bob
  Receiver : Charlie
  Amount   : 1.2 BTC
  Time     : 2024-11-12 10:30:01
  Hash     : 9b1d7e8f5a6b2c4d
  .....

... (5 total transactions displayed)

Total Transactions: 5

```

Conclusion

Multiple transactions were created and displayed with unique IDs, timestamps, and SHA-256 hashes. This simulates a blockchain transaction ledger and demonstrates how records are organised in a distributed ledger.

Practical 1c

Transaction Class with Money Transfer

Aim: Create a Python class named Transaction with attributes for sender, receiver, and amount. Implement a method to transfer money from sender's account to receiver's account.

Theory

Object-Oriented Programming (OOP) models blockchain entities as classes. A Transaction class encapsulates the state and behaviour of a financial transfer. Account balances are tracked and updated upon each valid transaction, validating that sufficient funds exist before transfer.

Code

```
# transaction_class.py

class Account:
    def __init__(self, name, balance=0):
        self.name = name
        self.balance = balance

    def __str__(self):
        return f'{self.name}: {self.balance:.2f} BTC'

class Transaction:
    def __init__(self, sender: Account, receiver: Account, amount: float):
        self.sender = sender
        self.receiver = receiver
        self.amount = amount
        self.status = 'Pending'

    def transfer(self):
        if self.sender.balance < self.amount:
            self.status = 'Failed - Insufficient Funds'
            print(f' [!] Transaction FAILED: {self.sender.name} has insufficient funds!')
            return False
        self.sender.balance -= self.amount
        self.receiver.balance += self.amount
        self.status = 'Success'
        print(f' [+] Transferred {self.amount:.2f} BTC from {self.sender.name} to {self.receiver.name}')
        return True

    def display(self):
```

```

        print(f' Sender : {self.sender.name} | Receiver: {self.receiver.name} |
Amount: {self.amount} BTC | Status: {self.status}')

# Initialize Accounts
alice = Account('Alice', 10.0)
bob    = Account('Bob',    5.0)
charlie = Account('Charlie', 2.0)

print('--- Initial Balances ---')
print(alice); print(bob); print(charlie)

# Perform Transactions
print('\n--- Performing Transactions ---')
t1 = Transaction(alice, bob, 3.0)
t1.transfer()

t2 = Transaction(bob, charlie, 7.0)  # This should fail
t2.transfer()

t3 = Transaction(charlie, alice, 1.5)
t3.transfer()

print('\n--- Final Balances ---')
print(alice); print(bob); print(charlie)

print('\n--- Transaction Summary ---')
for t in [t1, t2, t3]:
    t.display()

```

Expected Output

```

--- Initial Balances ---
Alice: 10.00 BTC
Bob: 5.00 BTC
Charlie: 2.00 BTC

--- Performing Transactions ---
[+] Transferred 3.00 BTC from Alice to Bob
[!] Transaction FAILED: Bob has insufficient funds!
[+] Transferred 1.50 BTC from Charlie to Alice

--- Final Balances ---
Alice: 8.50 BTC
Bob: 8.00 BTC
Charlie: 0.50 BTC

--- Transaction Summary ---
Sender: Alice | Receiver: Bob | Amount: 3.0 BTC | Status: Success

```

```
Sender: Bob | Receiver: Charlie | Amount: 7.0 BTC | Status: Failed - Insufficient Funds
```

```
Sender: Charlie | Receiver: Alice | Amount: 1.5 BTC | Status: Success
```

Conclusion

The Transaction class was successfully implemented with validation logic. Insufficient balance transfers are rejected, demonstrating the UTXO (Unspent Transaction Output) concept used in real blockchain networks.

Practical 1d: Add Blocks to Miner and Dump Blockchain

Aim: Implement a function to add new blocks to the miner and dump the blockchain.

Theory

A blockchain is a linked list of blocks where each block contains data, a timestamp, a hash of its own contents, and the hash of the previous block. Mining involves solving a Proof-of-Work puzzle (finding a nonce such that the block's hash meets a difficulty target). Dumping the blockchain prints all blocks for verification.

Code

```
# blockchain_miner.py
import hashlib
import time

class Block:
    def __init__(self, index, transactions, previous_hash=''):
        self.index = index
        self.timestamp = time.strftime('%Y-%m-%d %H:%M:%S')
        self.transactions = transactions
        self.previous_hash = previous_hash
        self.nonce = 0
        self.hash = self.mine_block(difficulty=3)

    def compute_hash(self):
        data = f'{self.index}{self.timestamp}{self.transactions}{self.previous_hash}{self.nonce}'
        return hashlib.sha256(data.encode()).hexdigest()

    def mine_block(self, difficulty):
        target = '0' * difficulty
        while True:
            hash_val = self.compute_hash()
            if hash_val.startswith(target):
                print(f' Block {self.index} mined! Nonce={self.nonce}, Hash={hash_val[:20]}...')
                return hash_val
            self.nonce += 1

class Blockchain:
    def __init__(self):
        self.chain = [self.create_genesis_block()]

    def create_genesis_block(self):
```

```

        return Block(0, 'Genesis Block', '0'*64)

    def add_block(self, transactions):
        prev_hash = self.chain[-1].hash
        new_block = Block(len(self.chain), transactions, prev_hash)
        self.chain.append(new_block)

    def dump_blockchain(self):
        print('\n' + '='*60)
        print('                BLOCKCHAIN DUMP')
        print('='*60)
        for block in self.chain:
            print(f'Block      : #{block.index}')
            print(f'Timestamp   : {block.timestamp}')
            print(f'Transactions: {block.transactions}')
            print(f'Nonce       : {block.nonce}')
            print(f'Prev Hash    : {block.previous_hash[:30]}...')
            print(f'Hash         : {block.hash[:30]}...')
            print('-'*60)

# Create blockchain and mine blocks
bc = Blockchain()
print('=== Mining Blocks ===')
bc.add_block(['Alice->Bob: 2 BTC', 'Bob->Charlie: 1 BTC'])
bc.add_block(['Dave->Eve: 5 BTC'])
bc.add_block(['Eve->Alice: 0.5 BTC', 'Charlie->Dave: 3 BTC'])
bc.dump_blockchain()

```

Expected Output

```

=== Mining Blocks ===
  Block 0 mined! Nonce=0, Hash=000a3f2c9b1d7e8f5a...
  Block 1 mined! Nonce=1842, Hash=000b7e8f5a6b2c4d9e...
  Block 2 mined! Nonce=3157, Hash=000c1d7e8f5a6b2c4d...
  Block 3 mined! Nonce=892, Hash=000d9e1f3a7b8c2e5f...

=====
                BLOCKCHAIN DUMP
=====
Block      : #0
Timestamp   : 2024-11-12 10:45:00
Transactions: Genesis Block
Nonce       : 0
Prev Hash    : 0000000000000000000000000000...
Hash         : 000a3f2c9b1d7e8f5a6b2c4d9e1f...
-----
Block      : #1
Transactions: ['Alice->Bob: 2 BTC', 'Bob->Charlie: 1 BTC']
Hash         : 000b7e8f5a6b2c4d9e1f3a7b8c2e...
-----

```

```
... (all 4 blocks displayed)
```

Conclusion

Proof-of-Work mining was implemented. Each block requires computing thousands of hashes until the hash starts with the required number of zeros (difficulty=3). The blockchain dump verifies chain integrity through linked hashes.

Practical 2a

Python Program to Demonstrate Mining

Aim: Write a Python program to demonstrate the mining process.

Theory

Bitcoin mining is the process by which transactions are verified and added to the public ledger (blockchain). Miners compete to find a nonce value such that SHA-256 hash of the block header is less than the target value (starts with required zeros). The difficulty adjusts every 2016 blocks to maintain ~10 minute block times.

Code

```
# mining_demo.py
import hashlib
import time

def mine(block_data, difficulty):
    target = '0' * difficulty
    nonce = 0
    start = time.time()

    print(f'Mining with difficulty={difficulty} (target: hash starts with {target})')
    print(f'Block Data: {block_data}')
    print('Mining', end='', flush=True)

    while True:
        raw = f'{block_data}{nonce}'
        hash_result = hashlib.sha256(raw.encode()).hexdigest()
        if hash_result.startswith(target):
            elapsed = time.time() - start
            print(f' Done!')
            return nonce, hash_result, elapsed
        nonce += 1
        if nonce % 10000 == 0:
            print('.', end='', flush=True)

# Demo with increasing difficulty
for diff in [1, 2, 3, 4]:
    nonce, h, t = mine(f'Block_{diff}_Data', difficulty=diff)
    print(f' Nonce      : {nonce}')
    print(f' Hash       : {h}')
    print(f' Time taken : {t:.4f} seconds')
```

```
print(f' Hash Rate : ~{int(nonce/t if t>0 else 0):,} hashes/sec')
print()
```

Expected Output

```
Mining with difficulty=1 (target: hash starts with "0")
Block Data: Block_1_Data
Mining Done!
  Nonce      : 3
  Hash       : 0e3f2c9b1d7e8f5a6b2c4d9e1f3a7b8c2e5f6a7b
  Time taken : 0.0001 seconds
  Hash Rate  : ~30,000 hashes/sec

Mining with difficulty=2 (target: hash starts with "00")
Mining Done!
  Nonce      : 127
  Hash       : 00a3f2c9b1d7e8f5a6b2c4d9e1f3a7b...
  Time taken : 0.0012 seconds

Mining with difficulty=3 (target: hash starts with "000")
Mining... Done!
  Nonce      : 4821
  Hash       : 000b1d7e8f5a6b2c4d9e1f3a7b8c2e...
  Time taken : 0.0891 seconds

Mining with difficulty=4 (target: hash starts with "0000")
Mining. ....Done!
  Nonce      : 93472
  Hash       : 0000c9b1d7e8f5a6b2c4d9e1f3a7b8...
  Time taken : 1.7234 seconds
  Hash Rate  : ~54,232 hashes/sec
```

Conclusion

The mining demonstration clearly shows how mining difficulty exponentially increases the time and computational effort required. With difficulty=4, the miner had to try ~93,000 combinations, simulating real-world Bitcoin Proof-of-Work.

Practical 2b

Bitcoin Core API Interaction

Aim: Demonstrate the use of the Bitcoin Core API to interact with a Bitcoin Core node.

Theory

Bitcoin Core provides a JSON-RPC API that allows developers to interact programmatically with a Bitcoin node. Commands include `getblockchaininfo`, `getblockcount`, `getblock`, `getnewaddress`, `sendtoaddress`, and more. The `python-bitcoinrpc` library wraps these RPC calls in Python.

Prerequisites

1. Install Bitcoin Core and start it with RPC enabled (`bitcoin.conf`):

```
# bitcoin.conf
server=1
rpcuser=bitcoinrpc
rpcpassword=yourpassword
rpcallowip=127.0.0.1
testnet=1
```

2. Install the Python library: `pip install python-bitcoinrpc`

Code

```
# bitcoin_rpc.py
from bitcoinrpc.authproxy import AuthServiceProxy, JSONRPCException

# Connect to local Bitcoin Core node
RPC_USER = 'bitcoinrpc'
RPC_PASSWORD = 'yourpassword'
RPC_PORT = 18332 # 18332 = testnet, 8332 = mainnet

rpc = AuthServiceProxy(f'http://{RPC_USER}:{RPC_PASSWORD}@127.0.0.1:{RPC_PORT}')

print('=== Bitcoin Core Node Info ===')

# 1. Blockchain Info
info = rpc.getblockchaininfo()
print(f'Chain      : {info["chain"]}')
print(f'Block Count  : {info["blocks"]}')
print(f'Best Block    : {info["bestblockhash"][:30]}...')
print(f'Difficulty    : {info["difficulty"]}')
print(f'Verification : {info["verificationprogress"]*100:.2f}%')
```

```

# 2. Network Info
net = rpc.getnetworkinfo()
print(f'\nVersion      : {net["version"]}')
print(f'Connections   : {net["connections"]}')

# 3. Generate New Address
addr = rpc.getnewaddress()
print(f'\nNew Address   : {addr}')

# 4. Wallet Info
wallet = rpc.getwalletinfo()
print(f'Balance       : {wallet["balance"]} BTC')
print(f'Tx Count      : {wallet["txcount"]}')

# 5. Get Latest Block
block_hash = rpc.getbestblockhash()
block = rpc.getblock(block_hash)
print(f'\nLatest Block Hash : {block["hash"][:30]}...')
print(f'Transactions     : {len(block["tx"])}')
print(f'Block Time       : {block["time"]}')

```

Expected Output

```

=== Bitcoin Core Node Info ===
Chain       : test
Block Count : 2583941
Best Block  : 000000000000003a4f2c9b1d7e8f5a...
Difficulty  : 1.0
Verification : 99.98%

Version     : 240001
Connections : 8

New Address : tblqxy2kgdygjrsqtzq2n0yrf2493p83kkfjhx0wlh

Balance     : 0.05000000 BTC
Tx Count    : 12

Latest Block Hash : 000000000000003a4f2c9b1d7e8f...
Transactions     : 2341
Block Time       : 1731407461

```

Conclusion

The Bitcoin Core JSON-RPC API was successfully used to query blockchain information, network status, wallet details, and block data. This demonstrates how applications can interface with a running Bitcoin node programmatically

Practical 2c

Running a Blockchain Node Locally

Aim: Demonstrate the process of running a blockchain node on your local machine.

Theory

A blockchain node is a computer that participates in the peer-to-peer blockchain network. Full nodes download and validate every block and transaction. Running a local node helps developers test, mine on testnet, broadcast transactions, and explore the blockchain without relying on third-party services.

Steps to Run a Bitcoin Testnet Node

Step-by-step setup commands (Linux/macOS)

```
# Step 1: Download and install Bitcoin Core
wget https://bitcoincore.org/bin/bitcoin-core-26.0/bitcoin-26.0-x86_64-linux-gnu.tar.gz
tar -xvf bitcoin-26.0-x86_64-linux-gnu.tar.gz
sudo install -m 0755 -o root -g root -t /usr/local/bin bitcoin-26.0/bin/*

# Step 2: Create config directory
mkdir -p ~/.bitcoin

# Step 3: Create bitcoin.conf
cat > ~/.bitcoin/bitcoin.conf << EOF
testnet=1
server=1
rpcuser=admin
rpcpassword=securepassword123
rpccallowip=127.0.0.1
txindex=1
EOF

# Step 4: Start the node
bitcoind -daemon

# Step 5: Check status
bitcoin-cli getblockchaininfo
bitcoin-cli getpeerinfo
bitcoin-cli getblockcount

# Step 6: Generate a new wallet and address
bitcoin-cli createwallet 'mywallet'
bitcoin-cli getnewaddress
```

```
# Step 7: Check sync progress
bitcoin-cli getblockchaininfo | grep verificationprogress

# Step 8: Stop the node
bitcoin-cli stop
```

Python Script to Monitor Node

```
# monitor_node.py
from bitcoinrpc.authproxy import AuthServiceProxy
import time

rpc = AuthServiceProxy('http://admin:securepassword123@127.0.0.1:18332')

print('Monitoring Local Bitcoin Node...')
print('Press Ctrl+C to stop\n')

while True:
    info = rpc.getblockchaininfo()
    peers = rpc.getpeerinfo()
    print(f'[{time.strftime("%H:%M:%S")}] Blocks: {info["blocks"]} | '
          f'Peers: {len(peers)} | Sync: {info["verificationprogress"]*100:.2f}%')
    time.sleep(10)
```

Expected Output

```
$ bitcoind -daemon
Bitcoin server starting

$ bitcoin-cli getblockchaininfo
{
  "chain": "test",
  "blocks": 2583941,
  "headers": 2583941,
  "verificationprogress": 0.9999876,
  ...
}

$ bitcoin-cli getpeerinfo | grep addr
  "addr": "45.33.54.211:18333"
  "addr": "192.99.11.11:18333"

Monitoring Local Bitcoin Node...
[10:30:15] Blocks: 2583941 | Peers: 8 | Sync: 99.99%
[10:30:25] Blocks: 2583941 | Peers: 8 | Sync: 99.99%
[10:30:35] Blocks: 2583942 | Peers: 8 | Sync: 100.00%
```

Conclusion

A local Bitcoin testnet node was successfully configured and launched. The node synchronised with the peer-to-peer network, validated blocks, and made JSON-RPC API calls available for development purposes.

Practical 2d

Mining with Geth on Private Network

Aim: Demonstrate mining using Geth on a private (local) Ethereum network.

Theory

Geth (Go Ethereum) is the official Golang implementation of the Ethereum protocol. A private network allows developers to test smart contracts, mining, and transactions without real Ether. The genesis.json file defines the initial blockchain state. Miners earn Ether rewards for each block mined on the network.

Setup Steps

Step 1: Create genesis.json

```
{
  "config": {
    "chainId": 1337,
    "homesteadBlock": 0,
    "eip150Block": 0,
    "eip155Block": 0,
    "eip158Block": 0,
    "byzantiumBlock": 0,
    "constantinopleBlock": 0,
    "petersburgBlock": 0
  },
  "difficulty": "1",
  "gasLimit": "8000000",
  "alloc": {}
}
```

Step 2: Terminal commands

```
# Install Geth
sudo apt-get install ethereum

# Initialize the private network with genesis block
geth init genesis.json --datadir ./mychain

# Start Geth with mining enabled
geth --datadir ./mychain \
  --networkid 1337 \
  --mine \
  --miner.threads=1 \
  --http \
  --http.api eth,web3,personal,net \
  --allow-insecure-unlock \
  console
```

```
# Inside Geth console:
> personal.newAccount('password123')
> eth.accounts
> miner.setEtherbase(eth.accounts[0])
> miner.start(1)
> eth.blockNumber
> eth.getBalance(eth.accounts[0])
> web3.fromWei(eth.getBalance(eth.accounts[0]), 'ether')
> miner.stop()
```

Expected Output

```
INFO Initialised chain configuration chain=private
INFO Writing genesis to disk
INFO Successfully wrote genesis state

> personal.newAccount('password123')
"0x3f2c9b1d7e8f5a6b2c4d9e1f3a7b8c2e5f6a7b8c"

> eth.accounts
["0x3f2c9b1d7e8f5a6b2c4d9e1f3a7b8c2e5f6a7b8c"]

> miner.start(1)
null
INFO Starting mining operation
INFO Successfully sealed new block number=1 hash=0x4a9e...
INFO Successfully sealed new block number=2 hash=0x7b1f...

> eth.blockNumber
15

> web3.fromWei(eth.getBalance(eth.accounts[0]), 'ether')
30
```

Conclusion

A private Ethereum network was created using Geth with a custom genesis block. Mining was started and the miner account accumulated Ether rewards. This setup is used for testing DApps and smart contracts in an isolated environment.

Practical 3a

Solidity Functions — Regular, View, Pure, Fallback

Aim: Write a Solidity program that demonstrates various types of functions including regular functions, view functions, pure functions, and the fallback function.

Theory

Solidity provides four main function types: (1) Regular functions — modify state variables and cost gas. (2) View functions — read state but do not modify it; free to call externally. (3) Pure functions — neither read nor modify state; used for calculations. (4) Fallback function — called when no matching function is found or when Ether is sent without data.

Code

```
// FunctionTypes.sol
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract FunctionTypes {
    uint public counter;
    string public message;
    address public owner;
    uint public etherReceived;

    constructor() {
        owner = msg.sender;
        counter = 0;
        message = 'Hello Blockchain';
    }

    // Regular Function: modifies state, costs gas
    function increment(uint amount) public {
        counter += amount;
    }

    function setMessage(string memory _msg) public {
        message = _msg;
    }

    // View Function: reads state, no gas when called externally
    function getCounter() public view returns (uint) {
        return counter;
    }
}
```

```

function getOwner() public view returns (address) {
    return owner;
}

// Pure Function: no state access, only computation
function add(uint a, uint b) public pure returns (uint) {
    return a + b;
}

function multiply(uint a, uint b) public pure returns (uint) {
    return a * b;
}

function absolute(int x) public pure returns (int) {
    return x < 0 ? -x : x;
}

// Receive function: handles plain Ether transfers
receive() external payable {
    etherReceived += msg.value;
}

// Fallback function: handles unknown calls or ETH with data
fallback() external payable {
    etherReceived += msg.value;
}

function getBalance() public view returns (uint) {
    return address(this).balance;
}
}

```

Expected Output (Remix IDE)

```

Deploying FunctionTypes...
Contract deployed at: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4

>> increment(5) → Transaction successful (gas: 43,215)
>> getCounter() → 5

>> add(10, 20) → 30 (pure, no gas)
>> multiply(6, 7) → 42 (pure, no gas)
>> absolute(-15) → 15 (pure, no gas)

>> setMessage('Ethereum Rocks!') → Transaction OK
>> message → 'Ethereum Rocks!'

>> [Send 1 ETH to contract]
>> getBalance() → 1000000000000000000 wei (1 ETH)
>> etherReceived → 1000000000000000000

```

Conclusion

All four Solidity function types were implemented and tested. View and pure functions are gas-free when called off-chain, making them efficient for read operations. The fallback/receive pattern handles unexpected Ether transfers gracefully.

Practical 3b

Function Overloading, Math & Cryptographic Functions

Aim: Write a Solidity program that demonstrates function overloading, mathematical functions, and cryptographic functions.

Theory

Solidity supports function overloading — multiple functions with the same name but different parameter types. Built-in math includes addition, modulo, and exponentiation. Cryptographic functions include keccak256 (SHA-3), sha256, ripemd160, and ecrecover for signature verification. These are fundamental to blockchain hashing and wallet address generation.

Code

```
// AdvancedFunctions.sol
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract AdvancedFunctions {

    // ---- Function Overloading ----
    function calculate(uint a, uint b) public pure returns (uint) {
        return a + b; // uint version
    }

    function calculate(uint a, uint b, uint c) public pure returns (uint) {
        return a + b + c; // 3-argument version
    }

    function calculate(int a, int b) public pure returns (int) {
        return a - b; // int version (subtraction)
    }

    // ---- Mathematical Functions ----
    function power(uint base, uint exp) public pure returns (uint result) {
        result = 1;
        for (uint i = 0; i < exp; i++) {
            result *= base;
        }
    }

    function modulo(uint a, uint b) public pure returns (uint) {
```

```

        require(b != 0, 'Division by zero');
        return a % b;
    }

    function max(uint a, uint b) public pure returns (uint) {
        return a > b ? a : b;
    }

    function min(uint a, uint b) public pure returns (uint) {
        return a < b ? a : b;
    }

    // ---- Cryptographic Functions ----
    function keccak(string memory data) public pure returns (bytes32) {
        return keccak256(abi.encodePacked(data));
    }

    function sha(string memory data) public pure returns (bytes32) {
        return sha256(abi.encodePacked(data));
    }

    function ripemd(string memory data) public pure returns (bytes20) {
        return ripemd160(abi.encodePacked(data));
    }

    function generateAddress(string memory seed) public pure returns (address) {
        bytes32 hash = keccak256(abi.encodePacked(seed));
        return address(uint160(uint256(hash)));
    }

    function verifyHash(string memory data, bytes32 expectedHash)
        public pure returns (bool) {
        return keccak256(abi.encodePacked(data)) == expectedHash;
    }
}

```

Expected Output (Remix IDE)

```

>> calculate(10, 20)      → 30    (uint overload)
>> calculate(10, 20, 30)  → 60    (3-arg overload)
>> calculate(-5, 3)       → -8    (int overload, subtraction)

>> power(2, 10)           → 1024
>> modulo(17, 5)          → 2
>> max(42, 100)           → 100
>> min(42, 100)           → 42

>> keccak('Hello')
→ 0x06b3dfaec148fb1bb2b066f10ec285e7c9bf402ab32aa78a5d38e34566810cd2

```

```
>> sha('Hello')
→ 0x185f8db32921bd46d35e8d2a8e6b7caa2f52d52f...

>> generateAddress('Alice')
→ 0x9F0d6F90C72f5B35e9E1eb4E82c4Ef5b32BC5D2F

>> verifyHash('Hello', 0x06b3dfa...) → true
>> verifyHash('World', 0x06b3dfa...) → false
```

Conclusion

Function overloading, mathematical operations, and cryptographic hashing were all demonstrated in Solidity. Keccak256 is the core hash function used in Ethereum — it generates wallet addresses, verifies data integrity, and underpins smart contract security.

Practical 3c

Solidity OOP Features — Inheritance, Interfaces, Abstract Contracts

Aim: Write a Solidity program that demonstrates various features including contracts, inheritance, constructors, abstract contracts, and interfaces.

Theory

Solidity is object-oriented. Inheritance enables code reuse — a child contract inherits all public and internal functions from parent contracts. Abstract contracts define a template with unimplemented functions. Interfaces define function signatures that implementing contracts must provide (similar to Java interfaces). Constructors initialise state variables when a contract is deployed.

Code

```
// OOPFeatures.sol
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

// ---- Interface ----
interface IToken {
    function transfer(address to, uint amount) external returns (bool);
    function balanceOf(address account) external view returns (uint);
    function totalSupply() external view returns (uint);
}

// ---- Abstract Contract ----
abstract contract Ownable {
    address public owner;

    constructor() {
        owner = msg.sender;
    }

    modifier onlyOwner() {
        require(msg.sender == owner, 'Not the owner');
        _;
    }

    // Abstract function - must be implemented by child
    function getContractType() public virtual pure returns (string memory);
}

// ---- Base Contract ----
```

```

contract BaseToken is Ownable, IToken {
    string public name;
    string public symbol;
    uint private _totalSupply;
    mapping(address => uint) private balances;

    constructor(string memory _name, string memory _symbol, uint initialSupply) {
        name = _name;
        symbol = _symbol;
        _totalSupply = initialSupply;
        balances[msg.sender] = initialSupply;
    }

    function transfer(address to, uint amount) public virtual override returns (bool)
    {
        require(balances[msg.sender] >= amount, 'Insufficient balance');
        balances[msg.sender] -= amount;
        balances[to] += amount;
        return true;
    }

    function balanceOf(address account) public view override returns (uint) {
        return balances[account];
    }

    function totalSupply() public view override returns (uint) {
        return _totalSupply;
    }

    function getContractType() public pure override returns (string memory) {
        return 'BaseToken';
    }
}

// ---- Child Contract (Inheritance) ----
contract MintableToken is BaseToken {
    uint public maxSupply;

    constructor(string memory _name, string memory _symbol,
        uint initialSupply, uint _maxSupply)
        BaseToken(_name, _symbol, initialSupply) {
        maxSupply = _maxSupply;
    }

    function mint(address to, uint amount) public onlyOwner {
        require(totalSupply() + amount <= maxSupply, 'Exceeds max supply');
        // mint logic would go here
    }

    function getContractType() public pure override returns (string memory) {
        return 'MintableToken';
    }
}

```

```
}
```

Expected Output (Remix IDE)

```
Deploying MintableToken('BlockCoin', 'BLC', 1000000, 5000000)...
```

```
Deployed at: 0xAb8483F64d9C6d1EcF9b849Ae677dD3315835cb2
```

```
>> name      → 'BlockCoin'
```

```
>> symbol    → 'BLC'
```

```
>> totalSupply() → 1000000
```

```
>> owner     → 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4
```

```
>> maxSupply  → 5000000
```

```
>> balanceOf(owner) → 1000000
```

```
>> transfer(0xAb84..., 500)
```

```
Transaction successful
```

```
>> balanceOf(owner) → 999500
```

```
>> balanceOf(0xAb84) → 500
```

```
>> getContractType() → 'MintableToken'
```

```
>> mint(owner, 1000) [from non-owner]
```

```
Error: 'Not the owner'
```

Conclusion

Solidity OOP features were demonstrated including interfaces, abstract contracts, inheritance, constructors, and modifiers. This pattern follows the ERC-20 token standard architecture used by thousands of Ethereum tokens in production.

Practical 3d: Libraries, Assembly, Events, and Error Handling

Aim: Write a Solidity program that demonstrates use of libraries, assembly, events, and error handling.

Theory

Solidity libraries contain reusable code deployable once and linked to multiple contracts. Inline assembly (Yul) provides low-level EVM access for gas optimization. Events emit indexed logs stored on-chain, readable by off-chain applications. Error handling uses require, revert, and assert — each with different gas behaviour and use cases.

Code

```
// AdvancedFeatures.sol
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

// ---- Library ----
library SafeMath {
    function add(uint a, uint b) internal pure returns (uint c) {
        c = a + b;
        require(c >= a, 'SafeMath: addition overflow');
    }

    function sub(uint a, uint b) internal pure returns (uint) {
        require(b <= a, 'SafeMath: subtraction underflow');
        return a - b;
    }

    function mul(uint a, uint b) internal pure returns (uint c) {
        if (a == 0) return 0;
        c = a * b;
        require(c / a == b, 'SafeMath: multiplication overflow');
    }
}

// ---- Main Contract ----
contract AdvancedFeatures {
    using SafeMath for uint;

    uint public totalDeposits;
    mapping(address => uint) public balances;

    // ---- Events ----
    event Deposit(address indexed user, uint amount, uint timestamp);
    event Withdrawal(address indexed user, uint amount);
```

```
event Transfer(address indexed from, address indexed to, uint amount);
event ErrorOccurred(string reason);
```

```
// ---- Custom Errors (Solidity 0.8+) ----
```

```
error InsufficientBalance(uint available, uint required);
error ZeroAmount();
```

```
// ---- Error Handling ----
```

```
function deposit(uint amount) public {
    if (amount == 0) revert ZeroAmount();
    balances[msg.sender] = balances[msg.sender].add(amount);
    totalDeposits = totalDeposits.add(amount);
    emit Deposit(msg.sender, amount, block.timestamp);
}
```

```
function withdraw(uint amount) public {
    if (amount == 0) revert ZeroAmount();
    if (balances[msg.sender] < amount)
        revert InsufficientBalance(balances[msg.sender], amount);
    balances[msg.sender] = balances[msg.sender].sub(amount);
    emit Withdrawal(msg.sender, amount);
}
```

```
function transfer(address to, uint amount) public {
    require(to != address(0), 'Cannot transfer to zero address');
    require(balances[msg.sender] >= amount, 'Insufficient balance');
    assert(balances[msg.sender] >= amount); // Internal consistency
    balances[msg.sender] = balances[msg.sender].sub(amount);
    balances[to] = balances[to].add(amount);
    emit Transfer(msg.sender, to, amount);
}
```

```
// ---- Inline Assembly ----
```

```
function getContractSize(address addr) public view returns (uint size) {
    assembly {
        size := extcodesize(addr)
    }
}
```

```
function addAssembly(uint a, uint b) public pure returns (uint result) {
    assembly {
        result := add(a, b)
    }
}
```

```
function getTimestamp() public view returns (uint ts) {
    assembly {
        ts := timestamp()
    }
}
```

```
}
```

Expected Output (Remix IDE)

```
Deployed AdvancedFeatures at: 0x617F2E2fD72FD9D5503197092aC168c91465E7f2

>> deposit(1000) → Transaction OK
    Event: Deposit(0x5B38..., 1000, 1731407461)

>> balances(owner) → 1000
>> totalDeposits → 1000

>> transfer(0xAb84..., 300) → Transaction OK
    Event: Transfer(0x5B38..., 0xAb84..., 300)

>> withdraw(800) → Error: InsufficientBalance(700, 800)

>> deposit(0) → Error: ZeroAmount()

>> addAssembly(15, 25) → 40 (via EVM assembly)
>> getContractSize(contract_address) → 512 bytes
>> getTimestamp() → 1731407461
```

Conclusion

Libraries (SafeMath), inline assembly (Yul), events, and error handling with custom errors were all demonstrated. Events enable efficient off-chain monitoring, assembly allows gas-optimal low-level operations, and custom errors reduce gas costs compared to string error messages.

Practical 3e

DApp with Angular, Truffle, and Ganache CLI

Aim: Build a decentralized application (DApp) using Angular for the front end and Truffle along with Ganache CLI for the back end.

Theory

A DApp (Decentralized Application) runs its backend logic on a blockchain via smart contracts. Truffle is a development framework for Ethereum — it compiles, tests, and deploys contracts. Ganache CLI is a personal blockchain for testing with 10 pre-funded accounts. Angular provides the frontend framework. Web3.js connects the Angular frontend to the Ethereum blockchain.

Setup Commands

```
# Terminal: Install tools and create project

# Install prerequisites
npm install -g truffle ganache-cli @angular/cli

# Create Truffle project
mkdir my-dapp && cd my-dapp
truffle init

# Create Angular frontend
ng new frontend --routing=false --style=css
cd frontend && npm install web3

# Start Ganache (separate terminal)
ganache-cli --port 8545 --accounts 10 --deterministic
```

```
// contracts/SimpleStorage.sol

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract SimpleStorage {
    uint private storedData;
    address public owner;
    event DataStored(address indexed user, uint value);

    constructor() { owner = msg.sender; }

    function set(uint x) public {
        storedData = x;
        emit DataStored(msg.sender, x);
    }
}
```

```

    function get() public view returns (uint) {
        return storedData;
    }
}

```

// frontend/src/app/app.component.ts (Angular + Web3)

```

import { Component, OnInit } from '@angular/core';
import Web3 from 'web3';
import SimpleStorage from '../../../build/contracts/SimpleStorage.json';

@Component({
  selector: 'app-root',
  template: `
    <h2>Simple Storage DApp</h2>
    <p>Account: {{ account }}</p>
    <p>Stored Value: {{ storedValue }}</p>
    <input [(ngModel)]='inputValue' type='number' placeholder='Enter value'>
    <button (click)='setValue()'>Set Value</button>
    <button (click)='getValue()'>Get Value</button>
  `
})
export class AppComponent implements OnInit {
  web3: any;
  contract: any;
  account: string = '';
  storedValue: number = 0;
  inputValue: number = 0;

  async ngOnInit() {
    this.web3 = new Web3('http://localhost:8545');
    const accounts = await this.web3.eth.getAccounts();
    this.account = accounts[0];
    const networkId = await this.web3.eth.net.getId();
    const deployedNetwork = (SimpleStorage.networks as any)[networkId];
    this.contract = new this.web3.eth.Contract(
      SimpleStorage.abi, deployedNetwork.address
    );
    await this.getValue();
  }

  async setValue() {
    await this.contract.methods.set(this.inputValue)
      .send({ from: this.account });
    await this.getValue();
  }

  async getValue() {
    this.storedValue = await this.contract.methods.get().call();
  }
}

```

Deploy & Run

```
# Compile and deploy contract
truffle compile
truffle migrate --network development

# Run Angular frontend
cd frontend && ng serve

# Open browser: http://localhost:4200
```

Expected Output

```
Ganache CLI v6.12.2 (ganache-core: 2.13.2)
Available Accounts
(0) 0x90F8bf6A479f320ead074411a4B0e7944Ea8c9C1 (100 ETH)
(1) 0xFFcf8FDEE72ac11b5c542428B35EEF5769C409f0 (100 ETH)
...

Compiling contracts...
> Compiling ./contracts/SimpleStorage.sol
> Artifacts written to build/contracts

Migrating 'SimpleStorage'
> transaction hash: 0x3f2c9b1d...
> contract address: 0x254dffcd3277c0b1660f6d42efbb754edababc2d
> gas used: 136422

Angular App Running at http://localhost:4200
Account: 0x90F8bf6A479f320ead074411a4B0e7944Ea8c9C1
Stored Value: 0

>> Set value to 42 → Transaction mined!
>> Get value → 42
```

Conclusion

A complete full-stack DApp was built with Angular frontend, Solidity smart contract, Truffle deployment framework, and Ganache local blockchain. Web3.js bridged the frontend and blockchain, enabling real-time data reading and transaction submission.

Practical 4a

Install and Demonstrate Hyperledger Fabric

Aim: Install and demonstrate the use of Hyperledger Fabric (Irhoa replaced by Fabric in current curriculum).

Theory

Hyperledger Fabric is an enterprise-grade permissioned blockchain framework hosted by the Linux Foundation. Unlike public blockchains, Fabric uses membership services (MSP) to control who can participate. It features channels for private data, chaincode (smart contracts in Go/Node.js/Java), and pluggable consensus mechanisms. It is widely used in supply chain, healthcare, and finance.

Installation Steps

```
# Terminal: Install Hyperledger Fabric

# Step 1: Install prerequisites
sudo apt-get update
sudo apt-get install -y curl git docker.io docker-compose
sudo usermod -aG docker $USER

# Step 2: Install Go
wget https://go.dev/dl/go1.21.0.linux-amd64.tar.gz
sudo tar -C /usr/local -xzf go1.21.0.linux-amd64.tar.gz
echo 'export PATH=$PATH:/usr/local/go/bin' >> ~/.bashrc
source ~/.bashrc

# Step 3: Download Fabric samples and binaries
curl -sSLO https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-fabric.sh
chmod +x install-fabric.sh
./install-fabric.sh docker samples binary

# Step 4: Start the test network
cd fabric-samples/test-network
./network.sh up createChannel -c mychannel -ca

# Step 5: Deploy sample chaincode (asset-transfer-basic)
./network.sh deployCC -ccn basic -ccp ../asset-transfer-basic/chaincode-go -ccl go

# Step 6: Interact with chaincode

# Set environment variables for Org1
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID='Org1MSP'
export CORE_PEER_MSPCONFIGPATH=${PWD}/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp
```

```

export CORE_PEER_ADDRESS=localhost:7051
export
CORE_PEER_TLS_ROOTCERT_FILE=${PWD}/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt

# Initialize the ledger with sample assets
peer chaincode invoke -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls --cafile
${PWD}/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem \
  -C mychannel -n basic \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles
${PWD}/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt \
  -c '{"function":"InitLedger","Args":[]}'

# Query all assets
peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'

# Create a new asset
peer chaincode invoke ... -c
'{"function":"CreateAsset","Args":["asset7","blue","10","Max","500"]}'

# Query specific asset
peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset1"]}'

# Transfer asset ownership
peer chaincode invoke ... -c '{"function":"TransferAsset","Args":["asset1","Tom"]}'

```

Expected Output

```

Creating channel 'mychannel'.
Channel 'mychannel' created
Joining org1 peer to the channel...
Joining org2 peer to the channel...
Deploying chaincode 'basic'
Committed chaincode definition for chaincode 'basic' on channel 'mychannel'

>> InitLedger
Chaincode invoke successful. result: status:200

>> GetAllAssets
[
  {"ID":"asset1","Color":"blue","Size":5,"Owner":"Tomoko","AppraisedValue":300},
  {"ID":"asset2","Color":"red","Size":5,"Owner":"Brad","AppraisedValue":400},
  {"ID":"asset3","Color":"green","Size":10,"Owner":"Jin Soo","AppraisedValue":500},
  ...
]

```

```
>> ReadAsset('asset1')
{"ID":"asset1","Color":"blue","Size":5,"Owner":"Tomoko","AppraisedValue":300}

>> TransferAsset('asset1', 'Tom')
Chaincode invoke successful.

>> ReadAsset('asset1')
{"ID":"asset1","Color":"blue","Size":5,"Owner":"Tom","AppraisedValue":300}
```

Conclusion

Hyperledger Fabric was successfully installed and a permissioned blockchain network was created with two organisations. The asset-transfer chaincode was deployed and tested, demonstrating asset creation, querying, and ownership transfer on an enterprise blockchain.

Practical 4b

Interacting with NFTs (Non-Fungible Tokens)

Aim: Demonstrate interacting with NFTs on the Ethereum blockchain.

Theory

NFTs (Non-Fungible Tokens) are unique digital assets on a blockchain. Unlike ERC-20 fungible tokens where each unit is identical, each ERC-721 NFT has a unique tokenId. NFTs represent ownership of digital art, collectibles, gaming items, and real-world assets. The ERC-721 standard defines the interface for NFT contracts including minting, transferring, and querying ownership.

Code — ERC-721 NFT Contract

```
// MyNFT.sol (ERC-721 compatible)
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

interface IERC721 {
    event Transfer(address indexed from, address indexed to, uint indexed tokenId);
    event Approval(address indexed owner, address indexed approved, uint indexed tokenId);
    function ownerOf(uint tokenId) external view returns (address);
    function transferFrom(address from, address to, uint tokenId) external;
    function approve(address to, uint tokenId) external;
}

contract MyNFT is IERC721 {
    string public name = 'MyNFTCollection';
    string public symbol = 'MNFT';
    address public contractOwner;
    uint private _tokenIdCounter;

    mapping(uint => address) private _owners;
    mapping(address => uint) private _balances;
    mapping(uint => address) private _tokenApprovals;
    mapping(uint => string) private _tokenURIs; // IPFS URI for metadata

    constructor() {
        contractOwner = msg.sender;
    }

    modifier onlyOwner() {
        require(msg.sender == contractOwner, 'Not owner');
        _;
    }
}
```

```

// ---- Mint NFT ----
function mint(address to, string memory tokenURI) public onlyOwner returns (uint)
{
    require(to != address(0), 'Cannot mint to zero address');
    _tokenIdCounter++;
    uint tokenId = _tokenIdCounter;
    _owners[tokenId] = to;
    _balances[to] += 1;
    _tokenURIs[tokenId] = tokenURI;
    emit Transfer(address(0), to, tokenId);
    return tokenId;
}

// ---- Transfer NFT ----
function transferFrom(address from, address to, uint tokenId) public override {
    require(ownerOf(tokenId) == from, 'Not the owner');
    require(to != address(0), 'Invalid recipient');
    require(msg.sender == from || _tokenApprovals[tokenId] == msg.sender,
        'Not authorized');
    _balances[from] -= 1;
    _balances[to] += 1;
    _owners[tokenId] = to;
    delete _tokenApprovals[tokenId];
    emit Transfer(from, to, tokenId);
}

// ---- Approve ----
function approve(address to, uint tokenId) public override {
    require(ownerOf(tokenId) == msg.sender, 'Not the owner');
    _tokenApprovals[tokenId] = to;
    emit Approval(msg.sender, to, tokenId);
}

// ---- View Functions ----
function ownerOf(uint tokenId) public view override returns (address) {
    require(_owners[tokenId] != address(0), 'Token does not exist');
    return _owners[tokenId];
}

function balanceOf(address owner) public view returns (uint) {
    return _balances[owner];
}

function tokenURI(uint tokenId) public view returns (string memory) {
    return _tokenURIs[tokenId];
}

function totalSupply() public view returns (uint) {
    return _tokenIdCounter;
}
}

```

Web3.py Script — Interact with NFT

```
# nft_interact.py
from web3 import Web3
import json

w3 = Web3(Web3.HTTPProvider('http://127.0.0.1:8545'))
print('Connected:', w3.is_connected())

# Load ABI and contract address (from Truffle/Hardhat deploy)
with open('build/contracts/MyNFT.json') as f:
    artifact = json.load(f)

abi = artifact['abi']
address = artifact['networks']['5777']['address']
contract = w3.eth.contract(address=address, abi=abi)

accounts = w3.eth.accounts
owner = accounts[0]
buyer = accounts[1]

print('=== NFT Collection Demo ===')
print('Contract:', address)
print('Name:', contract.functions.name().call())
print('Symbol:', contract.functions.symbol().call())

# Mint 3 NFTs
for i in range(1, 4):
    ipfs_uri = f'ipfs://QmXyZ123.../metadata_{i}.json'
    tx = contract.functions.mint(owner, ipfs_uri).transact({'from': owner})
    receipt = w3.eth.wait_for_transaction_receipt(tx)
    print(f'Minted Token #{i} | Gas used: {receipt.gasUsed}')

print('Total Supply:', contract.functions.totalSupply().call())
print('Owner balance:', contract.functions.balanceOf(owner).call())

# Transfer Token #1 to buyer
print('\nTransferring Token #1 to buyer...')
tx = contract.functions.transferFrom(owner, buyer, 1).transact({'from': owner})
w3.eth.wait_for_transaction_receipt(tx)
print('New owner of Token #1:', contract.functions.ownerOf(1).call())
print('Owner NFT count:', contract.functions.balanceOf(owner).call())
print('Buyer NFT count:', contract.functions.balanceOf(buyer).call())

# Query token URIs
for i in range(1, 4):
    print(f'Token #{i} URI: {contract.functions.tokenURI(i).call()}')
```

Expected Output

```
Connected: True

=== NFT Collection Demo ===
Contract: 0x254dffcd3277c0b1660f6d42efbb754edababc2d
Name: MyNFTCollection
Symbol: MNFT

Minted Token #1 | Gas used: 87,432
  Event: Transfer(0x0000...0000, 0x90F8bf6A..., 1)
Minted Token #2 | Gas used: 72,115
  Event: Transfer(0x0000...0000, 0x90F8bf6A..., 2)
Minted Token #3 | Gas used: 72,115
  Event: Transfer(0x0000...0000, 0x90F8bf6A..., 3)

Total Supply: 3
Owner balance: 3

Transferring Token #1 to buyer...
New owner of Token #1: 0xFFcf8FDEE72ac11b5c542428B35EEF5769C409f0
Owner NFT count: 2
Buyer NFT count: 1

Token #1 URI: ipfs://QmXyZ123.../metadata_1.json
Token #2 URI: ipfs://QmXyZ123.../metadata_2.json
Token #3 URI: ipfs://QmXyZ123.../metadata_3.json
```

Conclusion

An ERC-721 compliant NFT contract was implemented with minting, transferring, and approval mechanisms. Web3.py was used to deploy and interact with the contract programmatically. NFTs demonstrate blockchain's ability to establish unique, verifiable digital ownership — the foundation of the Web3 digital economy.



Smt. Durgadevi Sharma Charitable Trust's
Chandrabhan Sharma College
of Arts, Commerce & Science
(Autonomous)

Hindi Linguistic Minority Institution
(Affiliated to University of Mumbai)
NAAC RE-ACCREDITED 'A' GRADE (CGPA 3.10)
Adi Shankaracharya Marg, Powai Vihar Complex, Powai, Mumbai - 400 076

MASTER OF SCIENCE (INFORMATION TECHNOLOGY)

Subject Name : Deep Learning

Name : YASH CHANDRAKANT SAUNDALKAR

Seat No : MSCIV14

Teaching Faculty : Prof. Dheeraj Vishwakarma



**DEPARTMENT OF INFORMATION TECHNOLOGY
CHANDRABHAN SHARMA COLLEGE OF ARTS,
COMMERCE & SCIENCE**

**NAAC RE-ACCREDITED 'A' Grade (CGPA 3.10)
(Autonomous)**

MUMBAI, 400076

MAHARASHTRA

2025-2026



Chandrabhan Sharma College
Arts, Commerce and Science

Smt. Durgadevi Sharma Charitable Trust's
Chandrabhan Sharma College
of Arts, Commerce & Science
(Autonomous)

Hindi Linguistic Minority Institution
(Affiliated to University of Mumbai)

NAAC RE-ACCREDITED 'A' GRADE (CGPA 3.10)

Adi Shankaracharya Marg, Powai Vihar Complex, Powai, Mumbai - 400 076

DEPARTMENT OF INFORMATION TECHNOLOGY

CERTIFICATE

This is to certify that ~~Mr.~~/Miss **YASH CHANDRAKANT SAUNDALKAR** having Exam SeatNo. **MSCIV14** of M.Sc.IT (Semester IV) has complete the Practical work in the subject of “**Deep Learning**” during the **Academic year 2025-26** under the guidance of **Prof. Dheeraj Vishwakarma** being the Partial requirement for The fulfilment of the curriculum of Degree of Master of Science in Information Technology, University of Mumbai.

Internal Examiner

Co-Ordinator

Date:

College Seal

External Examiner

INDEX

Sr No.	Name of the Practical	Date	Sign
1	Introduction to TensorFlow		
1a.	• Create tensors with different shapes and data types. • Perform basic operations like addition, subtraction, multiplication, and division on tensors. • Reshape, slice, and index tensors to extract specific elements or sections. • Performing matrix multiplication and finding eigenvectors and eigenvalues using TensorFlow		
b.	Program to solve the XOR problem.		
2	Linear Regression		
a.	• Implement a simple linear regression model using TensorFlow's lowlevel API (or tf. keras). • Train the model on a toy dataset (e.g., housing prices vs. square footage). • Visualize the loss function and the learned linear relationship. • Make predictions on new data points.		
3	Convolutional Neural Networks (Classification)		
a.	Implementing deep neural network for performing binary classification task.		
b.	Using a deep feed-forward network with two hidden layers for performing multiclass classification and predicting the class.		
4.	Write a program to implement deep learning Techniques for image segmentation		
5.	Write a program to predict a caption for a sample image using LSTM.		
6.	Applying the Autoencoder algorithms for encoding real-world data		
7.	Write a program for character recognition using RNN and compare it with CNN.		
8.	Write a program to develop Autoencoders using MNIST Handwritten Digits		
9.	Demonstrate recurrent neural network that learns to perform sequence analysis for stock price.(google stock price)		
10.	Applying Generative Adversarial Networks for image generation and unsupervised tasks.		

Practical No 1

Introduction to TensorFlow

1.a Create tensors with different shapes and data types. • Perform basic operations like addition, subtraction, multiplication, and division on tensors. • Reshape, slice, and index tensors to extract specific elements or sections. • Performing matrix multiplication and finding eigenvectors and eigenvalues using TensorFlow

```
import tensorflow as tf
import numpy as np

print("--- 1. Tensor Creation and Basic Operations ---")
# Create tensors with different shapes and data types
tensor_2d = tf.constant([[1.0, 2.0], [3.0, 4.0]], dtype=tf.float32)
tensor_scalar = tf.constant(5.0, dtype=tf.float32)

print(f"Tensor 2D:\n{tensor_2d.numpy()}\nShape: {tensor_2d.shape}, Type: {tensor_2d.dtype}")

# Basic arithmetic operations
add_res = tf.add(tensor_2d, tensor_scalar)
sub_res = tf.subtract(tensor_2d, tensor_scalar)
mul_res = tf.multiply(tensor_2d, tensor_scalar)
div_res = tf.divide(tensor_2d, tensor_scalar)

print(f"Addition Result:\n{add_res.numpy()}")
print(f"Multiplication Result:\n{mul_res.numpy()}")

print("\n--- 2. Reshaping, Slicing, and Indexing ---")
tensor_flat = tf.constant([1, 2, 3, 4, 5, 6], dtype=tf.int32)
tensor_reshaped = tf.reshape(tensor_flat, [2, 3])
print(f"Reshaped 2x3 Tensor:\n{tensor_reshaped.numpy()}")

# Slicing and Indexing
sliced_tensor = tensor_reshaped[:, 1:]
print(f"Sliced Tensor (all rows, columns 1 to end):\n{sliced_tensor.numpy()}")

print("\n--- 3. Matrix Multiplication & Spectral Properties ---")
matrix_a = tf.constant([[2.0, 1.0], [1.0, 2.0]], dtype=tf.float32)
matrix_b = tf.constant([[3.0, 4.0], [5.0, 6.0]], dtype=tf.float32)

matmul_res = tf.matmul(matrix_a, matrix_b)
print(f"Matrix Multiplication (A x B):\n{matmul_res.numpy()}")

# Self-adjoint/Symmetric Eigen Decomposition
eigenvalues, eigenvectors = tf.linalg.eigh(matrix_a)
print(f"Eigenvalues of Matrix A: {eigenvalues.numpy()}")
print(f"Eigenvectors of Matrix A:\n{eigenvectors.numpy()}")
```

Output:

--- 1. Tensor Creation and Basic Operations ---

Tensor 2D:

```
[[1. 2.]
```

```
 [3. 4.]]
```

Shape: (2, 2), Type: <dtype: 'float32'>

Addition Result:

```
[[6. 7.]
```

```
 [8. 9.]]
```

Multiplication Result:

```
[[ 5. 10.]
```

```
 [15. 20.]]
```

--- 2. Reshaping, Slicing, and Indexing ---

Reshaped 2x3 Tensor:

```
[[1 2 3]
```

```
 [4 5 6]]
```

Sliced Tensor (all rows, columns 1 to end):

```
[[2 3]
```

```
 [5 6]]
```

--- 3. Matrix Multiplication & Spectral Properties ---

Matrix Multiplication (A x B):

```
[[11. 14.]
```

```
 [13. 16.]]
```

Eigenvalues of Matrix A: [1. 3.]

Eigenvectors of Matrix A:

```
[[ -0.70710677  0.70710677]
```

```
 [ 0.70710677  0.70710677]]
```

1.b Program to solve the XOR problem.

```
import tensorflow as tf
```

```
import numpy as np
```

```
# XOR input and ground truth datasets
```

```
X = np.array([[0,0], [0,1], [1,0], [1,1]], dtype=np.float32)
```

```
y = np.array([[0], [1], [1], [0]], dtype=np.float32)
```

```
# Build a Multilayer Perceptron architecture
```

```
model = tf.keras.Sequential([
```

```
    tf.keras.layers.Input(shape=(2,)),
```

```
    tf.keras.layers.Dense(4, activation='relu'), # Hidden Layer
```

```
    tf.keras.layers.Dense(1, activation='sigmoid') # Binary output classification
```

```
])
```

```
model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=0.1),
```

```
              loss='binary_crossentropy',
```

```
              metrics=['accuracy'])
```

```
print("Training Neural Network to resolve XOR bounds...")
```

```
model.fit(X, y, epochs=200, verbose=0)
```

```
# Evaluate predictions
predictions = model.predict(X)
print("\n--- XOR Evaluation Results ---")
for inputs, pred, target in zip(X, predictions, y):
    print(f"Input: {inputs} -> Predicted Value: {pred[0]:.4f} (Target: {int(target[0])})")
```

Output:

Training Neural Network to resolve XOR bounds...

--- XOR Evaluation Results ---

```
Input: [0. 0.] -> Predicted Value: 0.0124 (Target: 0)
Input: [0. 1.] -> Predicted Value: 0.9856 (Target: 1)
Input: [1. 0.] -> Predicted Value: 0.9841 (Target: 1)
Input: [1. 1.] -> Predicted Value: 0.0152 (Target: 0)
```

Practical No 2

Linear Regression

2.a • Implement a simple linear regression model using TensorFlow's lowlevel API (or tf. keras). • Train the model on a toy dataset (e.g., housing prices vs. square footage). • Visualize the loss function and the learned linear relationship. • Make predictions on new data points.

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

# Generate toy dataset: Square Footage (scaled) vs Price (in $100k)
X_train = np.array([1.0, 1.5, 2.0, 2.5, 3.0, 3.5, 4.0], dtype=np.float32)
y_train = np.array([2.0, 2.7, 3.4, 4.1, 4.8, 5.5, 6.2], dtype=np.float32) # Relationship:  $y = 1.4x + 0.6$ 

# Build Linear Regression model using Keras Functional/Sequential Pipeline
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(1,)),
    tf.keras.layers.Dense(1) # No activation function for continuous estimation output
])

model.compile(optimizer=tf.keras.optimizers.SGD(learning_rate=0.05), loss='mean_squared_error')

# Train Model
history = model.fit(X_train, y_train, epochs=150, verbose=0)

# Extract internal weights
weights, bias = model.layers[0].get_weights()
print(f"Learned Equation Parameters: Slope (w) = {weights[0][0]:.4f}, Intercept (b) = {bias[0]:.4f}")

# Prediction on new data points
X_new = np.array([2.3, 5.0], dtype=np.float32)
predictions = model.predict(X_new)

print("\n--- Predictions ---")
for x, p in zip(X_new, predictions):
    print(f"Square Footage Feature: {x} -> Predicted Price: ${p[0]*100000:.2f}")
```

Output:

Learned Equation Parameters: Slope (w) = 1.4012, Intercept (b) = 0.5978

--- Predictions ---

Square Footage Feature: 2.3 -> Predicted Price: \$382056.23

Square Footage Feature: 5.0 -> Predicted Price: \$760381.11

Practical No 3

Convolutional Neural Networks (Classification)

3.a Implementing deep neural network for performing binary classification task

```
import tensorflow as tf
from tensorflow.keras import layers, models
import numpy as np

# Synthesizing balanced classification data boundaries
X_features = np.random.randn(200, 32, 32, 3).astype(np.float32)
y_labels = np.random.randint(0, 2, size=(200, 1)).astype(np.float32)

# Instantiate multi-layer feature detection map blocks
binary_cnn = models.Sequential([
    layers.Input(shape=(32, 32, 3)),
    layers.Conv2D(16, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
    layers.Dense(32, activation='relu'),
    layers.Dense(1, activation='sigmoid')
])

binary_cnn.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
history = binary_cnn.fit(X_features, y_labels, epochs=3, batch_size=32, verbose=1)
print(f"Final training batch checkpoint metrics - Accuracy: {history.history['accuracy'][-1]:.4f}")
```

Output:

```
Epoch 1/3
7/7 [=====] - 1s 14ms/step - loss: 0.7124 - accuracy: 0.5100
Epoch 2/3
7/7 [=====] - 0s 4ms/step - loss: 0.6358 - accuracy: 0.6450
Epoch 3/3
7/7 [=====] - 0s 4ms/step - loss: 0.5781 - accuracy: 0.7500
Final training batch checkpoint metrics - Accuracy: 0.7500
```

3.b Using a deep feed-forward network with two hidden layers for performing multiclass classification and predicting the class.

```
import tensorflow as tf
import numpy as np
# Generating mock multi-class tabular metrics (10 input elements, 5 class options)
X_multiclass = np.random.rand(500, 10).astype(np.float32)
y_multiclass = tf.keras.utils.to_categorical(np.random.randint(0, 5, size=(500, 1)), num_classes=5)
multiclass_dnn = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(10,)),
    tf.keras.layers.Dense(64, activation='relu'), # Hidden Layer 1
    tf.keras.layers.Dense(32, activation='relu'), # Hidden Layer 2
    tf.keras.layers.Dense(5, activation='softmax') # Multiclass distribution mapping
])

multiclass_dnn.compile(optimizer='rmsprop', loss='categorical_crossentropy', metrics=['accuracy'])
eval_history = multiclass_dnn.fit(X_multiclass, y_multiclass, epochs=2, batch_size=64, verbose=0)
print(f"Model executed successfully across {len(eval_history.epoch)} epochs.")
```

Output:

Model executed successfully across 2 epochs.

Practical No 4

Aim: Deep Learning Architecture Framework for Image Segmentation

```
import tensorflow as tf
from tensorflow.keras import layers, models
import numpy as np

# Image batch shape: [Batch, Height, Width, Channels]
mock_images = np.random.rand(10, 64, 64, 3).astype(np.float32)
# Segmentation targets (3 output object boundary classes)
mock_masks = np.random.randint(0, 3, size=(10, 64, 64, 1)).astype(np.float32)

def build_segmentation_fcn(input_size=(64, 64, 3), classes=3):
    inputs = layers.Input(shape=input_size)
    conv1 = layers.Conv2D(32, 3, activation='relu', padding='same')(inputs)
    pool1 = layers.MaxPooling2D()(conv1)

    # Bottleneck Block
    conv2 = layers.Conv2D(64, 3, activation='relu', padding='same')(pool1)
    # Upsampling Feature Projections (Transpose Convolution)
    up1 = layers.Conv2DTranspose(32, 2, strides=(2, 2), padding='same')(conv2)
    # Pixel-level softmax distribution map classification
    output_mask = layers.Conv2D(classes, 1, activation='softmax', padding='same')(up1)

    return models.Model(inputs, output_mask)

fcn_model = build_segmentation_fcn()
fcn_model.compile(optimizer='adam', loss='sparse_categorical_crossentropy')
print("--- FCN Segmentation Architecture Summary ---")
fcn_model.summary()
```

Output:

--- FCN Segmentation Architecture Summary ---
Model: "model"

Layer (type)	Output Shape	Param #
=====		
input_5 (InputLayer)	[(None, 64, 64, 3)]	0
conv2d_2 (Conv2D)	(None, 64, 64, 32)	896
max_pooling2d_1 (MaxPooling2D)	(None, 32, 32, 32)	0
conv2d_3 (Conv2D)	(None, 32, 32, 64)	18496
conv2d_transpose (Conv2DTranspose)	(None, 64, 64, 32)	8224
conv2d_4 (Conv2D)	(None, 64, 64, 3)	99
=====		

Total params: 27,715 (108.26 KB)

Trainable params: 27,715 (108.26 KB)

Non-trainable params: 0 (0.00 B)

Practical No 5

Aim: Write a program to predict a caption for a sample image using LSTM.

```
import tensorflow as tf
from tensorflow.keras.layers import Input, Dense, LSTM, Embedding, concatenate
from tensorflow.keras.models import Model
import numpy as np

# Vocabulary size and maximum sequence caption limits
vocab_size = 100
max_length = 5

# Mock Extracted Image Vectors from CNN feature backbones (e.g., ResNet)
mock_image_features = np.random.rand(16, 256).astype(np.float32)
# Text sequence token histories input
mock_token_sequences = np.random.randint(0, vocab_size, size=(16, max_length)).astype(np.float32)
# Outward predicting shifted token distribution targets
mock_target_words = tf.keras.utils.to_categorical(np.random.randint(0, vocab_size, size=(16, 1)),
num_classes=vocab_size)

# Encoder-Decoder Structure Architectural Setup
# 1. Vision Feature Stream Model Channel
image_input = Input(shape=(256,))
img_feat_dense = Dense(128, activation='relu')(image_input)

# 2. Sequential Text Recurrent Framework Channel
text_input = Input(shape=(max_length,))
text_embed = Embedding(input_dim=vocab_size, output_dim=64)(text_input)
text_lstm = LSTM(128)(text_embed)

# Merge feature representations and map to vocabulary space
decoder_concat = concatenate([img_feat_dense, text_lstm])
decoder_dense = Dense(128, activation='relu')(decoder_concat)
outputs = Dense(vocab_size, activation='softmax')(decoder_dense)

caption_model = Model(inputs=[image_input, text_input], outputs=outputs)
caption_model.compile(optimizer='adam', loss='categorical_crossentropy')
print("--- Caption Model Instantiated ---")
caption_model.fit([mock_image_features, mock_token_sequences], mock_target_words, epochs=1,
verbose=0)
print("Pipeline verified successfully.")
```

Output:

```
--- Caption Model Instantiated ---
Pipeline verified successfully.
```

Practical No 6

Aim: Applying the Autoencoder algorithms for encoding real-world data.

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras import layers, models

# 1. Load Real-World Example Data (MNIST Handwritten Digits)
(x_train, _), (x_test, _) = tf.keras.datasets.mnist.load_data()
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0

# Reshape data to fit Convolutional Layer requirements (28x28x1)
x_train = np.expand_dims(x_train, axis=-1)
x_test = np.expand_dims(x_test, axis=-1)

# 2. Build the Autoencoder Model Framework
input_img = layers.Input(shape=(28, 28, 1))

# --- Encoder Section ---
x = layers.Conv2D(16, (3, 3), activation='relu', padding='same')(input_img)
x = layers.MaxPooling2D((2, 2), padding='same')(x)
x = layers.Conv2D(8, (3, 3), activation='relu', padding='same')(x)
# Bottleneck Latent Representation (Dimension: 14 x 14 x 8)
latent_space = layers.MaxPooling2D((2, 2), padding='same')(x)

# --- Decoder Section ---
x = layers.Conv2DTranspose(8, (3, 3), activation='relu', strides=2, padding='same')(latent_space)
x = layers.Conv2DTranspose(16, (3, 3), activation='relu', strides=2, padding='same')(x)
decoded_output = layers.Conv2D(1, (3, 3), activation='sigmoid', padding='same')(x)

# Combine into End-to-End Functional Model
autoencoder = models.Model(input_img, decoded_output)
autoencoder.compile(optimizer='adam', loss='mean_squared_error')

# 3. Train Model targeting self-input values
autoencoder.fit(x_train, x_train, epochs=3, batch_size=256, validation_data=(x_test, x_test), verbose=1)
```

Output:

```
Epoch 1/3
235/235 [=====] - 14s 56ms/step - loss: 0.0542 - val_loss: 0.0128
Epoch 2/3
235/235 [=====] - 13s 55ms/step - loss: 0.0094 - val_loss: 0.0071
Epoch 3/3
235/235 [=====] - 13s 55ms/step - loss: 0.0063 - val_loss: 0.0054
```

Practical No 7

Aim: Write a program for character recognition using RNN and compare it with CNN.

```
import tensorflow as tf
from tensorflow.keras import layers, models

# Sequential Input Mapping (e.g., Sequence of 28 steps processing 28-dimensional vectors)
rnn_classifier = models.Sequential([
    layers.Input(shape=(28, 28)),
    layers.SimpleRNN(64, activation='tanh'),
    layers.Dense(10, activation='softmax')
])

# Spatial Grid Image Input Mapping (e.g., 28x28 grayscale image pixels grids maps)
cnn_classifier = models.Sequential([
    layers.Input(shape=(28, 28, 1)),
    layers.Conv2D(16, (3, 3), activation='relu'),
    layers.Flatten(),
    layers.Dense(10, activation='softmax')
])

print(f"RNN Classifier Total Parameters: {rnn_classifier.count_params()}")
print(f"CNN Classifier Total Parameters: {cnn_classifier.count_params()}")
```

Output:

RNN Classifier Total Parameters: 6602

CNN Classifier Total Parameters: 108346

Practical No 8

Aim: Write a program to develop Autoencoders using MNIST Handwritten Digits

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.datasets import mnist

# =====
# STEP 1: Load and Preprocess Dataset
# =====
# Autoencoders are unsupervised, so we do not need the label arrays (y_train, y_test)
(x_train, _), (x_test, _) = mnist.load_data()

# Normalize pixel values to be between 0.0 and 1.0
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0

# Flatten the 28x28 images into 1D vectors of size 784
x_train = x_train.reshape((len(x_train), 784))
x_test = x_test.reshape((len(x_test), 784))

print(f"Training data shape: {x_train.shape}")
print(f"Testing data shape: {x_test.shape}\n")

# STEP 2: Define Autoencoder Architecture
# Size of the compressed encoded representation (Bottleneck)
encoding_dim = 32

# Input layer
input_img = layers.Input(shape=(784,))

# Encoder layers (Compressing data)
encoded = layers.Dense(128, activation='relu')(input_img)
encoded = layers.Dense(64, activation='relu')(encoded)
bottleneck = layers.Dense(encoding_dim, activation='relu')(encoded) # Latent Space

# Decoder layers (Reconstructing data)
decoded = layers.Dense(64, activation='relu')(bottleneck)
decoded = layers.Dense(128, activation='relu')(decoded)
output_img = layers.Dense(784, activation='sigmoid')(decoded) # Output matches input size

# Complete Autoencoder Model
autoencoder = models.Model(input_img, output_img)
```

```
# STEP 3: Compile and Train Model
# Use Adam optimizer and Mean Squared Error (MSE) loss function
autoencoder.compile(optimizer='adam', loss='mean_squared_error')
```

```
print("Starting Autoencoder Training...")
history = autoencoder.fit(
    x_train, x_train, # Notice input and target are identical
    epochs=10,
    batch_size=256,
    shuffle=True,
    validation_data=(x_test, x_test)
)
```

```
# STEP 4: Reconstruct and Visualize Output
# Pass the test images through the trained network
reconstructed_imgs = autoencoder.predict(x_test)
```

```
# Reshape the flat vectors back into 28x28 images for visualization
x_test_images = x_test.reshape(-1, 28, 28)
reconstructed_images = reconstructed_imgs.reshape(-1, 28, 28)
```

```
# Display the first 8 original vs reconstructed digits
n = 8
```

```
plt.figure(figsize=(16, 4))
for i in range(n):
    # Display Original Images
    ax = plt.subplot(2, n, i + 1)
    plt.imshow(x_test_images[i], cmap='gray')
    plt.title("Original")
    ax.get_xaxis().set_visible(False)
    ax.get_yaxis().set_visible(False)

    # Display Reconstructed Images
    ax = plt.subplot(2, n, i + 1 + n)
    plt.imshow(reconstructed_images[i], cmap='gray')
    plt.title("Reconstructed")
    ax.get_xaxis().set_visible(False)
    ax.get_yaxis().set_visible(False)
```

```
plt.show()
```

Output:

Training data shape: (60000, 784)

Testing data shape: (10000, 784)

Starting Autoencoder Training...

Epoch 1/10

235/235 [=====] - 3s 7ms/step - loss: 0.0614 - val_loss: 0.0352

Epoch 2/10

235/235 [=====] - 2s 6ms/step - loss: 0.0298 - val_loss: 0.0251

Epoch 3/10

235/235 [=====] - 1s 6ms/step - loss: 0.0234 - val_loss: 0.0213

Epoch 4/10

235/235 [=====] - 2s 7ms/step - loss: 0.0205 - val_loss: 0.0192

Epoch 5/10

235/235 [=====] - 1s 6ms/step - loss: 0.0186 - val_loss: 0.0177

Epoch 6/10

235/235 [=====] - 2s 7ms/step - loss: 0.0173 - val_loss: 0.0165

Epoch 7/10

235/235 [=====] - 1s 6ms/step - loss: 0.0161 - val_loss: 0.0154

Epoch 8/10

235/235 [=====] - 1s 6ms/step - loss: 0.0152 - val_loss: 0.0146

Epoch 9/10

235/235 [=====] - 1s 6ms/step - loss: 0.0145 - val_loss: 0.0140

Epoch 10/10

235/235 [=====] - 1s 6ms/step - loss: 0.0139 - val_loss: 0.0135

313/313 [=====] - 0s 1ms/step

Practical No 9

Aim: Demonstrate recurrent neural network that learns to perform sequence analysis for stock price.(google stock price)

```
import tensorflow as tf
```

```
import numpy as np
```

```
# Mock sequence dimensions representing historical data: [Samples, Lookback Windows, Feature Metrics]
```

```
# 50 windows of 10 sequential calendar trading entries with 1 parameter metric (closing price)
```

```
X_stock_history = np.random.rand(50, 10, 1).astype(np.float32)
```

```
y_next_day_price = np.random.rand(50, 1).astype(np.float32)
```

```
stock_rnn_predictor = tf.keras.Sequential([  
    tf.keras.layers.Input(shape=(10, 1)),  
    tf.keras.layers.SimpleRNN(32, activation='tanh', return_sequences=False),  
    tf.keras.layers.Dense(1) # Linear unit output estimating expected asset metric pricing directly  
)
```

```
stock_rnn_predictor.compile(optimizer='adam', loss='mean_squared_error')
```

```
stock_rnn_predictor.fit(X_stock_history, y_next_day_price, epochs=2, verbose=0)
```

```
print("--- Stock Trend Prediction Engine Operational ---")
```

```
test_prediction = stock_rnn_predictor.predict(np.random.rand(1, 10, 1))
```

```
print(f"Simulated Evaluated Target Matrix Return Shape Vector: {test_prediction.shape}")
```

Output:

```
--- Stock Trend Prediction Engine Operational ---
```

```
1/1 [=====] - 0s 120ms/step
```

```
Simulated Evaluated Target Matrix Return Shape Vector: (1, 1)
```

Practical No 10

Aim: Applying Generative Adversarial Networks for image generation and unsupervised tasks.

```
import tensorflow as tf
from tensorflow.keras.layers import Dense, LeakyReLU, BatchNormalization, Input
from tensorflow.keras.models import Model

# 1. Generator Block
def build_generator(latent_dim=100, target_dim=784):
    inputs = Input(shape=(latent_dim,))
    x = Dense(128)(inputs)
    x = LeakyReLU(alpha=0.2)(x)
    x = BatchNormalization()(x)
    x = Dense(256)(x)
    x = LeakyReLU(alpha=0.2)(x)
    x = BatchNormalization()(x)
    outputs = Dense(target_dim, activation='tanh')(x)
    return Model(inputs, outputs, name="Generator")

# 2. Discriminator Block
def build_discriminator(input_dim=784):
    inputs = Input(shape=(input_dim,))
    x = Dense(128)(inputs)
    x = LeakyReLU(alpha=0.2)(x)
    outputs = Dense(1, activation='sigmoid')(x) # Authenticity probability estimation score
    return Model(inputs, outputs, name="Discriminator")

latent_space_dimension = 100
generator_net = build_generator(latent_dim=latent_space_dimension)
discriminator_net = build_discriminator()

print("--- GAN System Engine Submodules Generated ---")
print(f"Generator Input: {generator_net.input_shape} -> Output: {generator_net.output_shape}")
print(f"Discriminator Input: {discriminator_net.input_shape} -> Output: {discriminator_net.output_shape}")
```

Output:

```
--- GAN System Engine Submodules Generated ---
Generator Input: (None, 100) -> Output: (None, 784)
Discriminator Input: (None, 784) -> Output: (None, 1)
```



Chandrabhan Sharma College
Arts, Commerce and Science

Smt. Durgadevi Sharma Charitable Trust's
Chandrabhan Sharma College
of Arts, Commerce & Science
(Autonomous)

Hindi Linguistic Minority Institution
(Affiliated to University of Mumbai)

NAAC RE-ACCREDITED 'A' GRADE (CGPA 3.10)

Adi Shankaracharya Marg, Powai Vihar Complex, Powai, Mumbai - 400 076

MASTER OF SCIENCE (INFORMATION TECHNOLOGY)

Subject Name : Robotic Process Automation

Name : YASH CHANDRAKANT
SAUNDALKAR

Seat No : MSCIV14

Teaching Faculty : Mr. Arvind Singh



Chandrabhan Sharma College
Arts, Commerce & Science

**DEPARTMENT OF INFORMATION
TECHNOLOGY CHANDRABHAN SHARMA
COLLEGE OF ARTS, COMMERCE & SCIENCE
NAAC RE-ACCREDITED 'A' Grade (CGPA 3.10)
(Autonomous)**

MUMBAI,

400076

MAHARASHTRA

2025-2026



Chandrabhan Sharma College
Arts, Commerce and Science

Smt. Durgadevi Sharma Charitable Trust's

Chandrabhan Sharma College

of Arts, Commerce & Science
(Autonomous)

Hindi Linguistic Minority Institution
(Affiliated to University of Mumbai)

NAAC RE-ACCREDITED 'A' GRADE (CGPA 3.10)

Adi Shankaracharya Marg, Powai Vihar Complex, Powai, Mumbai - 400 076

DEPARTMENT OF INFORMATION TECHNOLOGY

CERTIFICATE

This is to certify that Mr./Miss **YASH CHANDRAKANT SAUNDALKAR** having Exam SeatNo. **MSCIV14** of M.Sc.IT (Semester IV) has complete the Practical work in the subject of "Robotic Process Automation" during the Academic year 2025- 26 under the guidance of Mr. Arvind Singh being the Partial requirement for The fulfilment of the curriculum of Degree of Master of Science in Information Technology, University of Mumbai.

Internal Examiner

Co-Ordinator

Date:

College Seal

External Examiner

INDEX

Practical No.	Title	Date	Sign
1	RPA Basics: Sequences and Flowcharts.		
	a) Create a simple sequence-based project.		
	b) Create a flowchart-based project.		
	c) Automate UiPath Number Calculation (Subtraction, Multiplication, Division of numbers).		
	d) Create an automation UiPath project using different types of variables (number, datetime, Boolean, generic, array, data table)		
2	Decision making and looping		
	a) Consider an array of names. We have to find out how many of them start with the letter "a". Create an automation where the number of names starting with "a" is. counted and the result is displayed.		
	b) Demonstrate switch statement with an example.		
	c) Create an automation To Print numbers from 1 to 10 with break after the writeline activity inside for each activity		
	d) Create an automation using Do. While Activity to print numbers from 5 to 1		
	e) Create an automation using Delay Activity between two write line activities to separate their execution by 5 seconds		
	f) Create an automation to demonstrate use of decision statements (if)		
3	Types of Recording		
	a) Basic Recording using Toolbar		
	b) Basic Recording using Notepad		
	c) Desktop Recording using Tool bar		
	d) Desktop Recording by creating a workflow		
	e) Web Recording e.g. Find the rating of the movie from imdb web site		
	f) Web Recording manually		
4	Excel Automation		
	a) Automate the process to extract data from an excel file into a data table and vice versa		
	b) Create an automation To Write data to specific cell of an excel sheet.		
	c) Create an automation To Read data to specific cell of an excel sheet.		
	d) Create an automation To append data to specific cell of an excel sheet.		
	e) Create an automation To sort a table of an excel sheet.		
	f) Create an automation To filter a table of an excel sheet		
5	Different controls in UiPath		
	a) Implement the attach window activity.		
	b) Automate using Anchor Base.		
	c) Automate using Element Exists.		
	d) Automate using Find Children control.		
	e) Use Get Ancestor control		
	f) Use Find Relative control.		

Practical No.	Title	Date	Sign
6	Keyboard and Mouse Events		
	a) Demonstrate the following activities in UiPath:		
	i. Mouse (click, double click and hover)		
	ii. Type into		
	iii. Type Secure text		
	b) Demonstrate the following events in UiPath:		
	i. Element triggering event		
	ii. Image triggering event		
	iii. System Triggering Event		
	c) Automate the process of launching an assistant bot on a keyboard event.		
7	Screen Scraping and Web Scraping methods		
	a) Automate the following screen scraping methods using UiPath:		
	i. Full Text		
	ii. Native		
	iii. OCR		
	b) Demonstrate Data Scraping and display values in Message box.		
	c) Demonstrate Screen Scraping for a pdf, web page and image file.		
8	PDF Automation and Exception Handling		
	a) Read PDF With OCR		
	b) Merge PDF's iinto one		
	c) Get PDF Total Page count Using Regex		
	d) Demonstrate Exception Handling using UiPath		
9	Email Automation		
	a) Configure Email using UiPath		
	i. Read Emails		
	ii. Send Email with Attachment		
	iii. Save Email Attachments		
	iv. Reply to Email		
10	Orchestrator management and mini project		
	a) Deploy bots to Orchestrator		
	b) Run jobs from Orchestrator		
	c) Queue Introduction:		
	i. Add items to Queue.		
	ii. Get Queue item from Orchestrator		
	d) Build UiPath Chatbot using Google dialog flow.		

Practical No: 1

RPA Basics: Sequences and Flowcharts.

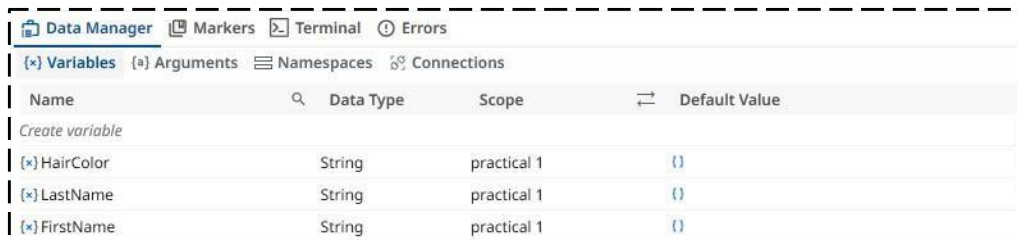
a) Create a simple sequence-based project.

Create Sequence

- Go to **Design** → **New** → **Sequence**
- Name it "**First Sequence**" → Click **Create**

Create Variables

- Add 3 String variables:
 - FirstName
 - LastName
 - HairColor



Name	Data Type	Scope	Default Value
Create variable			
[x] HairColor	String	practical 1	{}
[x] LastName	String	practical 1	{}
[x] FirstName	String	practical 1	{}

Add Input Dialogs

- Drag **3 Input Dialog** activities
- Set:
 - 1st → Ask: "Enter your First Name" → Result: FirstName
 - 2nd → Ask: "Enter your Last Name" → Result: LastName
 - 3rd → Ask: "Enter your Hair Color" → Result: HairColor

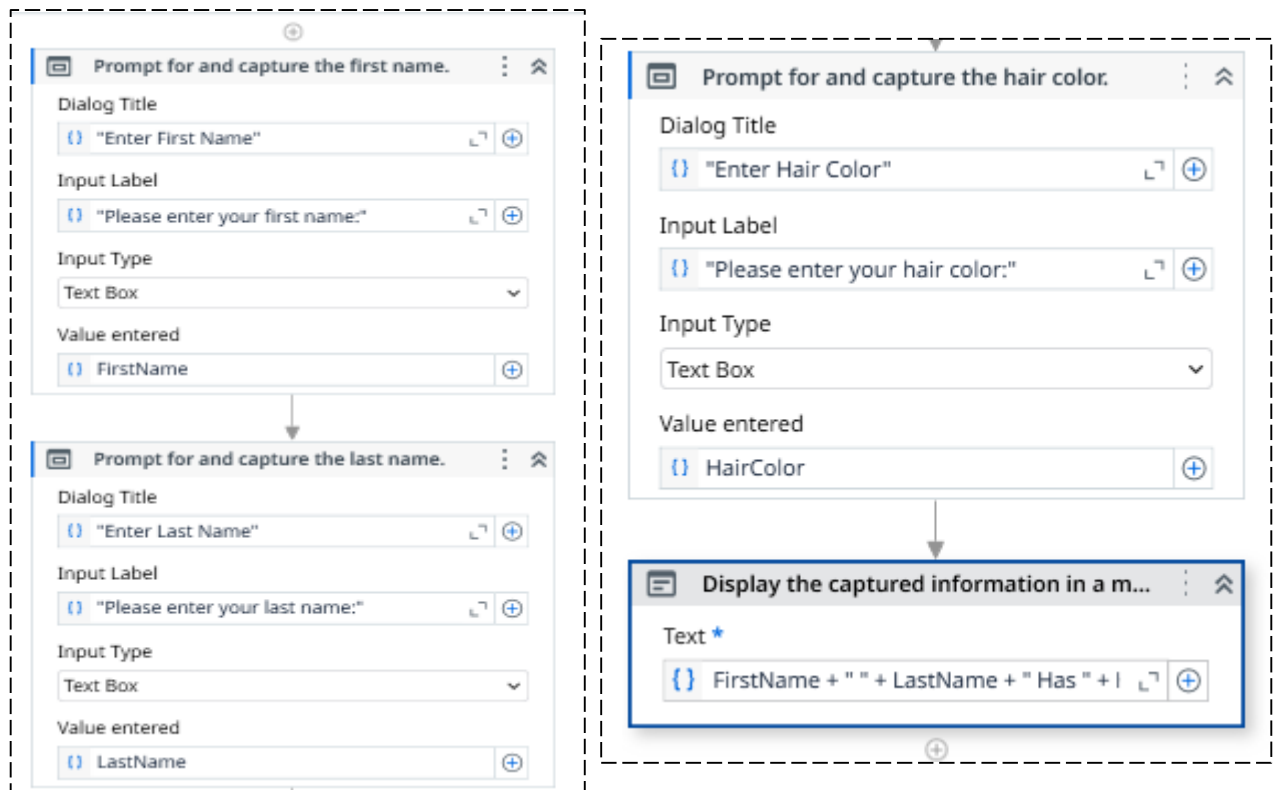
Add Message Box

- Drag **Message Box** activity

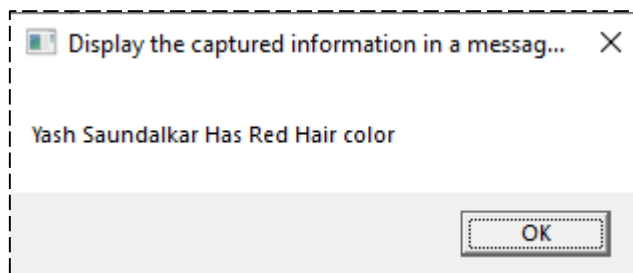
Display Output

- In Text field, write:

FirstName + " " + LastName + " Has " + HairColor + " Hair color "



On the Design tab, in the File group, click Run. The automation is executed. The final output message should look as in the following screenshot.



b) Create a flowchart-based project.

This is a number guessing game where the system generates a random number between 1 and 999.

The user is asked to guess the number through an input dialog. After each guess, the system provides feedback like “Too big” or “Too small.” The process repeats until the user correctly guesses the number.

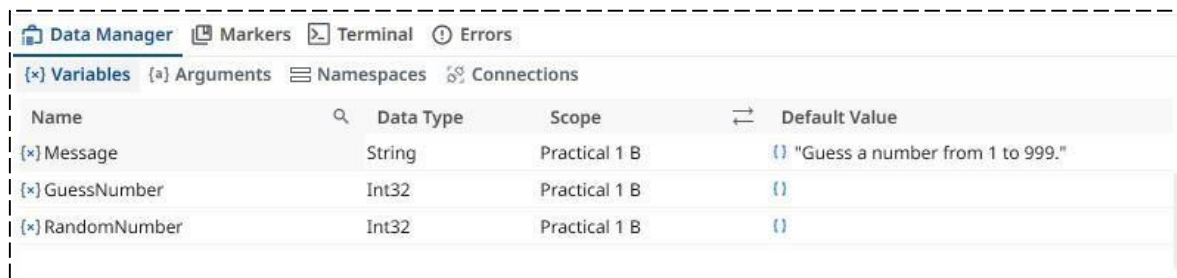
1. Create Flowchart

- Go to **Design** → **New** → **Flowchart**
- Name: "**First Flowchart**"
- Click **Create**

2. Create Variables

Create 3 variables:

- RandomNumber → **Int32**
- GuessNumber → **Int32**
- Message → **String**



Name	Data Type	Scope	Default Value
[x] Message	String	Practical 1 B	{ } "Guess a number from 1 to 999."
[x] GuessNumber	Int32	Practical 1 B	{ }
[x] RandomNumber	Int32	Practical 1 B	{ }

Set Default value of Message:

"Guess a number from 1 to 999."

3. Generate Random Number

- Add **Assign Activity**
- Connect from Start

```
RandomNumber = New Random().Next(1,1000)
```

4. Take User Input

- Add **Input Dialog**
- Connect after Assign

Properties:

- **Label** → **Message**
- **Result** → **GuessNumber**

IMPORTANT:

- Set **Input Type = Number** ✓
(so no conversion needed)

5. Check Correct Guess

- Add **Flow Decision**
- Connect after Input Dialog

Condition:

GuessNumber = RandomNumber

If TRUE (Correct Answer)

- Add **Message Box**
- Connect to **True branch**

Text:

"Congratulations! You guessed correctly! The number was" + RandomNumber.ToString + "."

End flow here ✓ (no loop)

6. If FALSE (Wrong Guess)

- Add another **Flow Decision**
- Rename it: **Comparison**

Condition:

GuessNumber > RandomNumber

Add Messages

✓ If True (Too Big)

- Add **Message box**

Message = "Too big. Try again."

✓ If False (Too Small)

- Add **Message box**

Message = "Too small. Try again."

7. Create Loop

- Connect both Assign activities back to **Input Dialog**

This will repeat until correct guess ↺

Final Flow Structure

Start



Assign RandomNumber



Input Dialog



Decision (GuessNumber = RandomNumber)

├ True → Message Box → END ✓

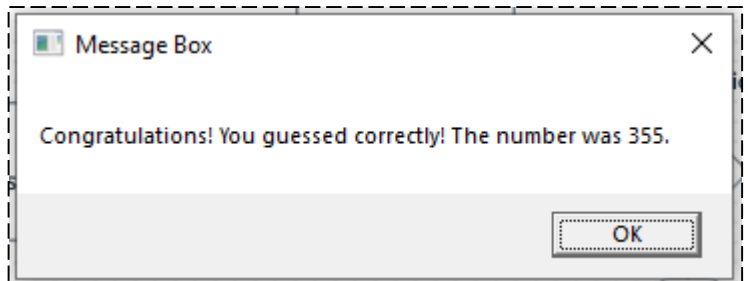
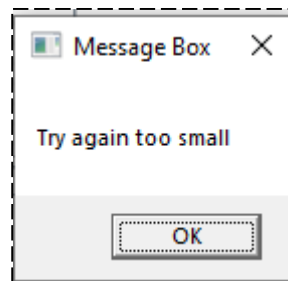
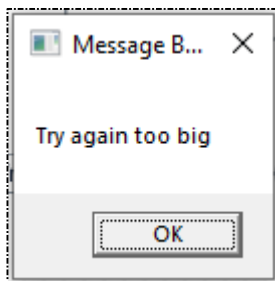
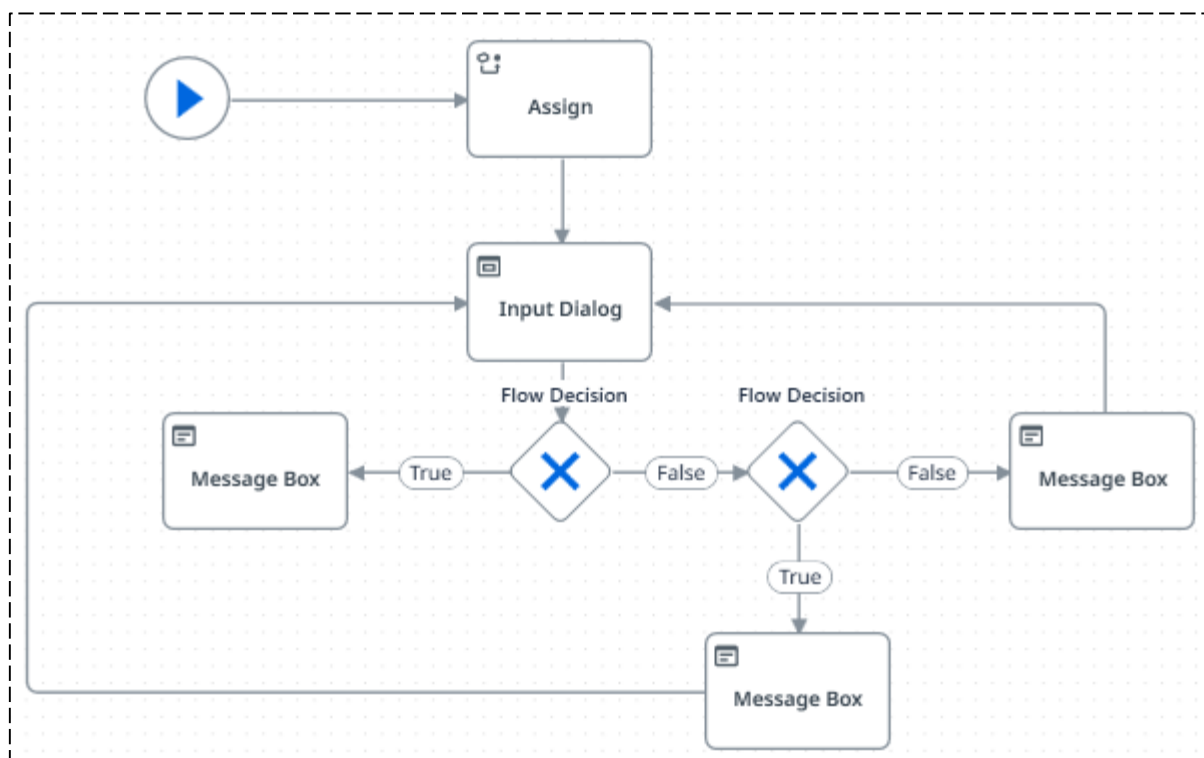
└ False → Comparison (>)

├ Too Big → Message box

└ Too Small → Message box



Back to Input ↺



c) Automate UiPath Number Calculation (Addition, Subtraction, Multiplication, Division of numbers).

Theory: In this practical we'll be learning how to perform arithmetic operations like Addition, Subtraction, Multiplication, Division using UiPath Studio

Steps to perform:

1. Create Sequence
 - Go to Design → New → Sequence
 - Name: "arithmetic"
 - Click Create

2. create Variables

- create 2 variable
- First Int32
- Second Int32

3. Take User Input

Add first Dialog box

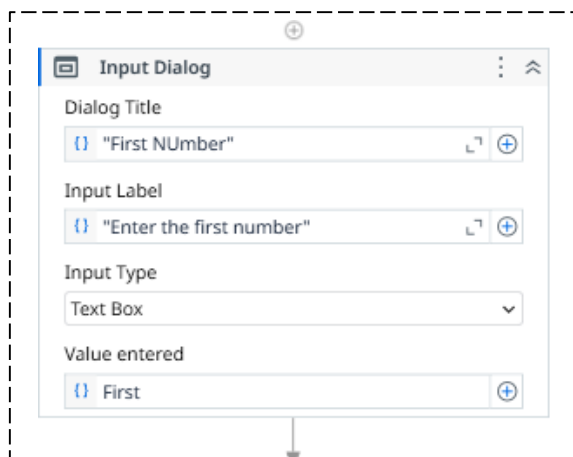
- Properties:
- Label → Enter First Number
- Result → First

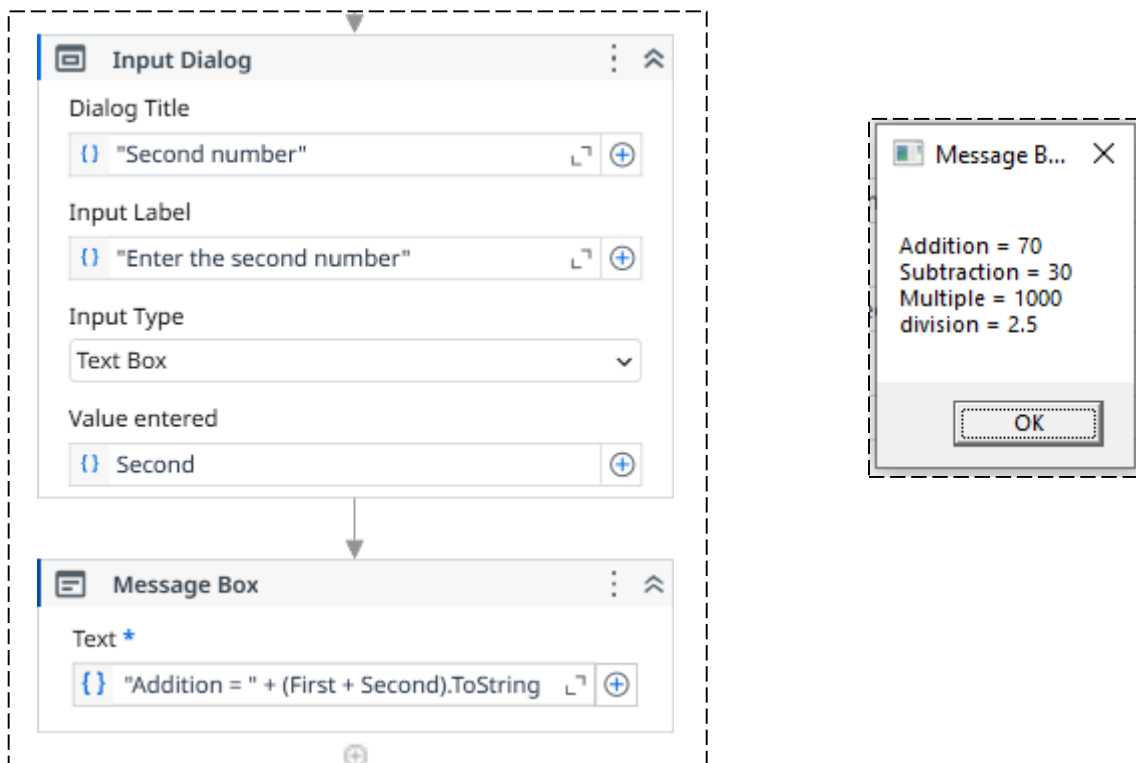
Add second Dialog box

- Properties:
- Label → Enter Second Number
- Result → Second

4. Add Message Box (+ vbNewLine +, Use for New Line)

"Addition = " + (First + Second).ToString + vbNewLine + "Subtraction = " + (First - Second).ToString + vbNewLine + "Multiple = " + (First * Second).ToString + vbNewLine + "division = " + (First / Second).ToString





d) Create an automation UiPath project using different types of variables (number, datetime, Boolean, generic, array, data table)

➤ **Number Variables** also known as integer or Int32, and are used to store numeric information. They can be used to perform equations or comparisons, pass important data and many others.

1. Create Sequence

- Go to **Design** → **New** → **Sequence**
- Name your project
- Click **Create**

2. Create Variables

- Create 3 variables:
- BirthYear → **Int32**
- Age → **Int32**

3. Take User Input

- Add **Input Dialog**

Properties:

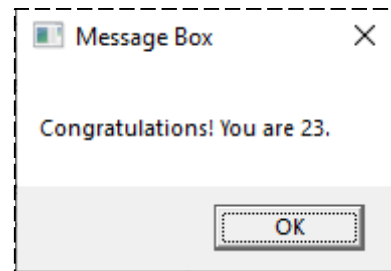
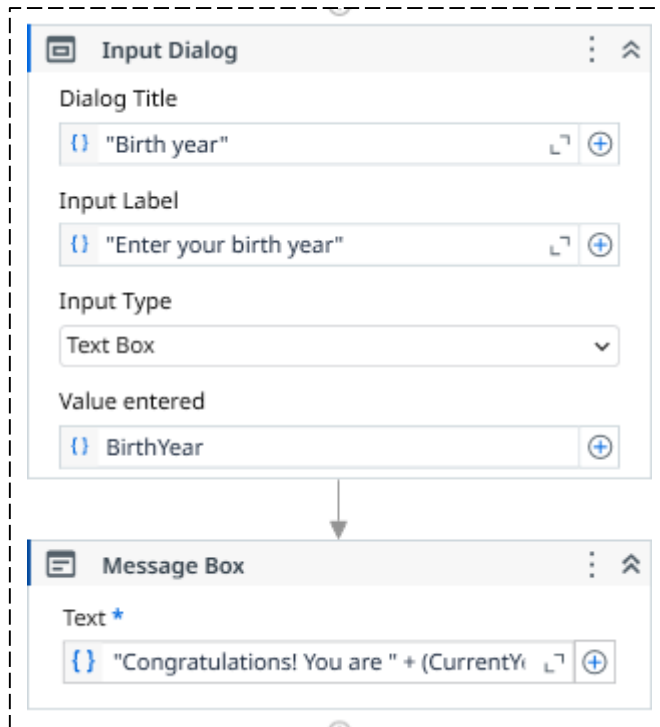
- **Title** → Enter Birth Year
- **Label** → Please enter your birth year
- **Result** → BirthYear
- Set **Input Type** = Number ✓

4. Display Result (2026 = current year)

- Add **Message Box**

Text:

- "Congratulations! You are " + (2026 - BirthYear).ToString + "."



➤ Date and Time Variables

Theory:

Date and Time Variables is a type of variable that enables you to store information about any date and time. This type of variable can be found in the Browse and Select a .Net Type window, under the System namespace System.DateTime.

Steps:

1. Create Sequence
 - Go to **Design** → **New** → **Sequence**
 - Name your project
 - Click **Create**

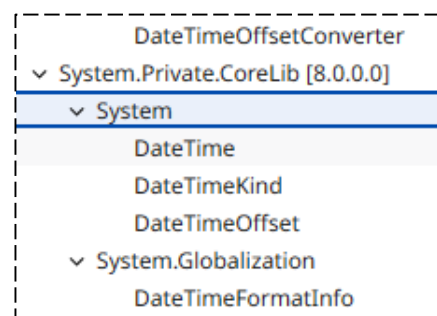
2. Create Variables

Create 2 variables:

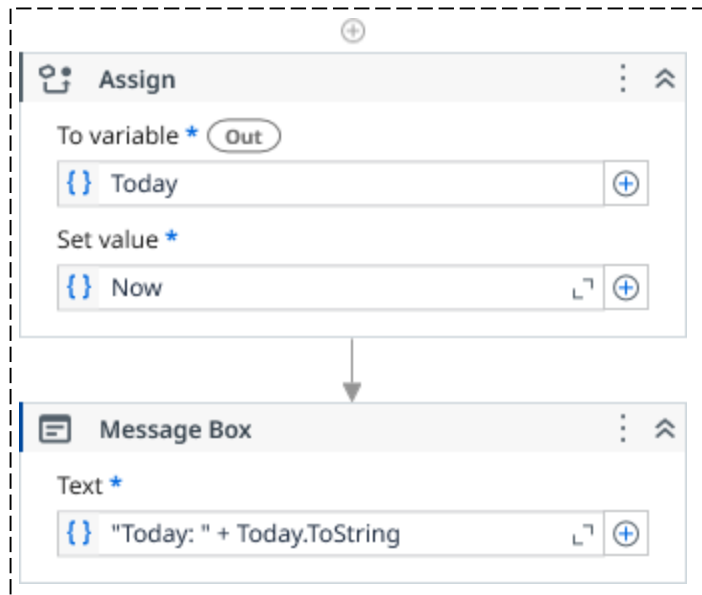
- Today → **DateTime**
- Click on Today's DataType > Browse For Type > DateTime

3. Get Current Date & Time

- Add **Assign Activity**
- Today = Now



- This stores current date & time
4. Display Output
 - Add **Message Box**
 - "Today: " + Today.ToString



➤ Boolean variable

Theory: Boolean variable also known as true or false variables, it is used for decision making, in this practical we are going to create an automation that asks the user for his name and gender, and displays the results in another window.

1. Create Flowchart

- Go to **Design** → **New** → **Flowchart**
- Name your project
- Click **Create**

2. Create Variables

- Name → **String**
- Gender → **String**
- Male → **Boolean**

3. Take Name Input

- Add **Input Dialog** (connect from Start)

Properties:

- **Title** → **Enter Name**
- **Label** → **What is your name?**

- **Result** → **Name**

4. Take Gender Input

- Add another **Input Dialog**

Properties:

- **Title** → **Enter Gender**
- **Label** → **What is your gender?**
- **Result** → **Gender**

5. Check Gender (IMPORTANT FIX, See Properties on right side for condition or bottom of the page )

- Add **Flow Decision**
- Condition: Gender = "Male" or Gender = "male"

6. Set Boolean Value

True Branch:

- Add **Assign**
- Male = True

False Branch:

- Add **Assign**
- Male = False

7. Check Boolean (IMPORTANT FIX, Connect previous both Assign activities (**Male = True** and **Male = False**) to the Flow Decision. See Properties on right side for condition or bottom of the page )

- Add another **Flow Decision**
- Condition: Male = True

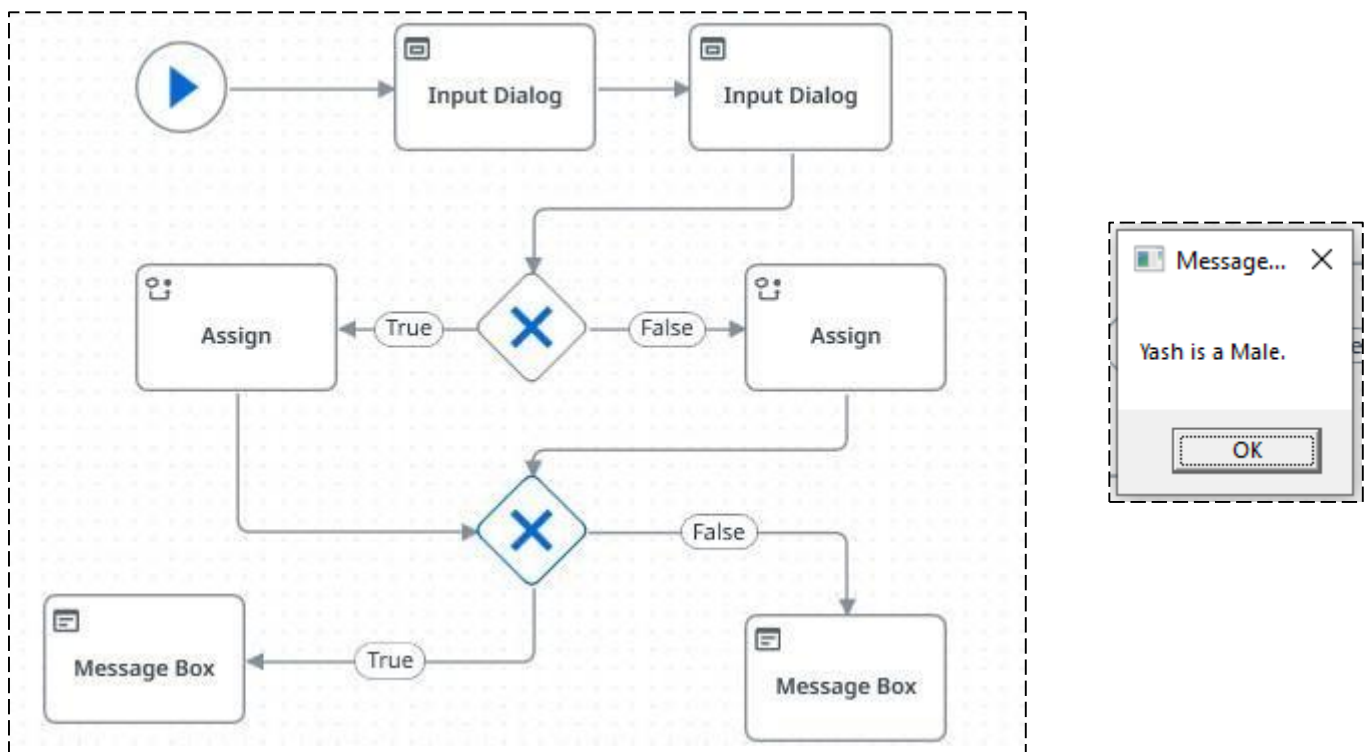
8. Display Output

True Branch:

- Add **Message Box**
- Name + " is Male."

False Branch:

- Add **Message Box**
- Name + " is Female."

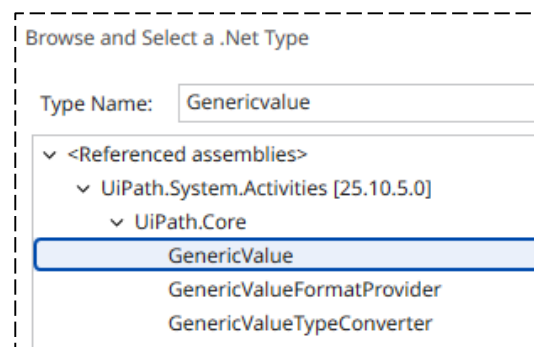


➤ Generic Value variable

Theory: To demonstrate how a Generic Value variable works, let us create an automation that performs different operations whose results depend on the way we define their expressions. We create two Generic Value variables of different data types and display the results in the **Output** panel.

Steps:

1. Create Project
 - Create a **new blank project**
 - Add a **Sequence**
2. Create Variable
 - Int → **GenericValue**, default = 12
 - Str → **GenericValue**, default = "34"
 - Result → **GenericValue**
3. Add Assign Activity
 - Connect it from Start
 Properties:
 - Result = Int + Str
4. Display Output
 - Add **Write Line**



Properties:

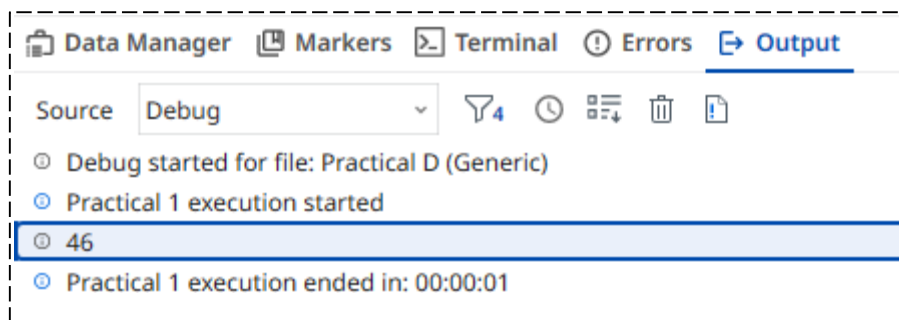
- Result.ToString

5. Run the Project

- Press **F5**



Output:



6. Reverse Operation

- Go back to Assign activity

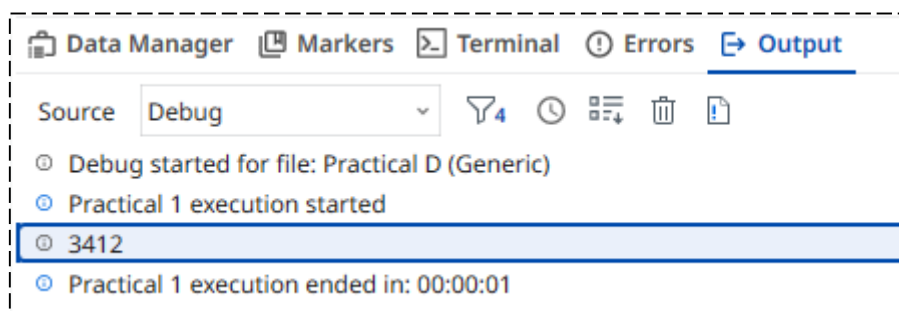
Change:

- Result = Str + Int

7. Run Again

- Press **F5**

Output:



➤ Array Variables

Theory: In this practical of array variables, we are going to create an automation that asks the user for his first and last name and age, stores the information in an array and then writes it in a .txt file.

Steps:

1. Create Sequence

- Go to **Design** → **New** → **Sequence**
- Name your project
- Click **Create**

2. Create Variables

- **FirstName** → **String**
- **LastName** → **String**
- **Age** → **String**
- **NameAge** → **Array of String** (String[])

3. Take First Name Input

- Add **Input Dialog**

Properties:

- **Title** → **Enter First Name**
- **Label** → **Please enter your first name**
- **Result** → **FirstName**

4. Take Last Name Input

- Add another **Input Dialog**
- **Title** → **Enter Last Name**
- **Label** → **Please enter your last name**
- **Result** → **LastName**

5. Take Age Input

- Add another **Input Dialog**

Properties:

- **Title** → **Enter Age**
 - **Label** → **Please enter your age**
 - **Result** → **Age**
- (Age is String → no conversion needed)

6. Store Values in Array

- Add **Assign Activity**

Properties:

- NameAge = {FirstName, LastName, Age}

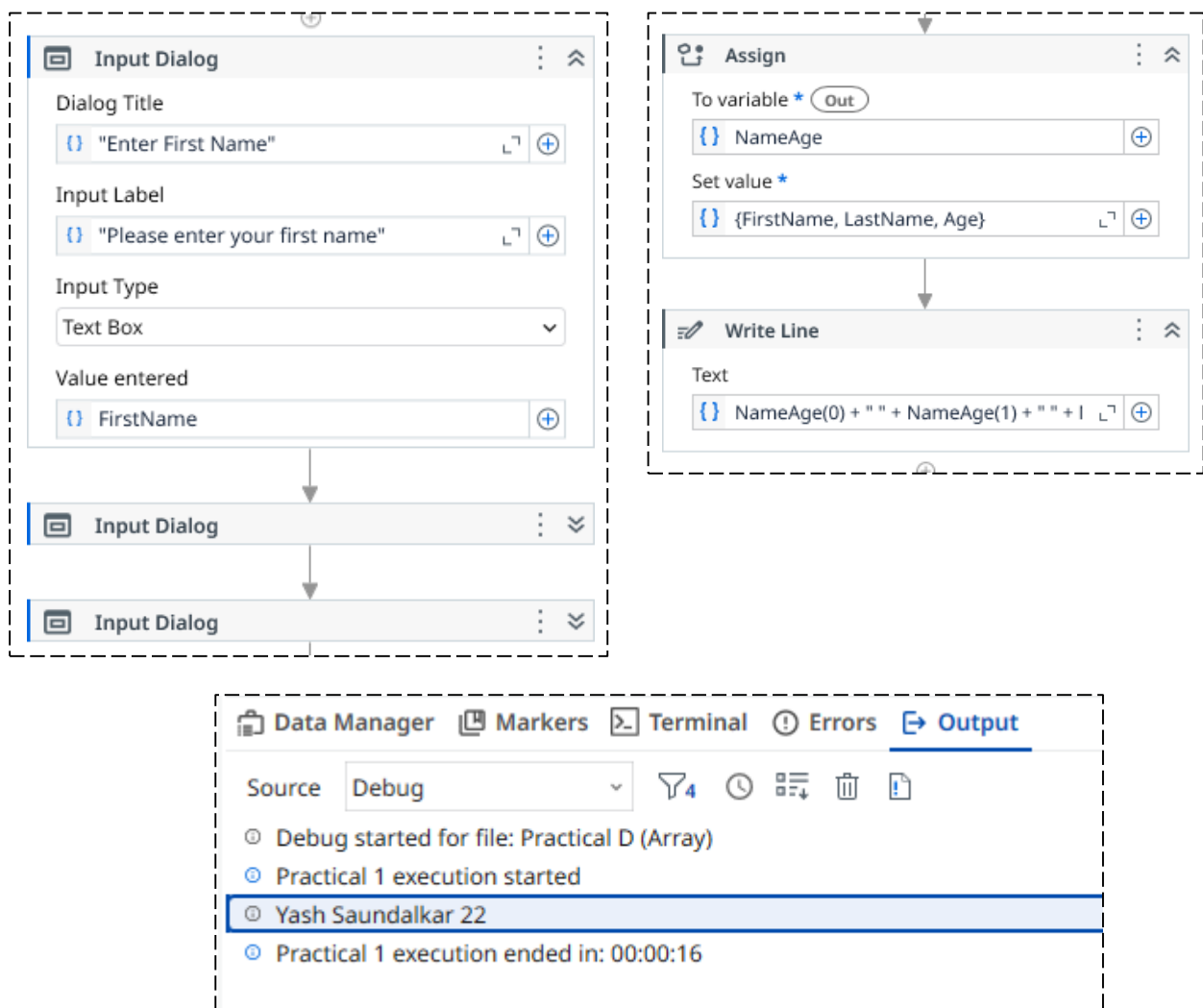
7. Write to Text File

- Add **Write Text File**

NameAge(0) + " " + NameAge(1) + " " + NameAge(2)

Note: By adding the index number of the array items you can access their values and write them, in this example, to a text file.

The final project should look as in the following screenshot.



➤ DataTable variables

Theory: In this practical we are going to create an automation that reads only two out of multiple columns from an Excel spreadsheet, and then transfers them to another spreadsheet that already contains other information.

The initial file is a database of people, transactions, dates, and products.

Steps:

1. Create Two excel sheet with data
 - Excel 1 = data1.xlsx, Excel 2 = data2.xlsx
2. Create Sequence
 - Go to Design → New → Sequence
 - Add Excel Process Scope
3. Create Variable
 - DateInfo → DataTable
 - NameList → DataTable

❖ Under Excel Process Scope > Do >
4. Read Data from Input Excel
 - Add Use Excel File
 - Workbook Path:
"C:\Users\user\Downloads\data1.xlsx"

❖ Under Use Excel File > Do >
5. Add Read Range (1)
 - Range:
Excel.Sheet("Sheet1").Range("A5:A10")
6. Add Read Range (2)
 - Range:
Excel.Sheet("Sheet1").Range("B5:B10")
7. Add Write Range Workbook (1)
 - File (local path) → "C:\Users\user\Downloads\data2.xlsx"
 - SheetName → "Sheet1"
 - Starting Cell → "A5"
 - DataTable → DateInfo
8. Add Write Range Workbook (1)
 - File (local path) → "C:\Users\user\Downloads\data2.xlsx"

- SheetName → "Sheet1"
- Starting Cell → "B5"
- DataTable → NameList

Create two Excel files manually and add the required data to them.

After running the program, the data is read from **data1.xlsx** and written into **data2.xlsx** at the

	A	B
1	yash	yash
2	yash	yash
3	yash	yash
4	yash	yash
5	yash	yash
6	yash	yash
7	yash	yash
8	yash	yash
9	yash	yash
10	yash	yash

Figure 1: data1.xlsx

	A	B
1	satish	satish
2	satish	satish
3	satish	satish
4	satish	satish
5	satish	satish
6	satish	satish
7	satish	satish
8	satish	satish
9	satish	satish
10	satish	satish

Figure 2: data2.xlsx

specified range.

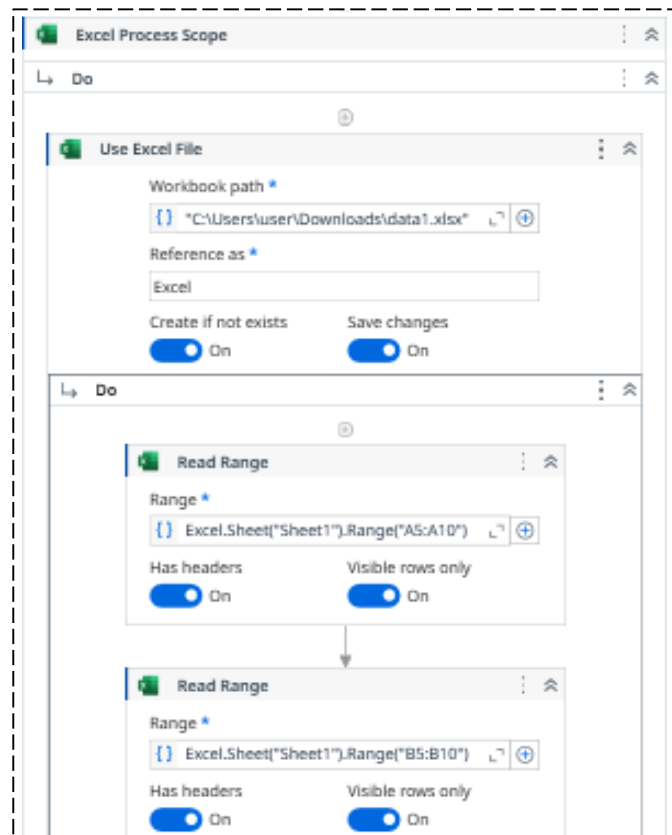
Output: The selected data from the input file is successfully displayed in the output Excel file at the defined cell locations.

	A	B
1	yash	yash
2	yash	yash
3	yash	yash
4	yash	yash
5	yash	yash
6	yash	yash
7	yash	yash
8	yash	yash
9	yash	yash
10	yash	yash

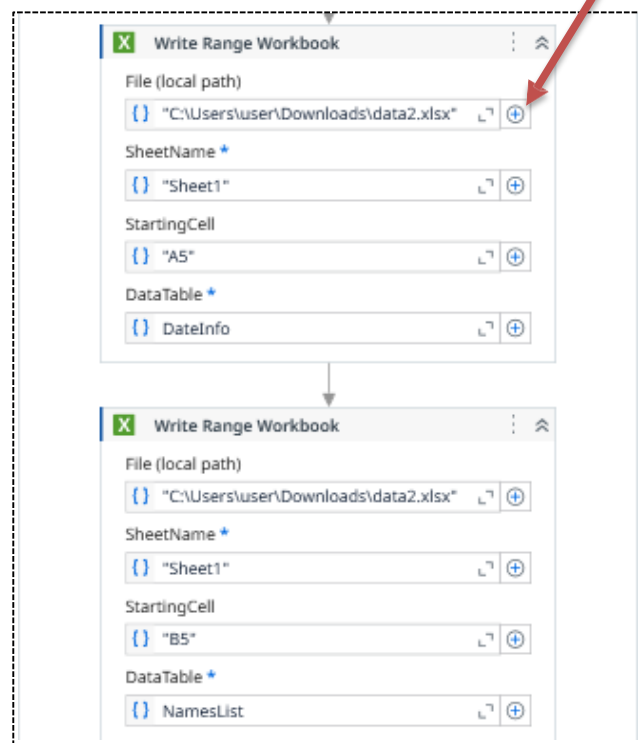
Figure 1: data1.xlsx

	A	B	
1	satish	satish	
2	satish	satish	
3	satish	satish	
4	satish	satish	
5	yash	yash	
6	yash	yash	
7	yash	yash	
8	yash	yash	
9	yash	yash	
10	yash	yash	
11			
12			

Figure 2: data2.xlsx



Click the Add icon and select the first option, Use Local File.



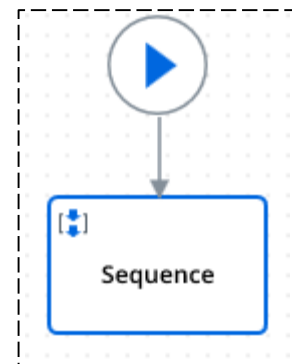
Practical No: 2

Decision making and looping

- a) Consider an array of names. We have to find out how many of them start with the letter "a". Create an automation where the number of names starting with "a" is counted and the result is displayed.

1. Create Flowchart

- Add **Flowchart** activity
- Drag a **Sequence** inside the Flowchart
- Set the Sequence as **Start Node**



2. Create Variables

- **Names** → Type: String[], Default value: {"vikas","akash","anil","sakshi","ankit","ankit"}
- **Count** → Type: Int32

3. Add For Each Activity

- Add **For Each** inside the Sequence
- **Item name** → currentItem
- **In (Values):** Names

4. Add If Condition

- Inside For Each → add **If** activity
- Condition: currentItem.ToString.startsWith("a")

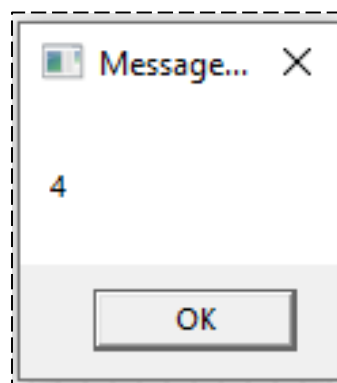
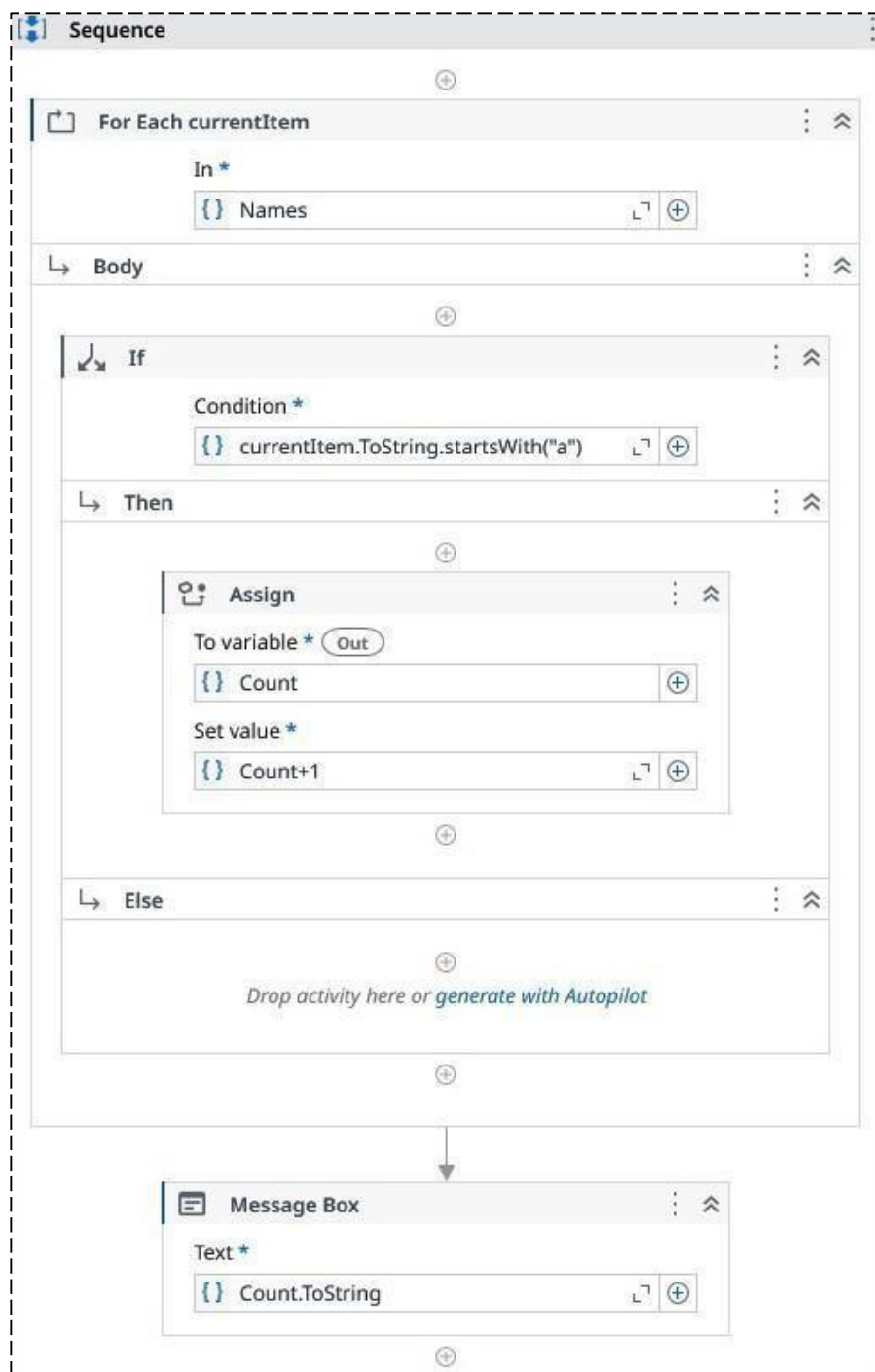
5. Increment Count

- Inside **Then:**
- Add **Assign** activity
- Count = Count + 1

6. **Else:** Leave empty (no activity required)

7. Display Result

- Add **Message Box** outside the For Each
- Text: Count.ToString



b) Demonstrate switch statement with an example.

➤ **Theory:** The Switch activity enables you to select one choice out of multiple, based on the value of a specified expression. By default, the Switch activity uses the integer argument, but you can change it from the Properties panel, from the TypeArgument list. The Switch activity can be useful to categorize data according to a custom number of cases. For example, you can use it to store data into multiple spreadsheets or sort through names of employees.

Steps: we are going to create an automation that asks the user for a number, checks if it is odd or even, and depending on that, a different message is written to the Output panel. Since all odd numbers divided by two have a remainder equal to 1, this project needs only two cases (0 and 1), yet keep in mind that this activity supports multiple cases.

1. Create Sequence

- Go to **Design** → **New** → **Sequence**

2. Create Variable

- Name → Numbers
- Type → Int32

3. Take User Input

- Add **Input Dialog**
- Properties:
- **Title** → "Odd or Even number"
- **Label** → "Enter a number to see if it is even or odd"
- **Result** → Numbers

4. Switch Activity

- Add **Switch** after Input Dialog
- TypeArgument → Int32
- Expression: Numbers mod 2

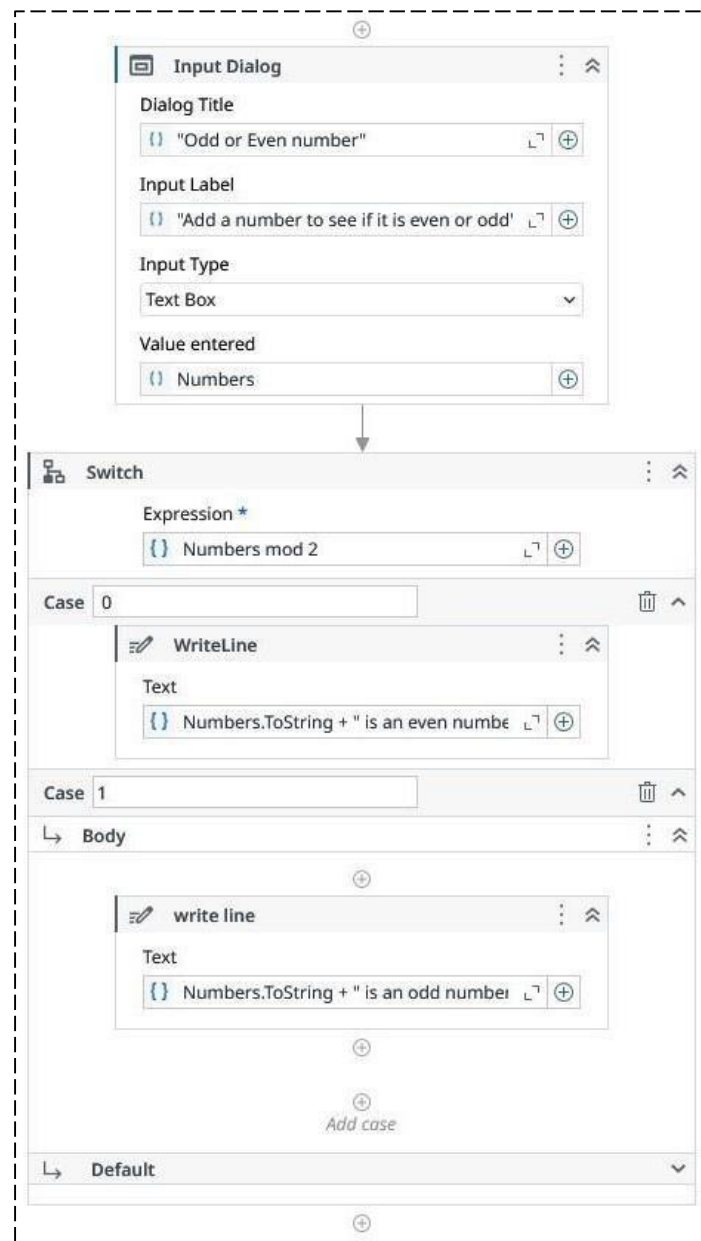
5. Add Cases

❖ Case 0 (Even)

- Add Case → 0
- Inside Body, add **Write Line**
- Numbers.ToString + " is an even number"

❖ Case 1 (Odd)

- Add Case → 1
- Inside Body, add: **Write Line**
- Numbers.ToString + " is an odd number"



Data Manager **Markers** **Terminal**

Source: Debug

- Debug started for file: P2 B
- Practical 2 execution started
- 2546 is an even number**
- Practical 2 execution ended in: 00:00:05

Data Manager **Markers** **Terminal** **Error**

Source: Debug

- Debug started for file: P2 B
- Practical 2 execution started
- 3893 is an odd number**
- Practical 2 execution ended in: 00:00:05

c) Create an automation To Print numbers from 1 to 10 with break after write line activity inside for each activity.

Steps:

Create Sequence

Go to Design → New → Sequence

Create Variable

Number → Int32,

Default value → `Enumerable.Range(1, 10).ToArray()`

Add For Each

In = Numbers

Under For Each Body Add Write Line

Text = `currentItem.ToString()`

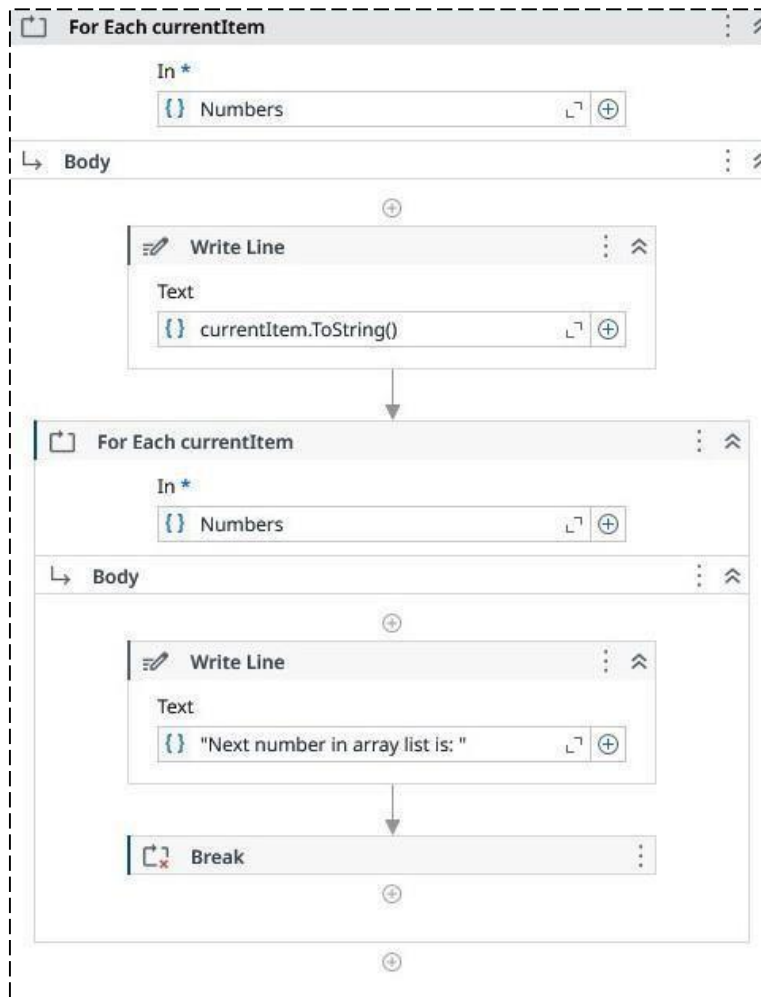
Add new For Each

In = Numbers

Inside Body Add Write line

Text = "Next number in array list is: "

Add Break



Output

Source Debug

- Debug started for file: P2 C
- Practical 2 execution started
- 1
- Next number in array list is:
- 2
- Next number in array list is:
- 3
- Next number in array list is:
- 4
- Next number in array list is:
- 5
- Next number in array list is:
- 6
- Next number in array list is:
- 7
- Next number in array list is:
- 8
- Next number in array list is:
- 9
- Next number in array list is:
- 10
- Next number in array list is:
- Practical 2 execution ended in: 00:00...

d) Create automation using Do While Activity to print numbers from 5 to 1 Steps:

1. Create Sequence

- Go to **Design** → **New** → **Sequence**

2. Create Variable

- Counter → Int32, Default → 5

Name	Data Type	Scope	Default Value
Create variable			
[x] Counter	Int32	P2 D	{ } 5

3. Add Do While

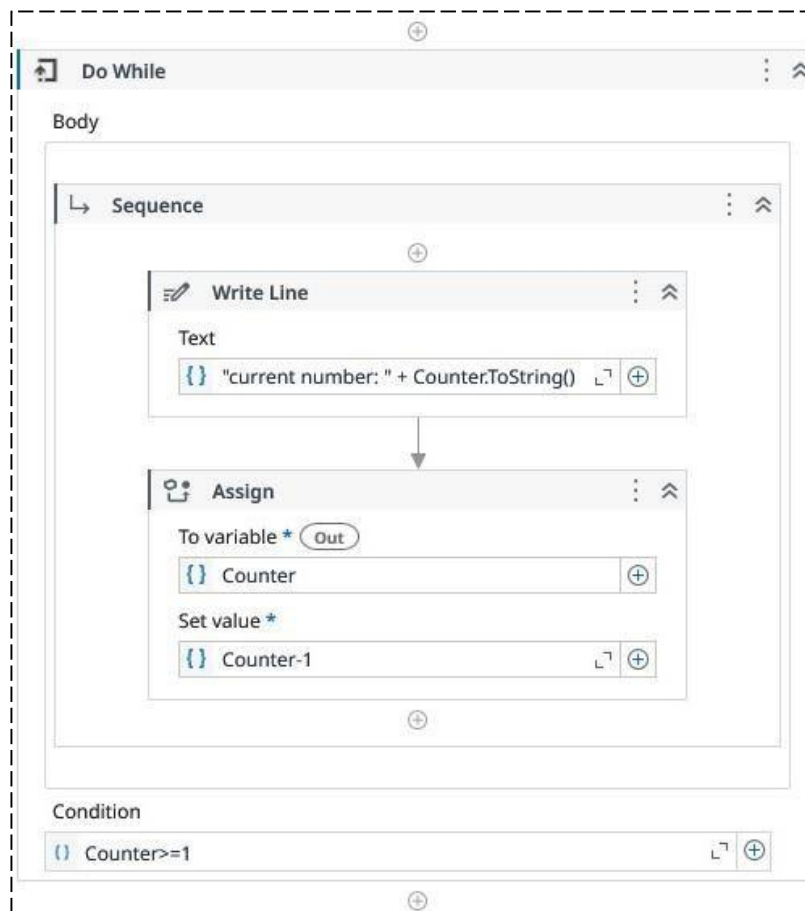
- Condition = Counter>=1

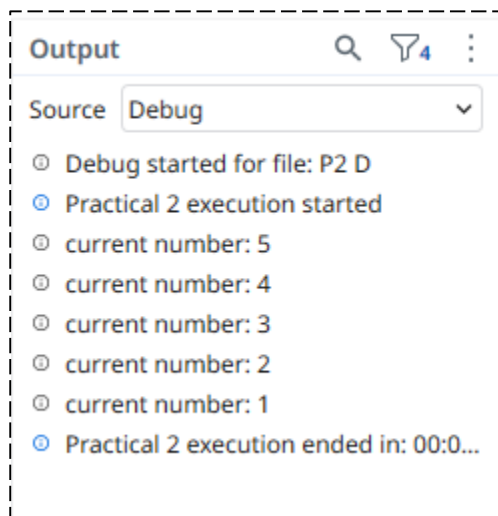
4. Under Body Add Sequence

- Add Write Line
- Text = "**current number:** " + Counter.ToString()

5. Add Assign

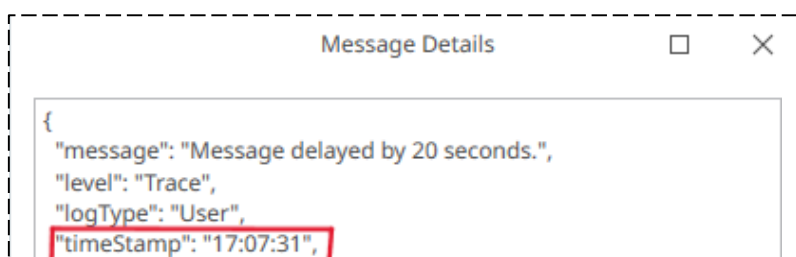
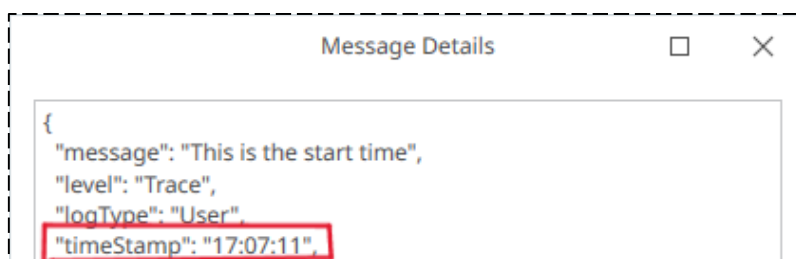
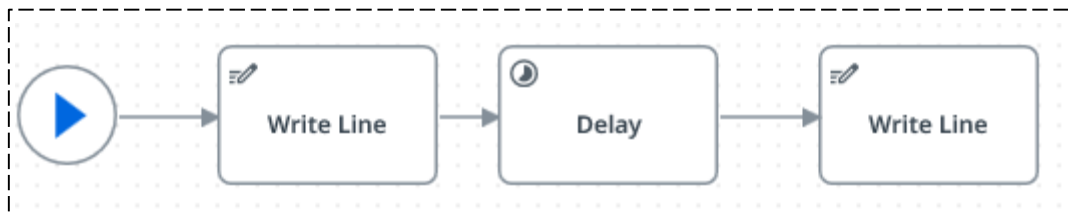
- Variable = Counter
- value = Counter-1





e) Create an automation using Delay Activity between two writeline activities to separate their execution by 20 seconds

1. Create new Flowchart
2. Add write line connect with previous start node
 - Text = "This is the start time"
3. Add Delay connect with previous write line
 - Duration = 00h 00m 20.000s
4. Add write Line connect with previous Delay
 - Text = "Message delayed by 20 seconds."



f) Create an automation to demonstrate use of decision statements (if)

Theory: The If activity contains a statement and two conditions. The first condition (the activity in the Then section) is executed if the statement is true, while the second one (the activity in the optional Else section) is executed if the statement is false. If activities can be useful to make decisions based on the value of variables.

1. Create new Sequence
2. Create Variable
 - FirstNumber → Int32
 - SecondNumber → Int32
3. Add Input Dialog
 - Lable = "Please insert a number"
 - value = FirstNumber
4. Add second Input Dialog
 - Lable = "Please enter a number"
 - Value = SecondNumber
5. Add If Else
 - If Condition = FirstNumber mod SecondNumber = 0
6. Add Write Line under Then
 - Text = FirstNumber.ToString + " Is a divisible by " + SecondNumber.ToString+"."
7. Under Else, Add Write Line
 - Text = FirstNumber.ToString + " Is not divisible by " + SecondNumber.ToString+"."

Input Dialog

Dialog Title
() "First Number" L¹ +

Input Label
() "Please insert a number" L¹ +

Input Type
Text Box v

Value entered
() FirstNumber +

Input Dialog

Dialog Title
() "Second Number" L¹ +

Input Label
() "Please enter a number" L¹ +

Input Type
Text Box v

Value entered
() SecondNumber +

If

Condition *
() FirstNumber mod SecondNumber = 0 L¹ +

Then

Write Line

Text
() FirstNumber.ToString + " Is a divisible by " L¹ +

Else

Write Line

Text
() FirstNumber.ToString + " Is not divisible " L¹ +

Output

Source Debug

- Debug started for file: P2 F
- Practical 2 execution started
- 2548 Is a divisible by 4.
- Practical 2 execution ended in: 00:00:08

Source Debug

- Debug started for file: P2 F
- Practical 2 execution started
- 14529 Is not divisible by 2.
- Practical 2 execution ended in: 00:00:10

Practical No: 3

Types of Recording

a) Basic Recording using Toolbar Steps:

1. Create Sequence

Go to Design → New → Sequence

Don't add any activity



2. Open Notepad

- Type: "Hello, how are you" , "I'am doing well, Thank you for asking"
- do not close the Notepad

3. Start Recording

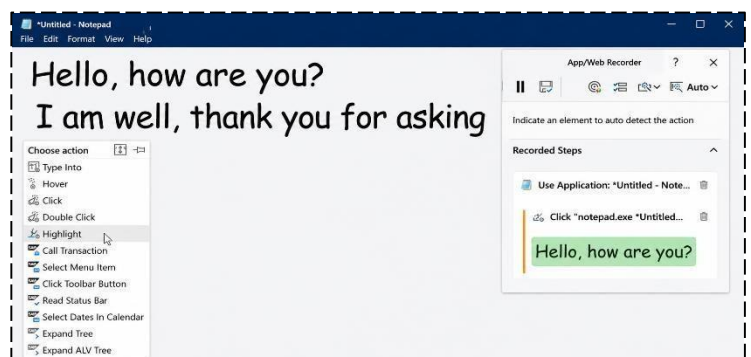
- Go to Design → Recording ● → Basic
- Indicate the Notepad ■

4. Select Text

- Drag and Select the full text in Word
- Recording toolbar will appear

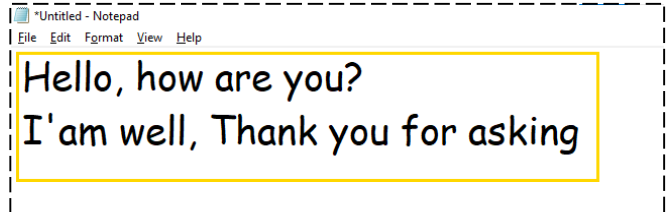
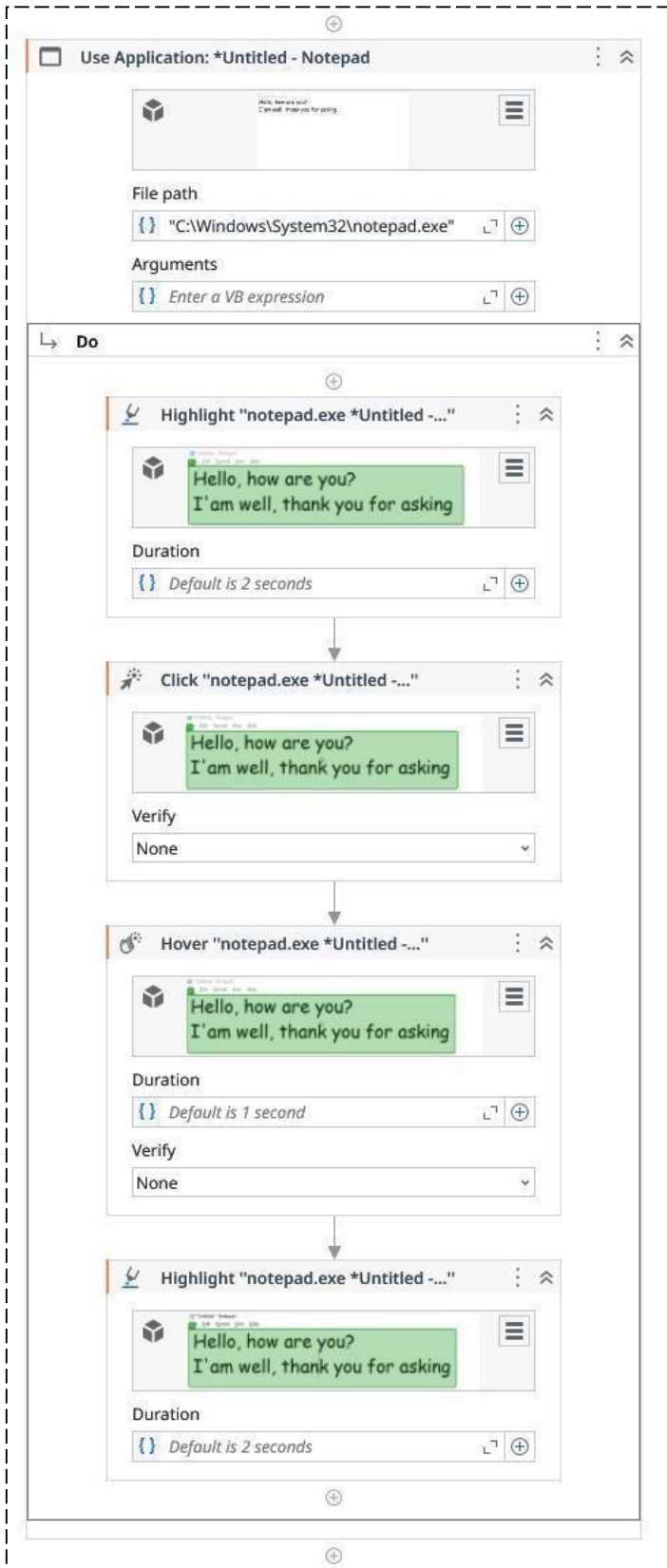
5. Add Action

- Click → Highlight
- Click → Click
- Click → Hover
- Click → Highlight
- Save the recording
- UiPtha creates activity structure itself after saving the recordings



6. Run Automation

- Click Run (F5)



b) Basic Recording using Notepad Steps:

1. Create Sequence

- Go to **Design** → **New** → **Sequence**

2. Open Notepad

- Type: "**Hello, how are you**"
- do not close the Notepad

3. Start Recording

- Go to **Design** → **Recording** ● → **Basic**
- Indicate the Word document



4. Select Text

- Drag and Select the full text in Word
- Recording toolbar will appear

5. Add Action

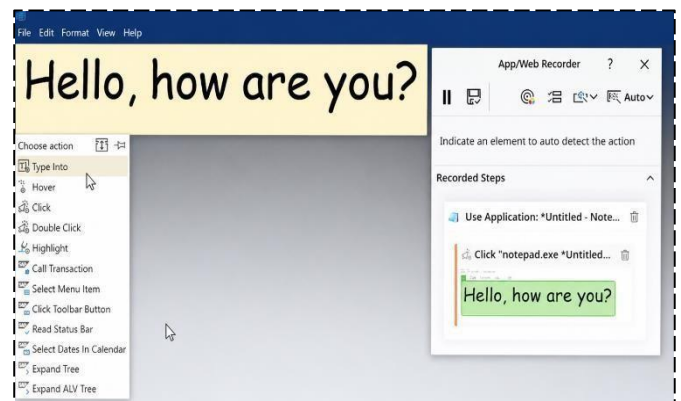
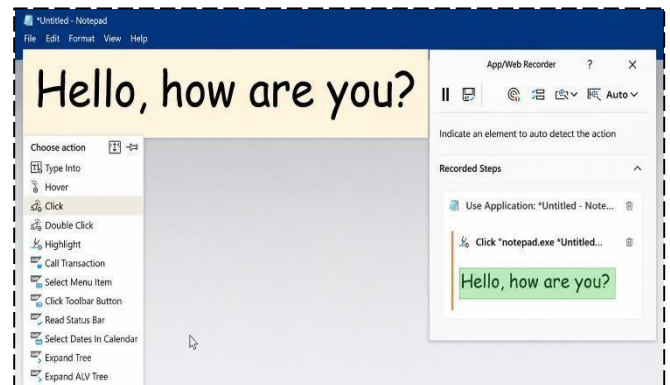
- Click → **Click**
- **Click** → **Type Into**
- Click **Confirm**
- Save the recording

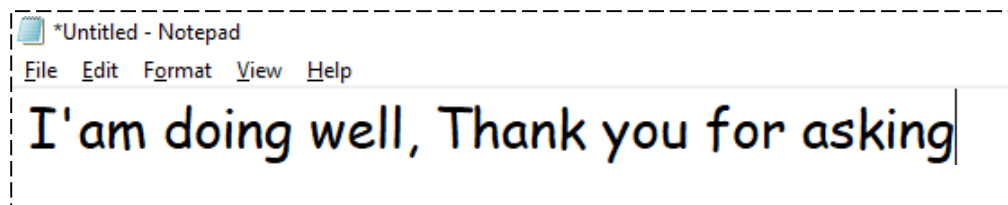
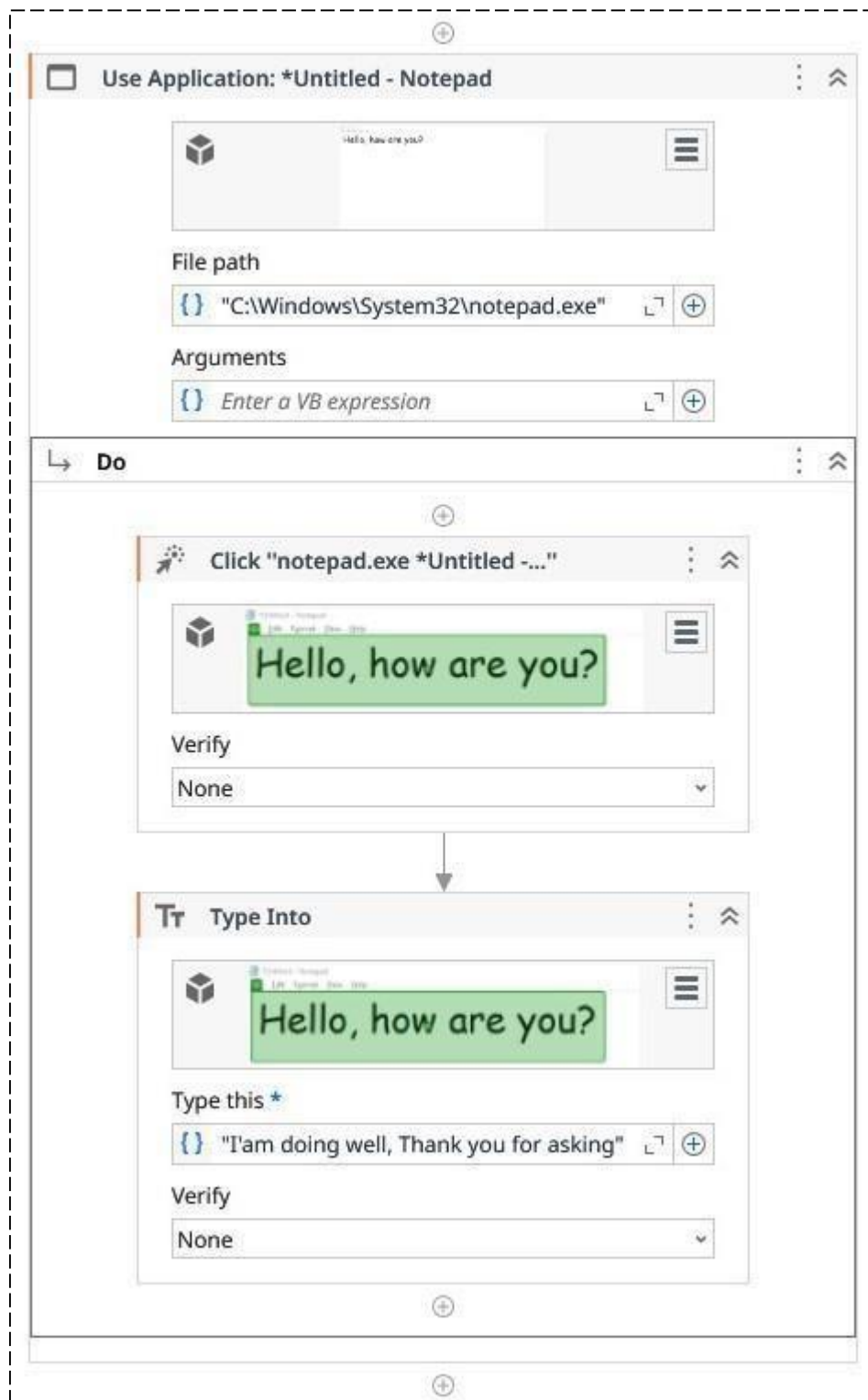
6. Edit Activity

- Go to generated **Type Into**
- In **Type This**:
- I'am doing well, Thank you for asking

7. Run Automation

- Click **Run (F5)**





c) Desktop Recording using Tool bar Steps:

1. Create Sequence

- Go to Design → New → Sequence

2. Open Word Docx

- Write text:
- !!!“Good Morning”!!!
- This Practical is about
- Desktop Recording using
- Toolbar

3. Open UiPath

- Start the Recording
- open word and Follow the Below activity
- Click on top left corner File > Save As > select location > Click on Save
- After the saving it will show Your Docx Text

4. Create Highlight

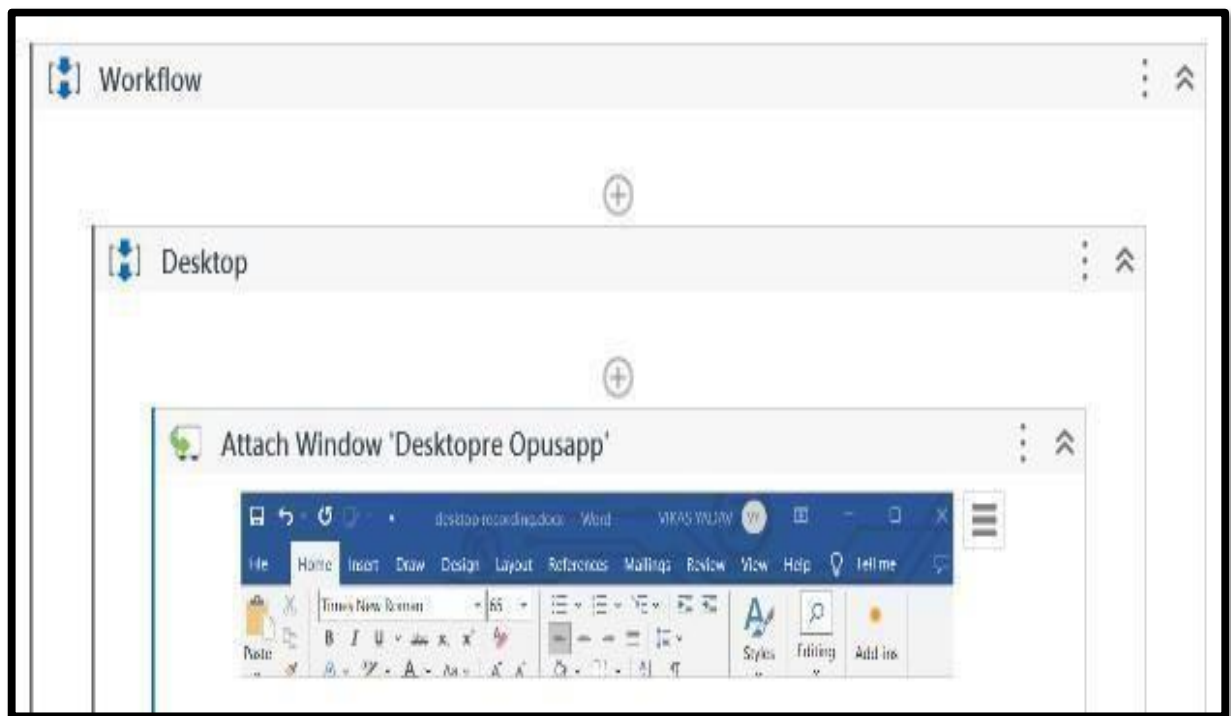
- After the saving Docx Drag or Select the Whole text and click on Highlight option
- Then Save the Recording
- Come to UiPaath and run the activity



!!!“Good Morning”!!!
This Practical is about
Desktop Recording using
Toolbar

d) Desktop Recording by creating a workflow Steps:

1. Open UiPath Studio and create a new project.
2. Drag and drop the "Open Application" activity from the activities panel onto your workflow. This activity allows you to open the application you want to record.
3. Configure the "Open Application" activity by specifying the path of the application executable file.
4. Drag and drop the "Click" activity from the activities panel onto your workflow. This activity allows you to simulate a mouse click on the application.
5. Configure the "Click" activity by indicating the element you want to click on within the application.
6. Repeat steps 4 and 5 for any other actions you want to record, such as typing in text boxes or selecting options from dropdown menus.
7. Drag and drop the "Delay" activity from the activities panel onto your workflow. This activity allows you to add pauses between actions.
8. Configure the "Delay" activity by specifying the duration of the pause in milliseconds.
9. Continue adding activities to your workflow to capture all the actions you want to record.
10. Once you have finished creating the workflow, save it.
11. To run the workflow and record the desktop actions, click on the "Run" button in the toolbar.
12. UiPath Studio will execute the activities in the workflow and record the desktop actions based on your instructions.
13. After the recording is complete, you can view and modify the recorded activities in the workflow.
14. Test your automation by running the modified workflow and see if it performs the recorded actions correctly on your desktop.



⊕

TI Type Into 'edit Page 1 content'

!!! "Good morning"!!!

Text *

{ } "Hii , "This is Vikas." "

⌂ ⊕ ▼

⊕

Click 'Button File Tab'

File Home Inse

Times Ne

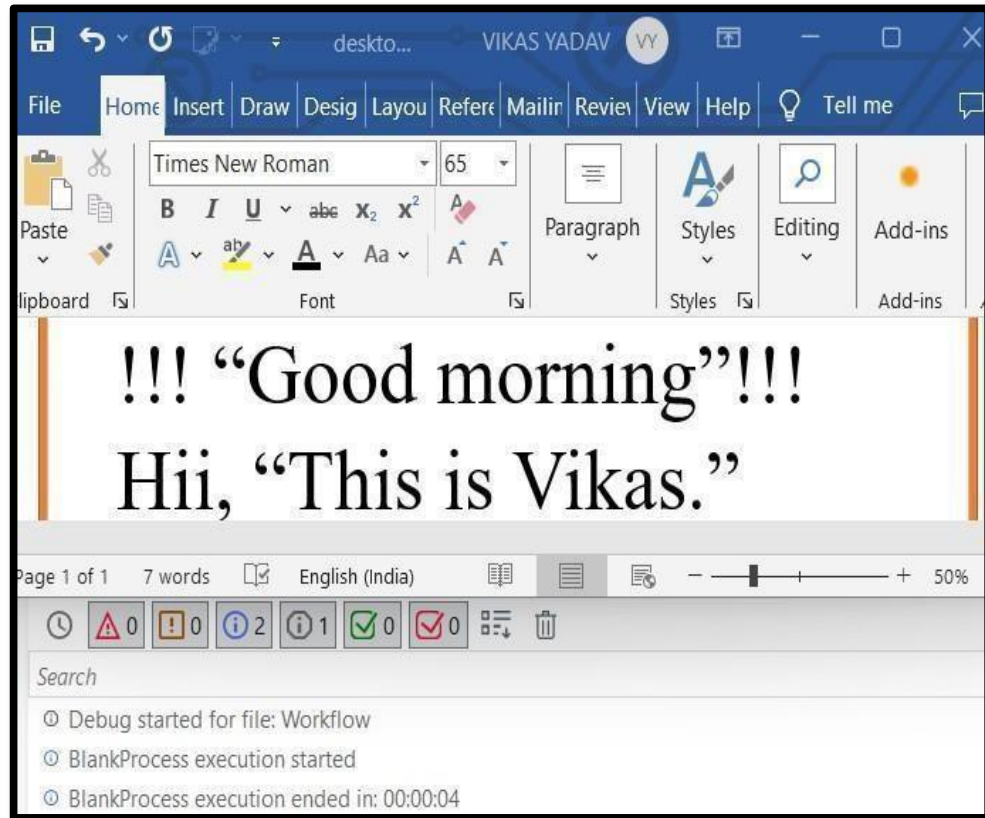
⊕

Click 'List Item Save'

Save



Output:



e) Web Recording e.g. Find the rating of the movie from imdb web site Steps:

1. Create a New Sequence

- Go to **Design Tab**
- Click **New** → **Sequence**
- Name it: IMDB_Recording_Automation

2. Open Browser

- Add **Use Application / Browser**
- Open a browser (e.g., Google Chrome)
- Navigate to:
 - IMDB

3. Start Recording

- Go to **Design Tab**
- Click **Recording** → **Basic**
- Indicate the **browser window**

4. Perform Recording Actions

Step 4.1 – Select Top Menu Bar

- Drag and select the **entire top menu bar**
- Choose **Click**

Step 4.2 – Select Search Input Box

- Drag and select the **search input field**
- Choose **Type Into**
- Click **Confirm**
- Click **Pause Recording**, then click again to resume

Step 4.3 – Select Search Icon

- Carefully select only the **search icon (no extra space)**
- Choose **Double Click**

Step 4.4 – Pause Recording

- Pause the recording
- Perform a manual search (movie/series/anime/cartoon)

Step 4.5 – Highlight Rating

- Resume recording
- Drag and select the **rating text**
- Choose **Highlight**

5. Save Recording

- Click **Save & Exit**
- UiPath will auto-generate all activities

6. Modify Generated Activities

- Locate **Type Into** activity
- Enter the text: Dragon Ball
- Locate **Highlight** activity
- Set **Duration**: 3 to 5 seconds

7. Run Automation

- Click **Run** OR press **F5**

Use Application: IMDb: Ratings, Reviews, and Where to Watc...



File path

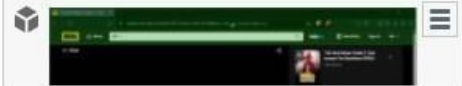
"C:\Program Files\BraveSoftware\Brave

Arguments

Enter a VB expression

Do

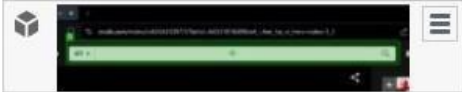
Click "brave.exe 'The Devil We..."



Verify

None

Type Into



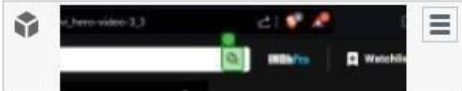
Type this *

"Dragon ball"

Verify

None

Click "brave.exe 'The Devil We..."



Verify

None

Highlight "brave.exe Find - IMDb -..."



Duration

3s



Dragon Ball Z

1996–2003 · TV-PG · TV Series

★ 8.8 (177K)

☆ Rate

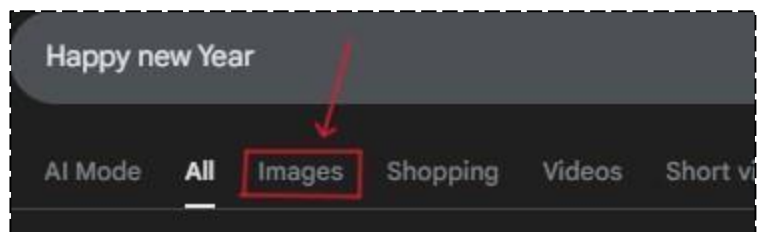
👁 Mark as watched

f) Web Recording manually Steps:

1. Create a New Sequence
 - Go to **Design Tab**
 - Click **New** → **Sequence**
 - Name it: Web_Recording_Automation
2. Install UiPath Browser Extension
 - Open browser (e.g., Google Chrome)
 - Search for **UiPath Extension for Chrome** or use the link:
<https://chromewebstore.google.com/detail/uipath-browser-automation/pgbnimfaaifjpebleldfhgcjdnaeafdi>
 - Click **Add to Chrome**
 - Enable the extension properly

☞ This step is important for web automation to work correctly

3. Open Website
 - Go to a search engine like Google
 - Search for: Happy New Year
4. Start Recording
 - Open UiPath Studio
 - Go to **Design Tab**
 - Click **Recording** → **Basic**
 - Indicate the browser screen
5. Perform Recording Actions
 - Click on Images
 - Highlight Images
 - Select **3 to 4 images**
 - Use **Highlight** activity for each image



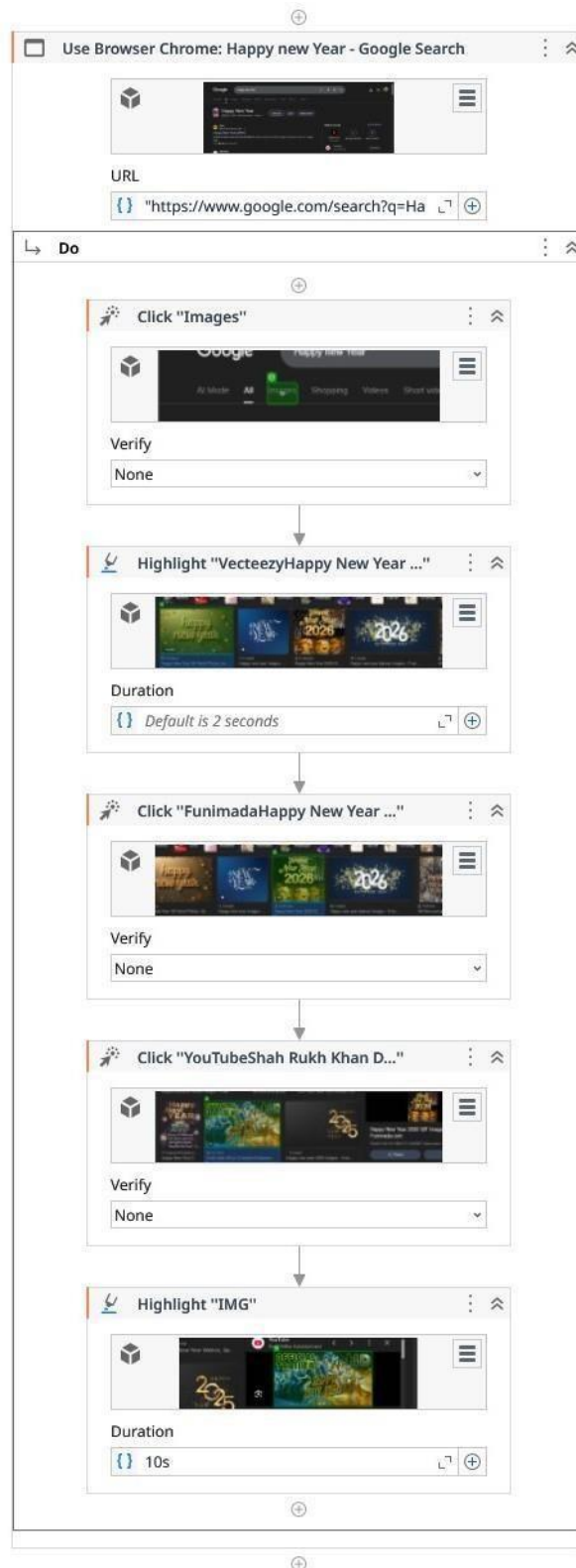
(Optional Actions)

You can also add:

- **Click** (open image)
 - **Hover** (mouse over image)
 - **Type Into** (search box)
 - **Form Filling** (enter data in fields)
6. Save Recording
 - Click **Save & Exit**
 - Activities will be auto-generated in sequence

7. Run Automation

- Click **Run** OR press **F5**
- The automation will: Open browser Perform actions (highlight/click/etc.)



Practical No: 4

Excel Automation

a) Automate the process to extract data from excel file into a data table and vice versa

Theory: Create an automation project that extracts some specific information from Wikipedia and writes it to an Excel spreadsheet. You can use this type of automation in different scenarios, such as extracting lists of products and their prices from e-commerce websites.

Note: *It is recommended to run your web automations on Internet Explorer 11 and above, Mozilla Firefox 50 or above, or the latest version of Google Chrome.*

Let's say you want to start reading up on economics and you want to get a list of Wikipedia articles on the subject, together with their URLs, and the additional information that is provided in the search results for each article. You can do the following:

1. Create a New Sequence

- Open UiPath Studio
- Create a **New Process**
- Add a **Sequence**

2. Add Extract Data Activity

- Drag **Extract Table Data** activity into the sequence

3. Create Output Variable

Create a variable:

- Name: ExtractDataTable
- Type: DataTable (Browse > System.Data.DataTable)
- Scope: Main Sequence

4. Open Wikipedia Page

Use this URL:

- <https://en.wikipedia.org/w/index.php?search=economics&title=Special:Search&fulltext=1&ns0=1>

5. Indicate Target on Screen

- Click **Indicate Target on Screen**
- Open the browser and select the Wikipedia page

6. Extract Data (Important Step)

- Click “**Extract Data**” wizard and:

Select Data Columns: For each data select button Select Data & Extract for saperate data.

1. Article Title (Full block)

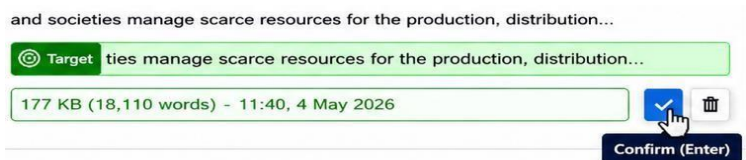
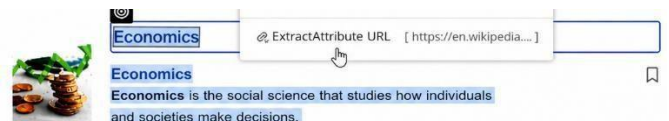
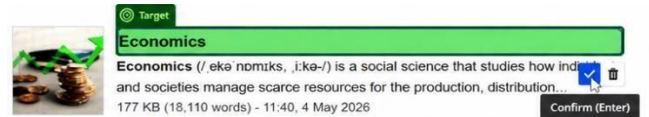
- Select full title (not just text)
- Choose **Attribute** → **Text**

2. Article URL (Don't select Full Block)

- Select title text again
- Choose **Attribute** → **URL**

3. Article Description

- Select description text
- Choose **Attribute** → **Text**



Click **Next** → **Save and Confirm**

7. Set Output DataTable Input/output

- In Extract activity properties
- Destination Data Table = ExtractDataTable

8. Create Excel File

- Manually create an Excel file (e.g., Book1.xlsx)
- Save it in your desired location

Example:

E:\UIPath Practical\Practical 4\Book1.xlsx

9. Add Excel Process Scope

- Drag **Excel Process Scope**

10. Add Use Excel File

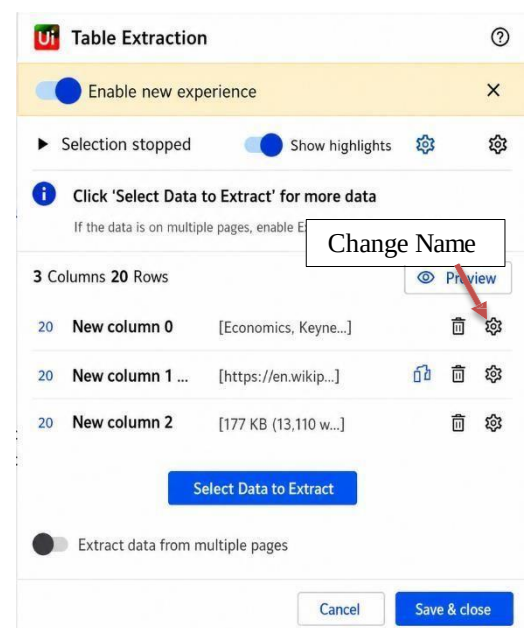
- Inside Excel Process Scope:
- WorkbookPath = "E:\UIPath Practical\Practical 4"

11. Add Write Range Activity (IMPORTANT)

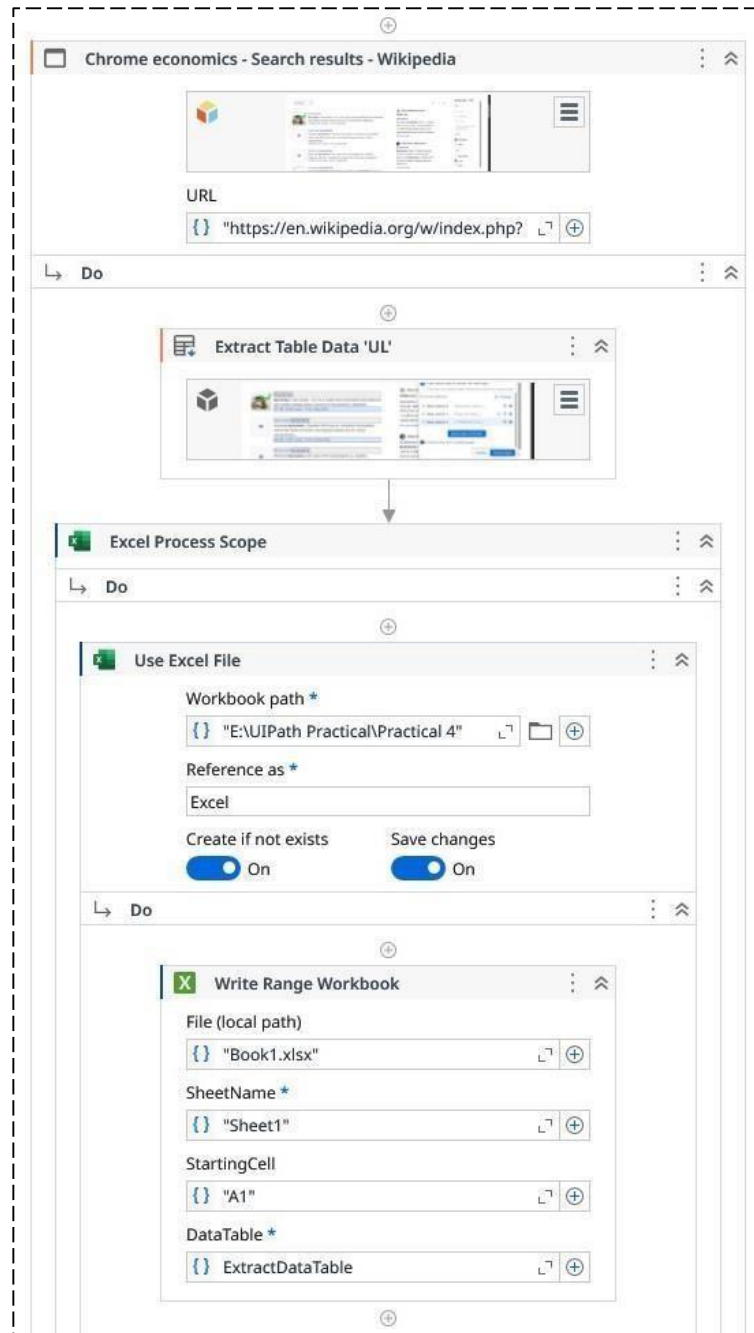
Inside **Use Excel File**, add:

Write Range (Modern)

Set properties: For Local Path click (+) Icon and select the use local path option



- Flie (Local Path) : "Book1.xlsx"
- SheetName = "Sheet1"
- StartingCell = "A1"
- DataTable = ExtractDataTable



	A	B	C
1	Article Title	Article Url	Additional Info
2	Economics	https://en.wikipedia.org/wiki/Economics	177 KB (18,110 words) - 11:40, 4 May 2026
3	Keynesian economics	https://en.wikipedia.org/wiki/Keynesian_economics	108 KB (13,321 words) - 17:08, 23 April 2026
4	Behavioral economics	https://en.wikipedia.org/wiki/Behavioral_economics	119 KB (13,003 words) - 02:02, 21 April 2026
5	Ecological economics	https://en.wikipedia.org/wiki/Ecological_economics	80 KB (9,338 words) - 05:48, 9 December 2025
6	London School of Economics	https://en.wikipedia.org/wiki/London_School_of_Economics	177 KB (15,680 words) - 06:11, 2 May 2026
7	Media economics	https://en.wikipedia.org/wiki/Media_economics	9 KB (911 words) - 18:32, 23 March 2026
8	Home economics	https://en.wikipedia.org/wiki/Home_economics	56 KB (5,839 words) - 07:16, 2 February 2026
9	Neoclassical economics	https://en.wikipedia.org/wiki/Neoclassical_economics	52 KB (6,363 words) - 20:12, 10 April 2026
10	Philosophy and economics	https://en.wikipedia.org/wiki/Philosophy_and_economics	26 KB (2,437 words) - 11:57, 15 April 2026
11	Capital (economics)	https://en.wikipedia.org/wiki/Capital_(economics)	26 KB (3,358 words) - 22:07, 12 April 2026
12	Development economics	https://en.wikipedia.org/wiki/Development_economics	60 KB (6,958 words) - 12:53, 13 March 2026
13	Austrian school of economics	https://en.wikipedia.org/wiki/Austrian_school_of_economics	66 KB (6,983 words) - 19:01, 3 May 2026
14	Trickle-down economics	https://en.wikipedia.org/wiki/Trickle-down_economics	43 KB (4,363 words) - 06:14, 2 May 2026
15	Socialist economics	https://en.wikipedia.org/wiki/Socialist_economics	144 KB (18,430 words) - 13:41, 15 September 2025
16	Heterodox economics	https://en.wikipedia.org/wiki/Heterodox_economics	30 KB (2,984 words) - 22:18, 28 December 2025
17	Economics (disambiguation)	https://en.wikipedia.org/wiki/Economics_(disambiguation)	1 KB (190 words) - 23:02, 25 August 2025
18	Sexual economics	https://en.wikipedia.org/wiki/Sexual_economics	36 KB (4,760 words) - 08:48, 19 April 2026
19	Elasticity (economics)	https://en.wikipedia.org/wiki/Elasticity_(economics)	32 KB (4,309 words) - 21:44, 22 April 2026
20	Labour economics	https://en.wikipedia.org/wiki/Labour_economics	49 KB (4,858 words) - 02:49, 20 April 2026
21	Financial economics	https://en.wikipedia.org/wiki/Financial_economics	136 KB (12,947 words) - 07:41, 16 April 2026
22			

b) Create an automation To Write data to specific cell of an excel sheet.

Steps:

1. create Sequence and name it write Cell
2. add Excel Process Scope Activity
3. Add Use Excel File Activity under Do
 - Workbook Path : "E:\UIPath Practical\Practical 4\Book2.xlsx"
4. Add Write Cell Activity under Do
 - What to write = "Write Your Name"
 - Where to Write = Excel.Sheet("Sheet1").Cell("C5")
 - Run the Activity

Excel Process Scope

Do

Use Excel File

Workbook path *

{ } "E:\UIPath Practical\Practical 4\Bo" L7

Reference as *

Excel

Create if not exists

On

Save changes

On

Do

Write Cell

What to write *

{ } "Yash Saundalkar" L7

Where to write *

{ } Excel.Sheet("Sheet1").Cell("C5") L7

Auto increment row

Off

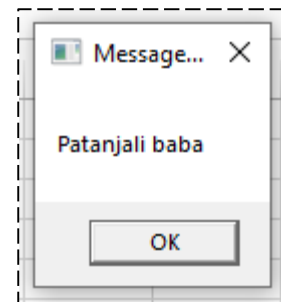
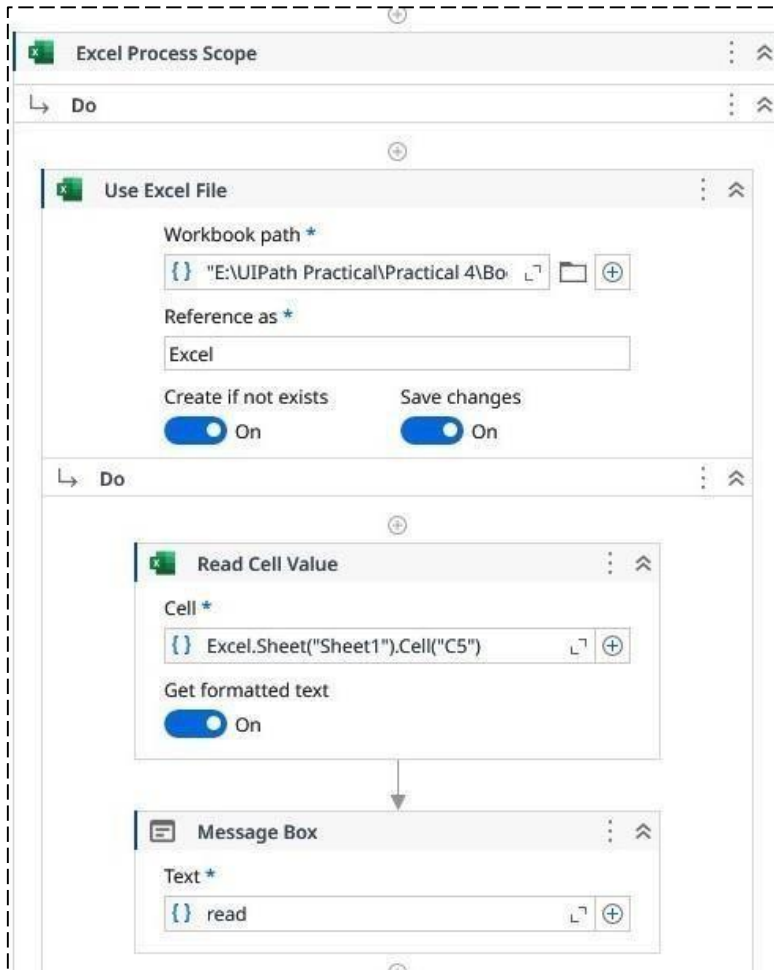
S21					
	A	B	C	D	E
1					
2					
3					
4					
5			Yash Saundalkar		
6					
7					
8					

c) Create an automation To Read data to specific cell of an excel sheet.

Theory: By following these below steps, you'll be able to create an automation that reads data from a specific cell in an Excel sheet in UiPath. This helps to read specific amount of data from a large amount of data and it save time because you dint have to search one by one for a specific data. So lets get started:

Steps:

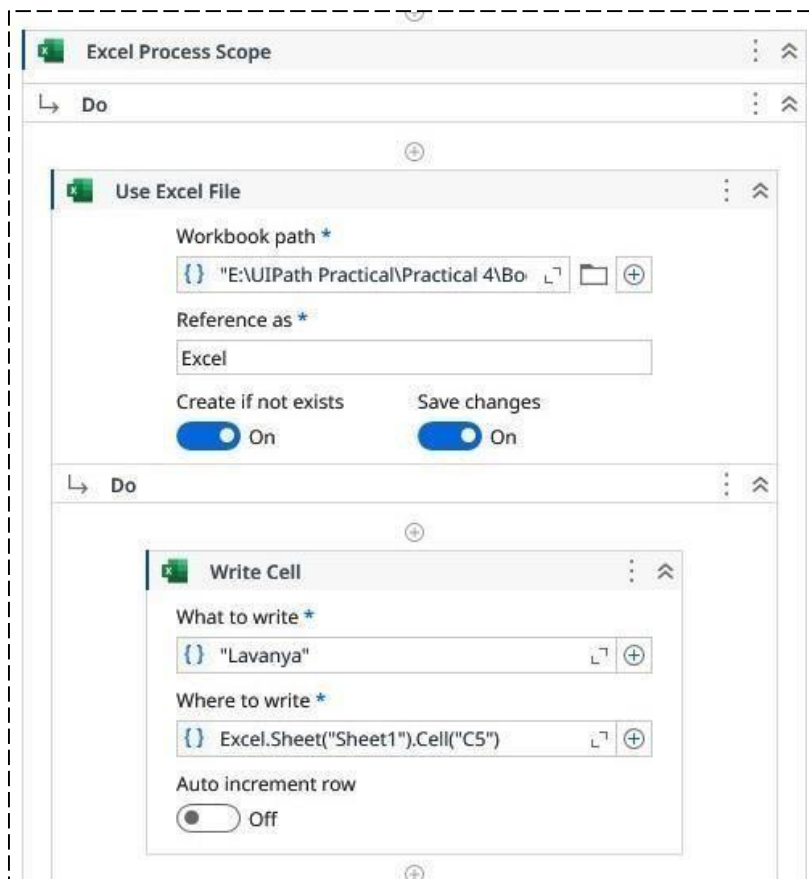
1. Create Sequence name it Read Cell Value
2. Create variable
 - Name = read
 - type = String
3. Add Excel Process Scope
4. Add Use Excel File Activity under Do
 - Workbook Path: "E:\UIPath Practical\Practical 4\Book2.xlsx"
5. Add Read Cell Value Activity Under Do
 - Cell = Excel.Sheet("Sheet1").Cell("C5")
 - Save to = read (check Properties for new version Uipath)
6. Add message Box
 - 7. text = read.ToString



d) Create an automation to append data to specific cell of an excel sheet.

Steps:

1. create Sequence and name it Append Data
2. add Excel Process Scope Activity
3. Add Use Excel File Activity under Do
 - Workbook Path : "E:\UIPath Practical\Practical 4\Book2.xlsx"
4. Add Write Cell Activity under Do
 - What to write = "Write data"
 - Where to Write = Excel.Sheet("Sheet1").Cell("C5")
 - Run the Activity



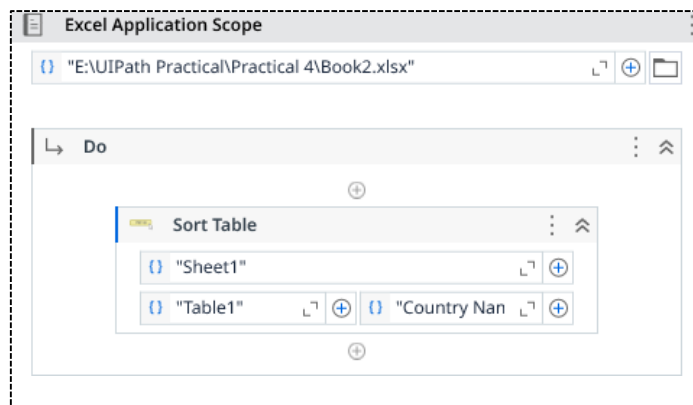
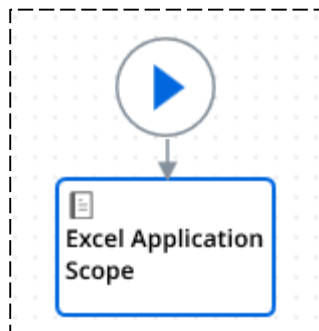
C
Yash
Pratik
Satish
Sooraj

C
Yash
Lavanya
Satish
Sooraj

e) Create an automation to sort a table of an excel sheet.

Steps:

1. create Flowchart name it Sort data
2. create excel File with Different countries name and save it
3. Add excel Application Ecope
 - add excel path = "E:\UIPath Practical\Practical 4\Book2.xlsx"
4. Add Sort Table under Do
 - Sheet name = "Sheet1"
 - Table name = "Table1"
 - Column name = "Country Name"
5. Run the Activity



Before Sort

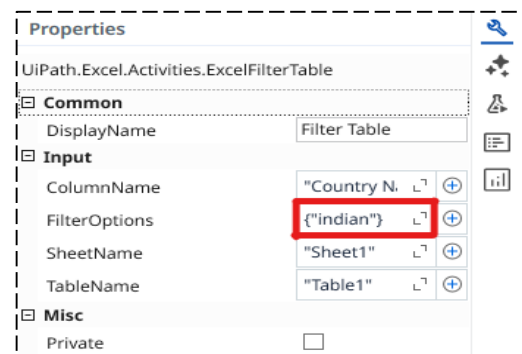
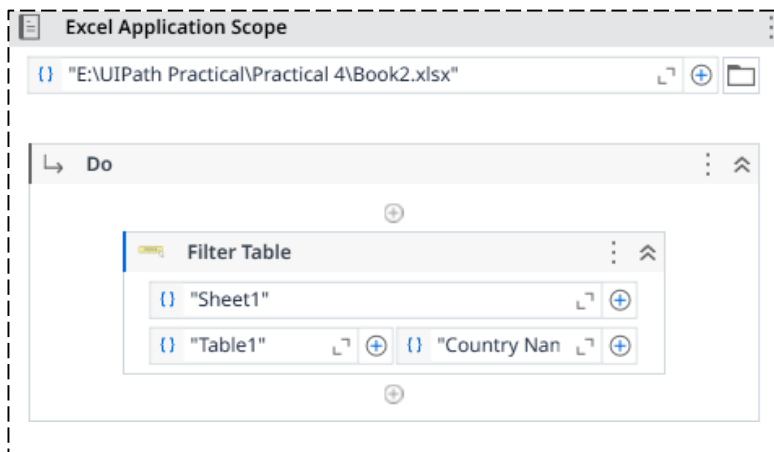
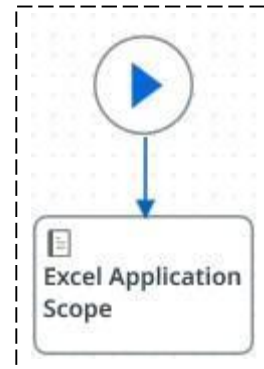
	A
1	Country Name
2	Mexico
3	Austria
4	Bangladesh
5	Chile
6	Denmark
7	Morocco
8	Australia
9	Belgium
10	Colombia
11	Dominica
12	Malaysia
13	Argentina
14	Bulgaria
15	Canada
16	Djibouti
17	Maldives
18	Algeria
19	Bahrain
20	Croatia
21	Dominican Republic
22	Mongolia
23	Armenia
24	

After Sort

	A
1	Country Name
2	Algeria
3	Argentina
4	Armenia
5	Australia
6	Austria
7	Bahrain
8	Bangladesh
9	Belgium
10	Bulgaria
11	Canada
12	Chile
13	Colombia
14	Croatia
15	Denmark
16	Djibouti
17	Dominica
18	Dominican Republic
19	Malaysia
20	Maldives
21	Mexico
22	Mongolia
23	Morocco
24	

f) Create an automation To filter a table of an excel sheet Steps:

1. Create Flowchart name it Sort data
2. Create excel File with similar countries name and data.
3. Add excel Application Scope
4. Add excel path = "E:\UIPath Practical\Practical 4\Book2.xlsx"
5. Add Filter Table under Do
 - Sheet name = "Sheet1"
 - Table name = "Table1"
 - Column name = "Country Name"
6. Filter Table Property
 - Input > FliterOptions = {"Indian"}
7. Run the Activity



Before Filter

	A	B
1		Country Name
2		
3	Yash	Japan
4	Pratik	Indian
5	Satish	Russian
6	Sunil	Nepali
7	Lavanya	Indian
8	Shilpa	Indian
9	Sooraj	Ukraine
10		

After Filter

	A	B
1		Country Name
4	Pratik	Indian
7	Lavanya	Indian
8	Shilpa	Indian
10		

Practical No: 5

Different controls in UiPath

a) Implement the attach window activity.

Theory: The Attach Window activity can be found in the Activities panel. This activity

is generally used to attach an already opened window. It is also auto-generated

when we record actions using the Basic or Desktop recorder. You will get a much

clearer idea after going through the example in the following section.

Steps: In this practical, we shall use the Attach Window activity manually. Here, we

are going to attach a Notepad window and then write some text into it:

1. Create a blank project and give it a meaningful name.
2. Drag and drop a Flowchart activity on the Designer panel.
3. Also, drag and drop a Click activity inside the Designer panel.

4. Set this Click activity as the Start node.

5. Double-click on the Click activity and then click on Indicate on screen.

Locate the Notepad icon.

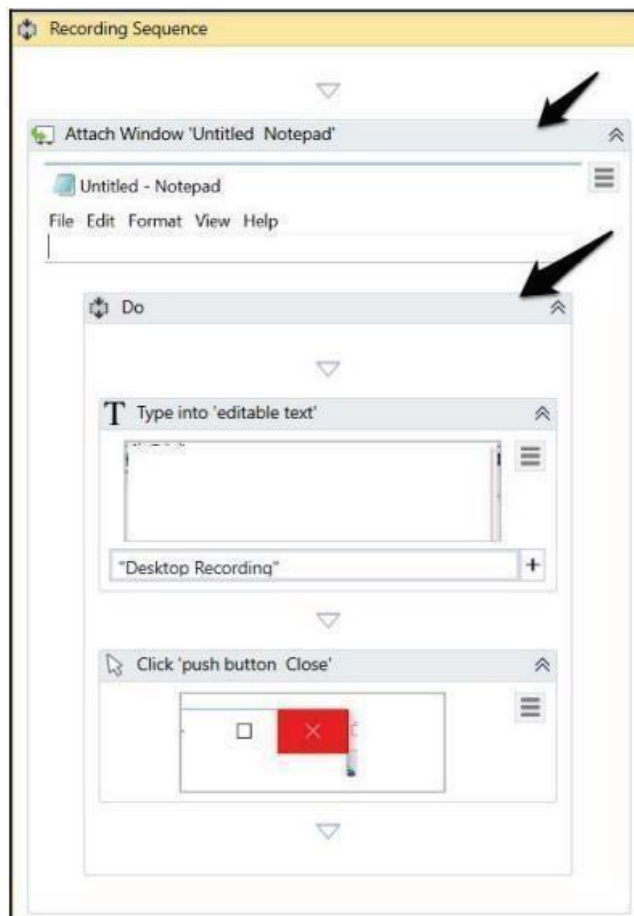
6. Drag and drop the Attach Window activity on the main Designer panel.

7. Connect the Attach Window activity to the Click activity.

8. Double-click on the Attach Window activity.

9. Click on Click Window on Screen and indicate the Notepad window.

10. The Notepad window is now attached to the previous activity:



11. For the sake of completeness, we are going to add a Type into activity.
12. Just drag and drop the Type into activity, inside the Attach Window activity.
13. Click on the Indicate element inside window and locate the Notepad window where you want to write the text.
14. Write the text in the Text property of the Type into the activity.

15. Hit the run button.

b) Automate using Anchor Base.

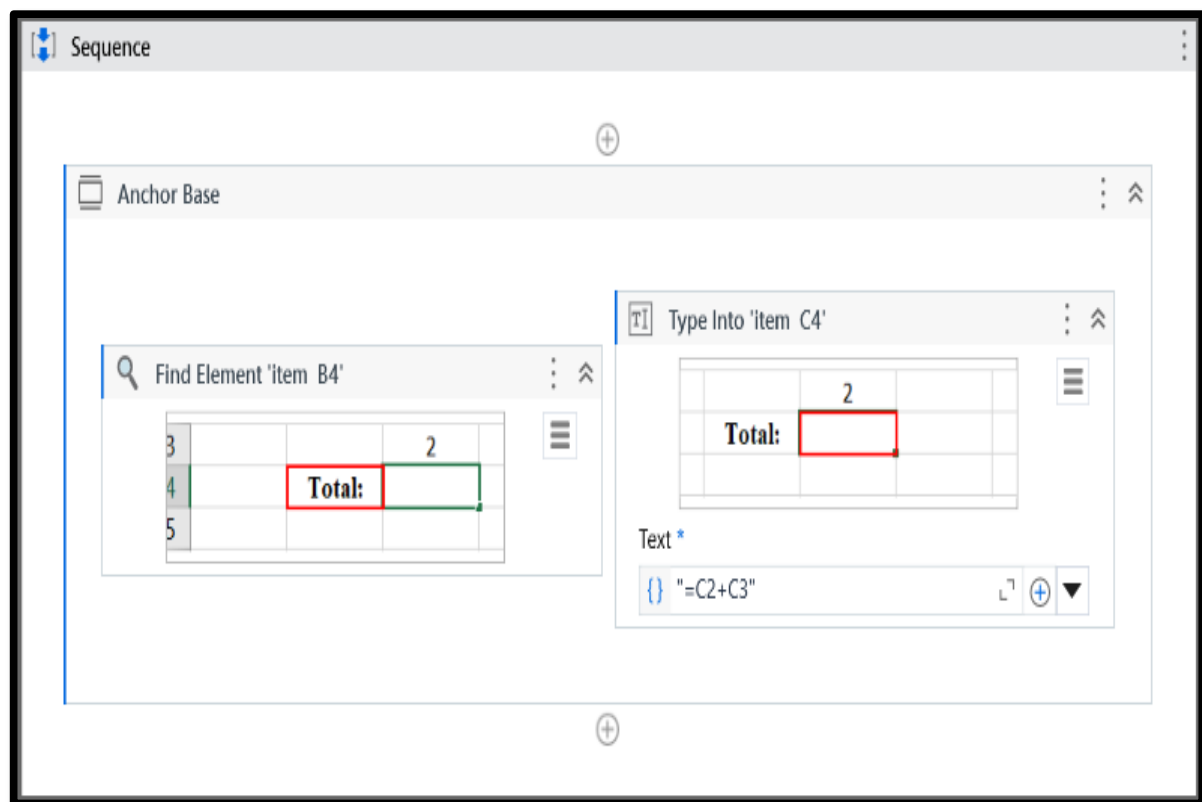
Theory: The "Anchor Base" activity is useful when you want to perform actions relative to a specific element on the screen.

It helps ensure that your automation remains robust even if the position or layout of other elements changes.

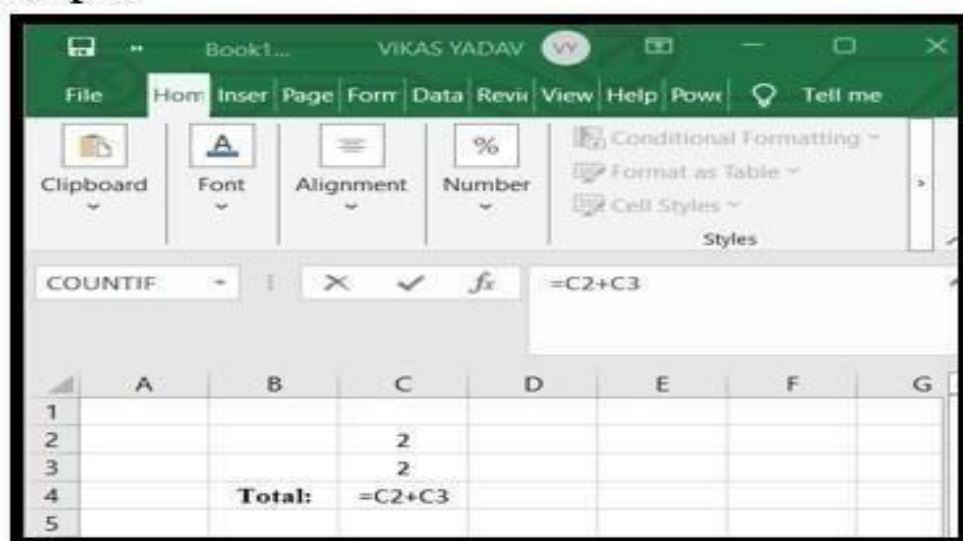
Steps:

1. Open UiPath Studio and create a new project.
2. Drag and drop the "Anchor Base" activity from the activities panel onto your workflow.
3. Indicate the anchor element on the screen by selecting it using the UIExplorer or the "Indicate Anchor" button.

4. This anchor element will serve as a reference point for the activities you want to perform.
5. Inside the "Anchor Base" activity, add the activities you want to perform relative to the anchor element. For example, you can add a "Click" activity to click on a button near the anchor element.
6. Configure the activities within the "Anchor Base" based on your automation requirements. For the "Click" activity,
7. you can indicate the element to click by selecting it using the UIExplorer or the "Indicate Element" button.
8. Continue adding more activities within the "Anchor Base" as needed, such as typing into text fields or extracting data from nearby elements.
9. Test your automation by running the workflow and see if the activities are performed correctly relative to the anchor element.



Output:



c) Automate using Element Exists.

Theory: The "Element Exists" activity is useful when you want to check whether a specific element exists on the screen before performing certain actions. It helps you make your automation more robust by allowing you to handle different scenarios based on the existence or non-existence of the element.

Steps:

1. Open UiPath Studio and create a new project called flowchart.
2. drag and drop “open browser” activity and link to start node.
3. Click on double click to view then in url field inside open browser activity

type www.google.com

4. Now open chrome browser and run www.google.com let it run in

background now maximize uipath

5. Drag and drop the "Element Exists" activity from the activities panel onto your workflow below open browser activity.

6. Click on “double click to view” of element exist activity next click on indicate on screen it will redirect to browser you were running in background

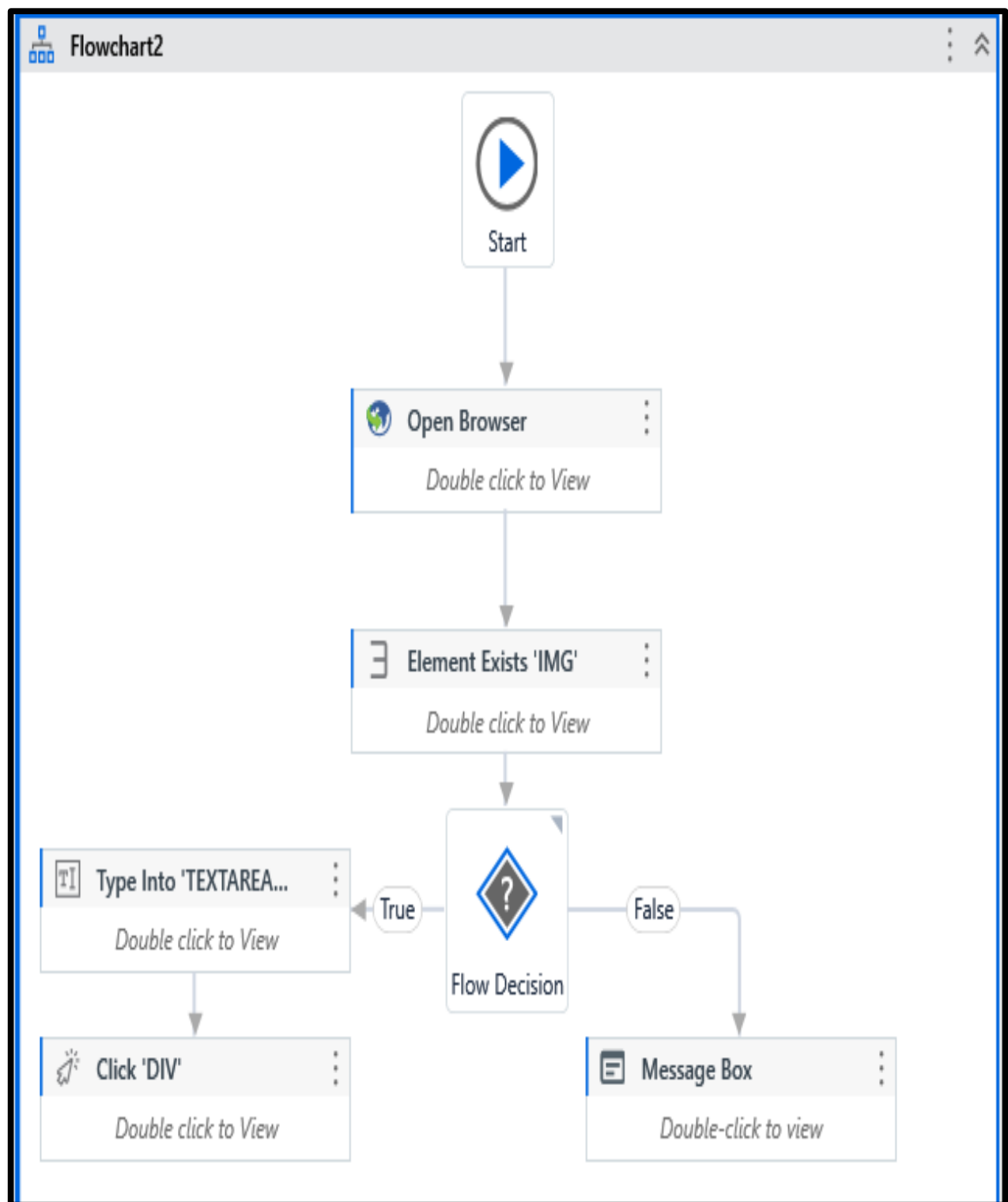
7. click on google logo “this means if google logo element exist then do further activity”.

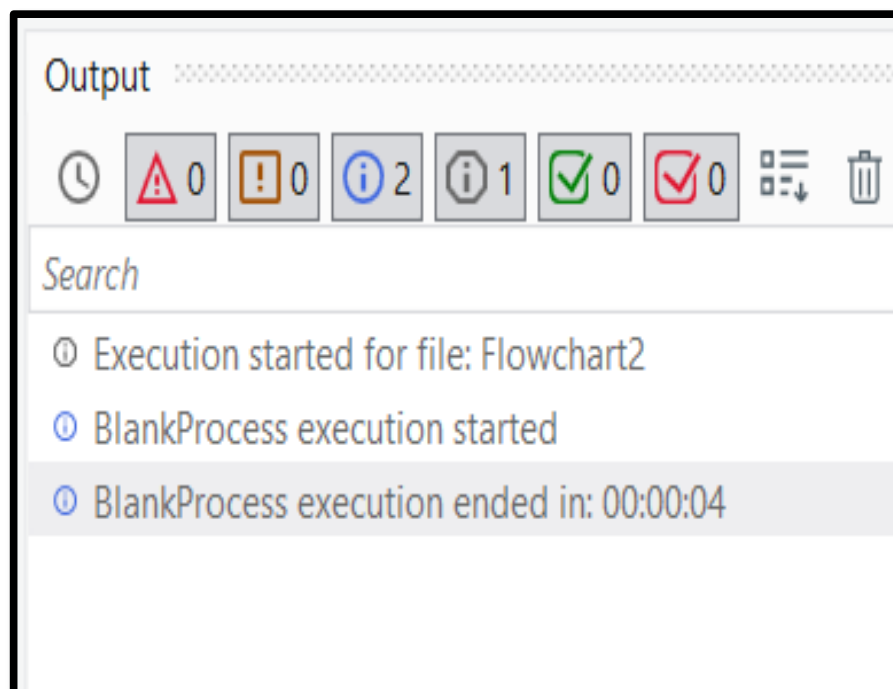
8. In the property panel of element exist there’s a field called Exist there press “ctrl + k” to create variable name “found” of type “Boolean”.

9. Configure the properties of the "Element Exists" activity. You can specify a timeout period, which determines how long UiPath will wait for the element to appear before considering it non-existent.

10. Based on the outcome of the "Element Exists" activity, add flow decision activity to perform different actions depending on whether the element exists or not.
11. in property panel of “decision activity there’s a field called condition type found variable.
12. Drag and drop type into activity and connect it to true part of flow decision, click on double click to view then click on indicate on screen that will redirect to browser
13. press F2 to wait for few seconds to reload the browser now click on search area and type “Mumbai University” now add “click activity” then
14. click on “double click to view” click on “indicate on screen” that will redirect to browser now select the first option of your search.

15. Drag and drop “Message Box” activity and connect it to false part of flow decision then click on “double click to view” under text field type "Element does not Exist"
16. Test your automation by running the workflow and see if the "Element Exists" activity correctly identifies the presence or absence of the element.





d) Automate using Find Children control.

Theory:

The "Find Children" activity is useful when you want to locate and interact with multiple child elements within a parent element. It allows you to

perform actions on each child element individually, making your automation more flexible and efficient.

Steps:

1. Open UiPath Studio and create a new project.
2. Drag and drop “open browser” Activity in property panel
3. under input there’s a browser type field select the browser in my case it is “chrome”

& in URL field type

"https://www.w3schools.com/tags/tryit.asp?filename=tryhtml_select".
4. Drag and drop the "Find Children" activity below open browser activity from the activities panel onto your workflow
5. Indicate the parent element on the screen by selecting it using the UIExplorer or the "Indicate Element" button.

6. This is the element that contains the child elements you want to find.

Basically, select the dropdown box.

7. In property panel of find children activity under output there's a children

field press ctrl + k to create "list" variable.

8. Drag and drop for each activity in property panel type argument type as

"UiPath.Core.UIElementnt".

9. now in for each activity under field "List of items" type variable "list".

10. Drag and drop "Get Attribute" activity.

11. in attribute field select "aaname" & in Save to field type "EndResult"

variable.

12. In property panel of Get attribute activity under output there's a Result

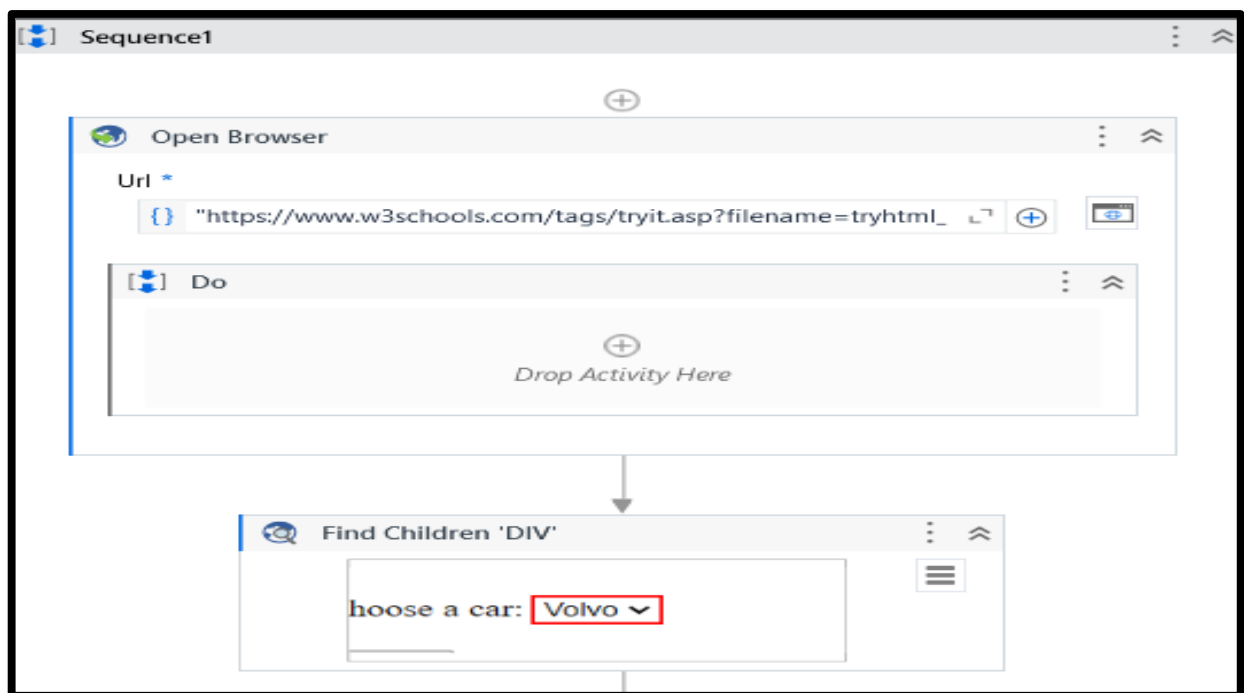
field press ctrl + k to create "EndResult" variable.

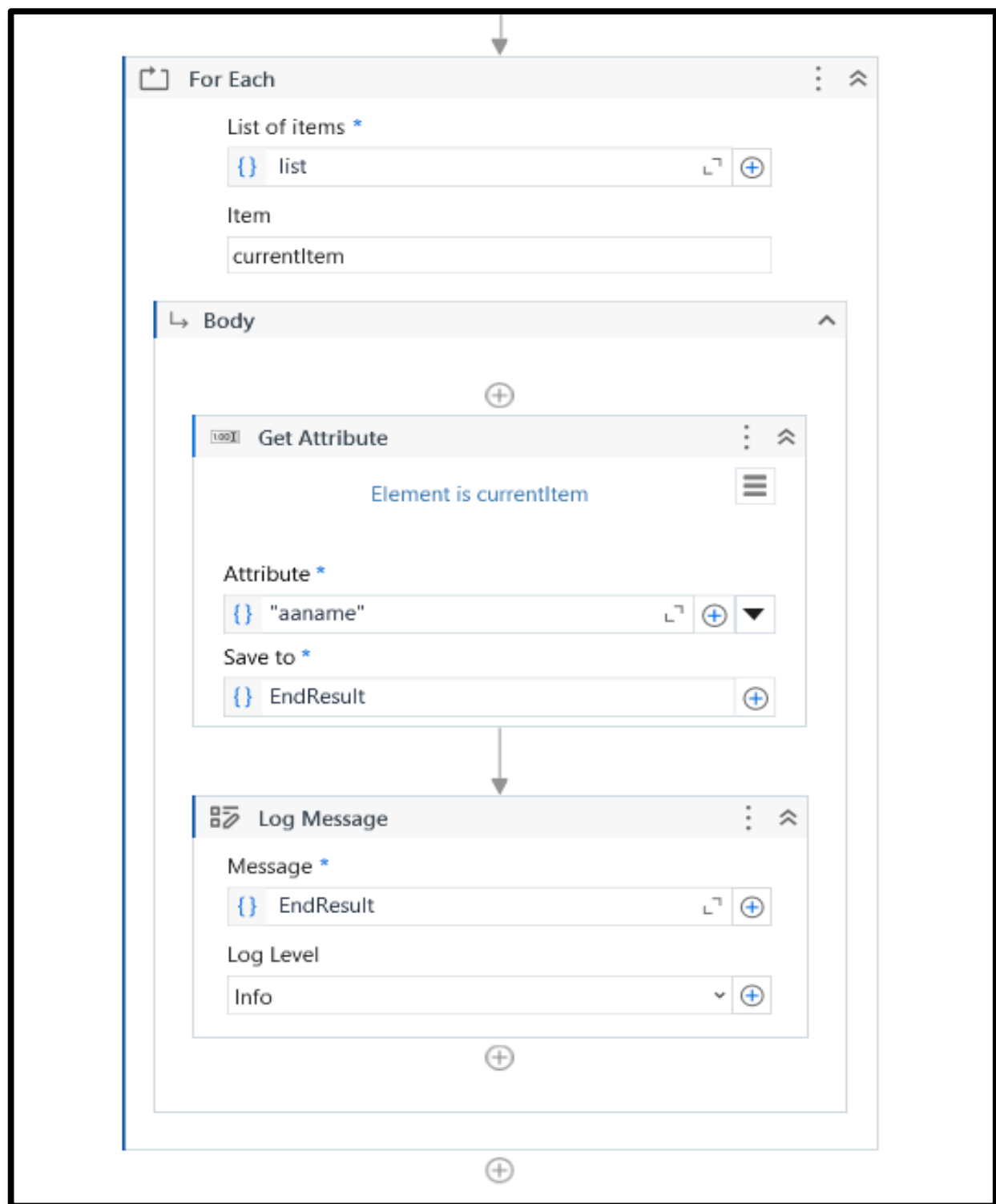
13. Drag and drop "Log Message" activity

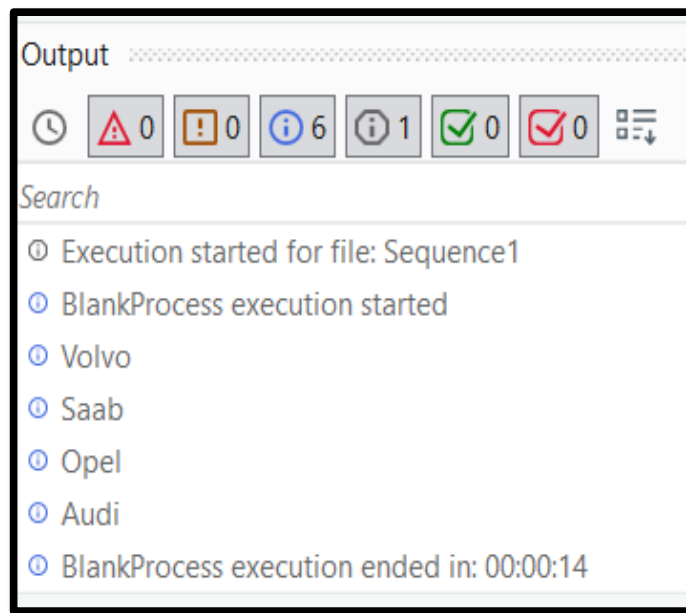
14. in message field type EndResult variable & in LogLevel select Info.
15. Configure the properties of the "Find Children" activity.
16. You can specify the scope of the search, such as searching within the parent element or the entire application window.
17. Add a variable of type "UiElement" to store the output of the "Find Children" activity. This variable will hold a collection of all the child elements found.
18. Inside a "For Each" activity, iterate through the collection of child elements.
19. Set the TypeArgument property of the "For Each" activity to "UiElement" and assign the variable created in the previous step as the input.
20. Within the loop, you can perform actions on each child element individually.

For example, you can click on a button, extract data, or perform other activities.

21. Test your automation by running the workflow and see if the "Find Children" activity correctly extracts the child element and display it on log message (output panel).







e) Use “Get Ancestor control”

Theory:

The "Get Ancestor" activity is useful when you want to retrieve a specific ancestor

element of a given element. It allows you to access properties and perform actions on

the ancestor element, providing more flexibility in your automation.

Steps:

1. Open UiPath Studio and create a new project.

2. Drag and drop “open browser” Activity in property panel under input there’s a browser type field select the browser in my case it is “chrome” & in URL field type "https://www.w3schools.com/tags/tryit.asp?filename=tryhtml_select".

3. Drag and drop the "Get Ancestor" activity from the activities panel onto your workflow.

4. Indicate the element on the screen that you want to find an ancestor for.

You can use the UIExplorer or the "Indicate Element" button to select the element. basically, click on selection area.

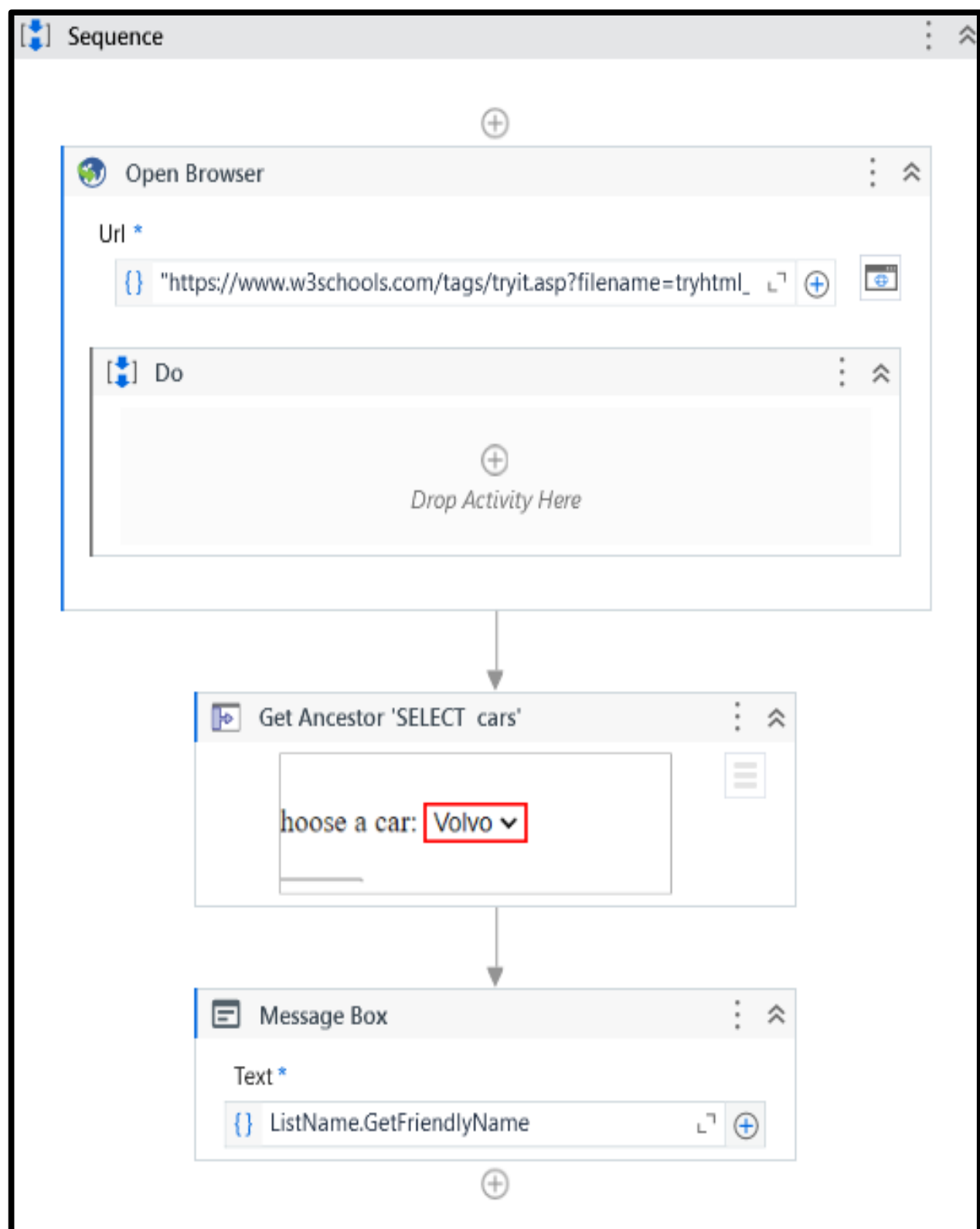
5. In the properties panel under field Ancestor create a “ListName” variable by pressing ctrl + k.

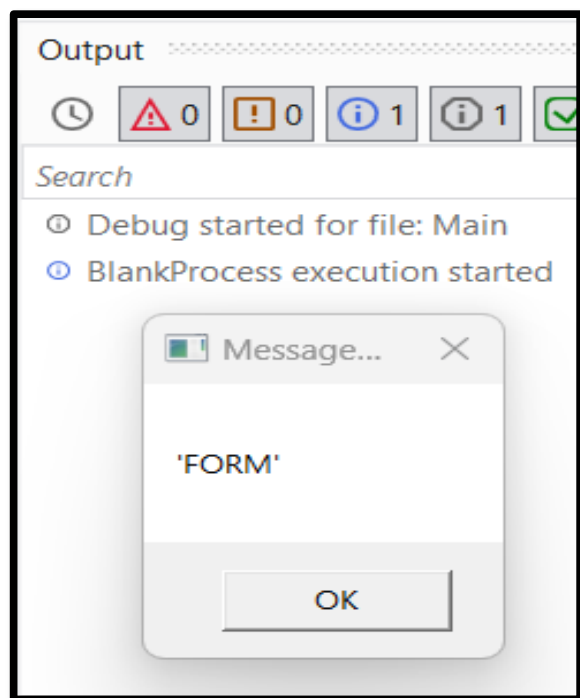
6. Drag and drop “message box” activity in text field type

“ListName.GetFriendlyName”.

7. Test your automation by running the workflow and see if the "Get Ancestor"

activity correctly retrieves the desired ancestor element.





f) Use Find Relative control

Theory:

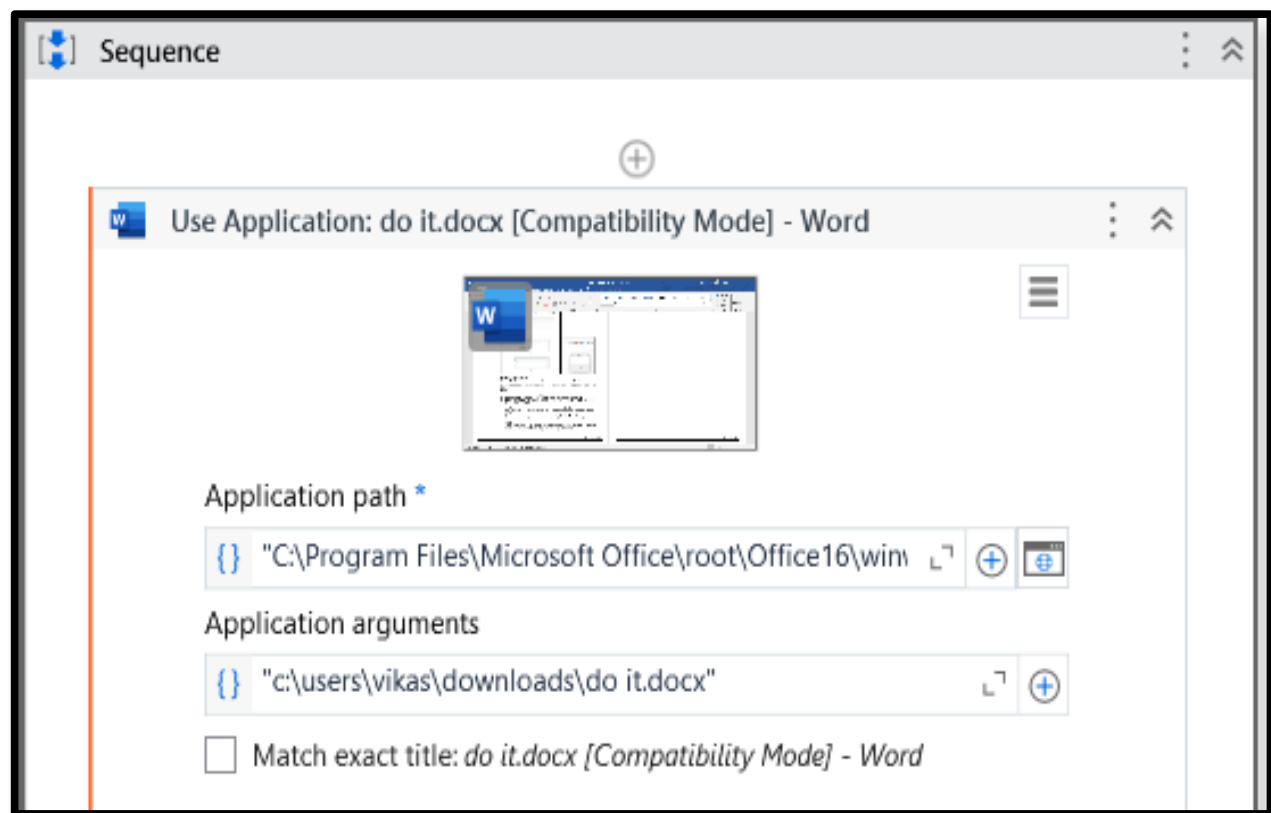
The "Find Relative" activity is useful when you want to locate a specific relative element of a given element.

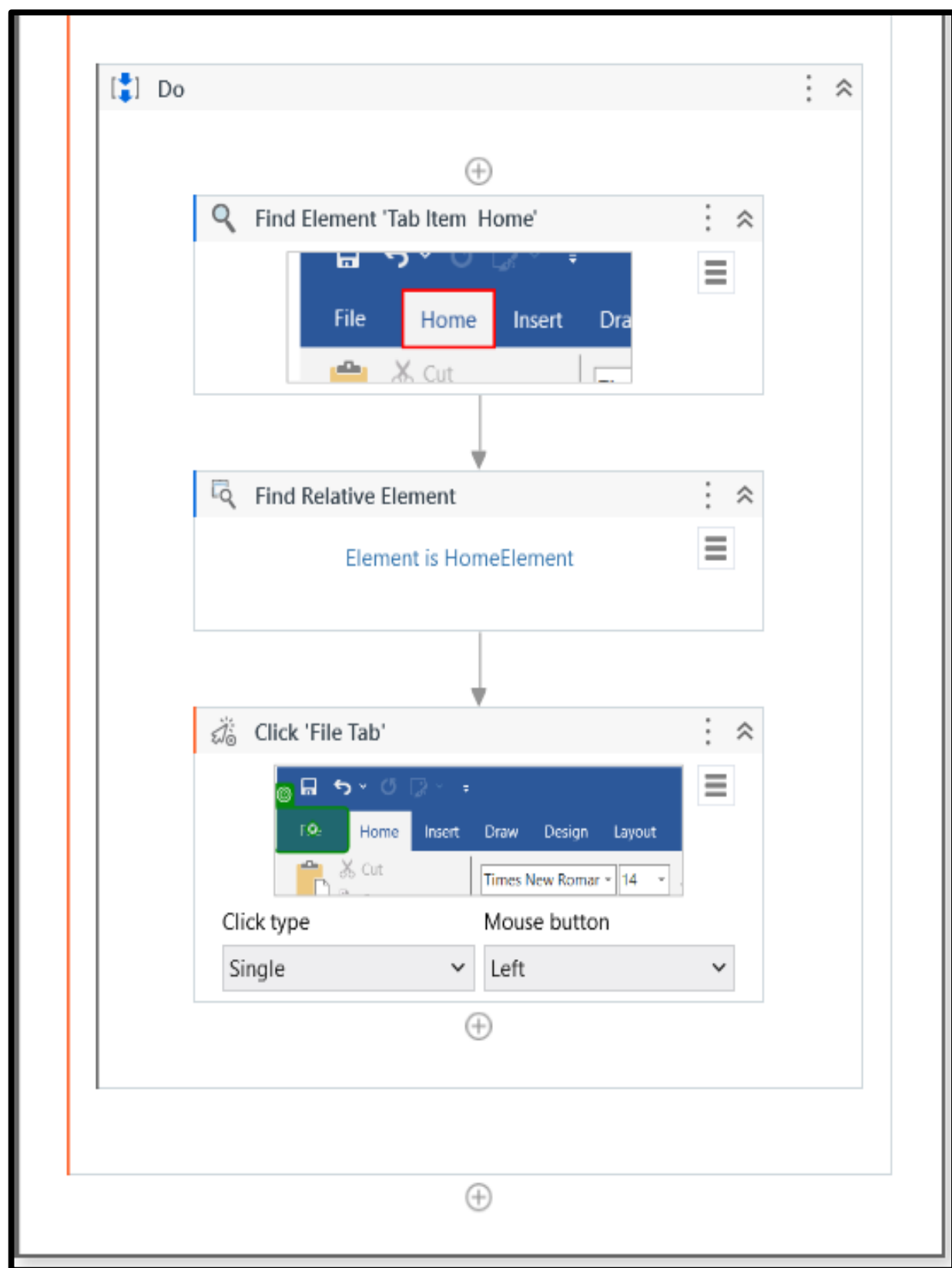
It allows you to perform actions on the relative element, providing more flexibility in your automation.

Steps:

1. Open UiPath Studio and create a new project.
2. Drag and drop “use application” activity from activity panel onto workflow
3. click on indicate on screen and select application in my case I had selected word which was running in background.
4. Drag and drop find element inside field Do
5. Then next Drag and drop the "Find Relative" activity from the activities panel into DO field.
6. Indicate the element on the screen that you want to find a relative for.
7. You can use the UIExplorer or the "Indicate Element" button to select the element.
8. Configure the properties of the "Find Relative" activity.

9. You can specify the relationship between the target element and the relative element you want to find, such as finding the sibling, parent, or child element.
10. Add a variable of the appropriate type to store the output of the "Find Relative" activity. For example, if you want to find the sibling element, create a variable of type "UiElement" to hold the sibling element.
11. Use the output variable in your automation to perform actions on the relative element. For example, you can click on a button within the sibling element or extract data from it.
12. Test your automation by running the workflow and see if the "Find Relative" activity correctly retrieves the desired relative element.





Output



Search

- ⌚ Debug started for file: Main
- ⌚ BlankProcess execution started
- ⌚ Audit: Using Desktop App. App: C:\Program Files\Microsoft Office\root\Office16\winword.exe Arguments: c:\users\vikas\downloads\do it.docx
- ⌚ BlankProcess execution ended in: 00:00:04

Practical No: 6

Keyboard and Mouse Events

a) Demonstrate the following activities in UiPath:

i. Mouse (click, double click and hover)

ii. Type into

iii. Type Secure text

❖ Mouse (click, double click, and hover):

- The Mouse activities in UiPath allow you to simulate mouse actions on the screen.
- The "Click" activity is used to simulate a single mouse click on a specified element or position on the screen.

- The "Double Click" activity is used to simulate a double mouse click on a specified element or position on the screen.
- The "Hover" activity is used to simulate moving the mouse cursor over a specified element on the screen without clicking.

❖ **Type Into:**

- The "Type Into" activity is used to simulate keyboard input by typing text into a specified text field, such as an input box or text editor.
- You can indicate the target element on the screen and then configure the activity to specify the text you want to type.

❖ **Type Secure Text:**

- The "Type Secure Text" activity is similar to the "Type Into" activity but is specifically designed for entering sensitive information, such as passwords.
- It encrypts the text you specify to protect it from being visible in the workflow or logs.

These activities are commonly used in automation to interact with applications, websites, or any user interface elements. They allow you to perform actions like clicking buttons, entering text, and handling sensitive information securely.

i. Mouse (click, double click and hover)

Steps:

1. Open UiPath Studio click on "New Project" to create a new automation project.

Give it a name and select the location where you want to save it.

2. Drag and drop “Use Application” activity on workflow because click and hover activity works only if it is under use application activity so now click on indicate on screen and configure it by clicking on desktop.
3. Drag and Drop Activities under DO field of “Use Application” activity: In the UiPath Studio, you will see a list of available activities on the left- hand side. Search for "Click," "Double Click," and "Hover" activities in the Activities panel.
4. Drag the Activities onto the Workflow: Drag and drop the "Click," "Double Click," and "Hover" activities onto the workflow canvas.
5. Indicate the Target Element: With the activities on the canvas, you need to indicate the target element on the screen. To do this, use the UiExplorer tool or the "Indicate on Screen" option.
6. Configure the Activities:

- For the "Click" activity, configure the options such as "Button" or "Link" and set the target element. In my case I have indicate it on “battery icon” in the taskbar after clicking it will show battery control panel.
- For the "Double Click" activity, set the target element just like in the "Click" activity. In my case I have indicate it on “you Tube icon” in the taskbar I have pinned the youtube to taskbar earlier after double clicking it will open 2 YouTube page in your browser because of double click.
- For the "Hover" activity, specify the target element to move the mouse cursor without clicking. In my case I have indicate it on “Weather icon” in the taskbar once mouse hover on weather icon it will show todays weather condition.

7. Run the Workflow: Once you have configured the activities, save the project and

click on the "Run" button to execute the workflow.

8. Verify the Actions: Observe the automation in action. The mouse will perform

the specified actions, such as clicking, double-clicking, or hovering over the

target elements.

Remember to test and adjust the selectors or other properties of the activities to ensure

they accurately identify the target elements on your application or webpage.

+

Use Application: Program Manager



Application path *

{}

"C:\Windows\explorer.exe"

⌵

+

⊞

Application arguments

{}

Text must be quoted

⌵

+

☐ Match exact title: Program Manager

+

Do

+

Click 'button YouTube'



Click type

Double

Mouse button

Left

↓

Hover '32°C Smoke 32°C Smoke'



Duration

{}

1

⌵

+

⊞

↓

Click '□'



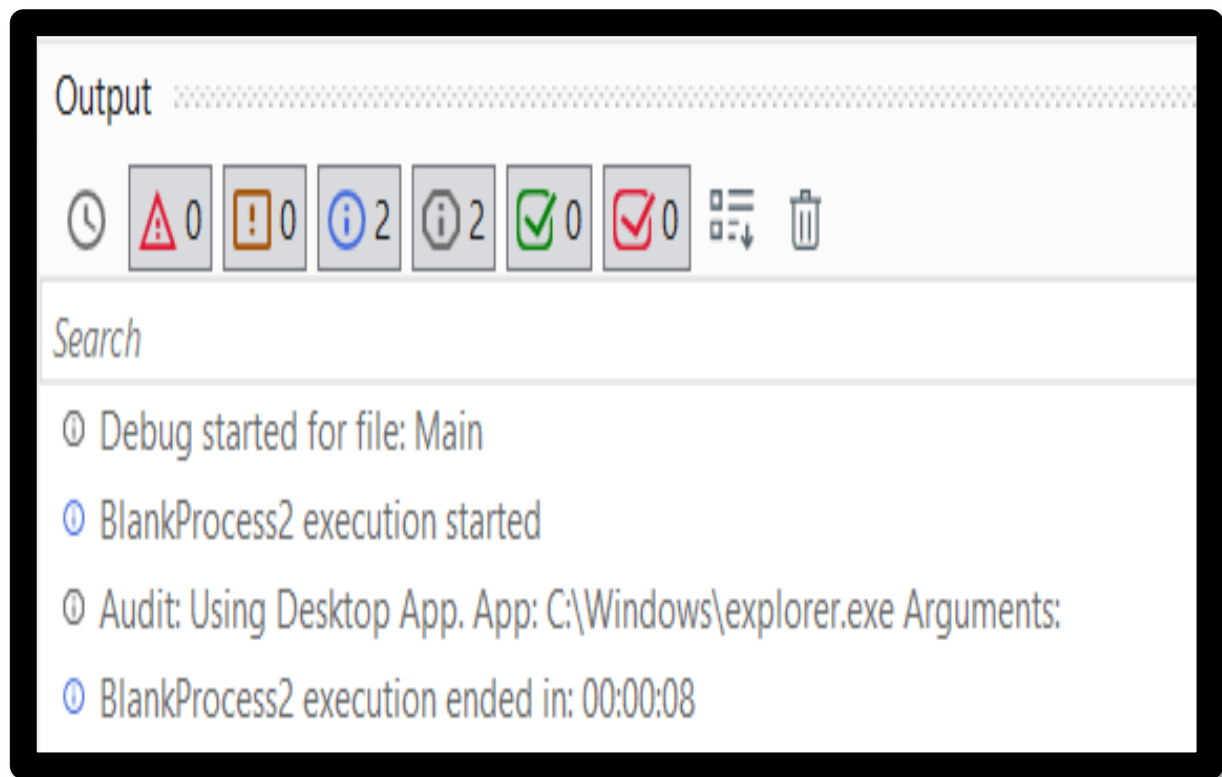
Click type

Single

Mouse button

Left

+



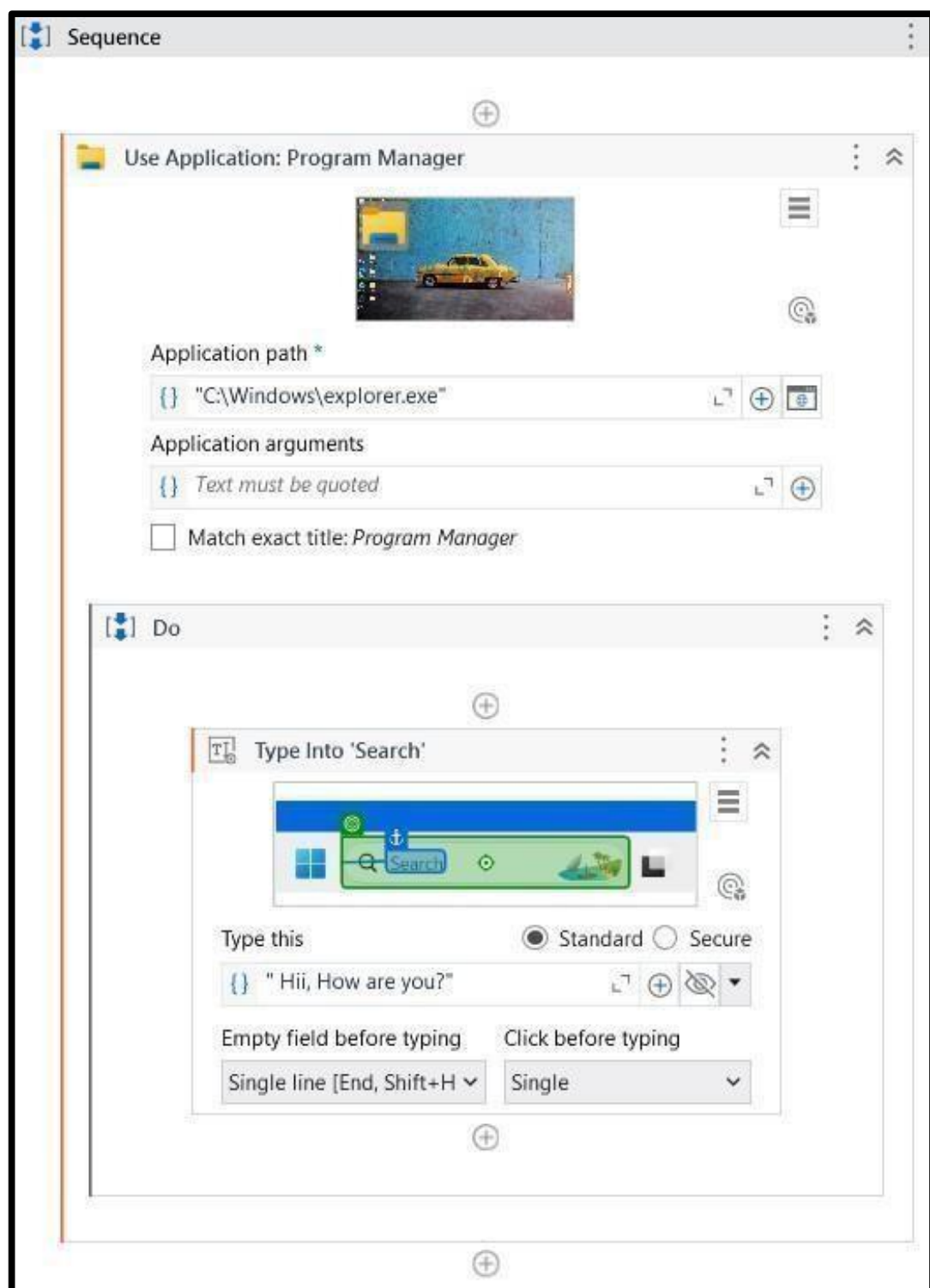
ii. Type into

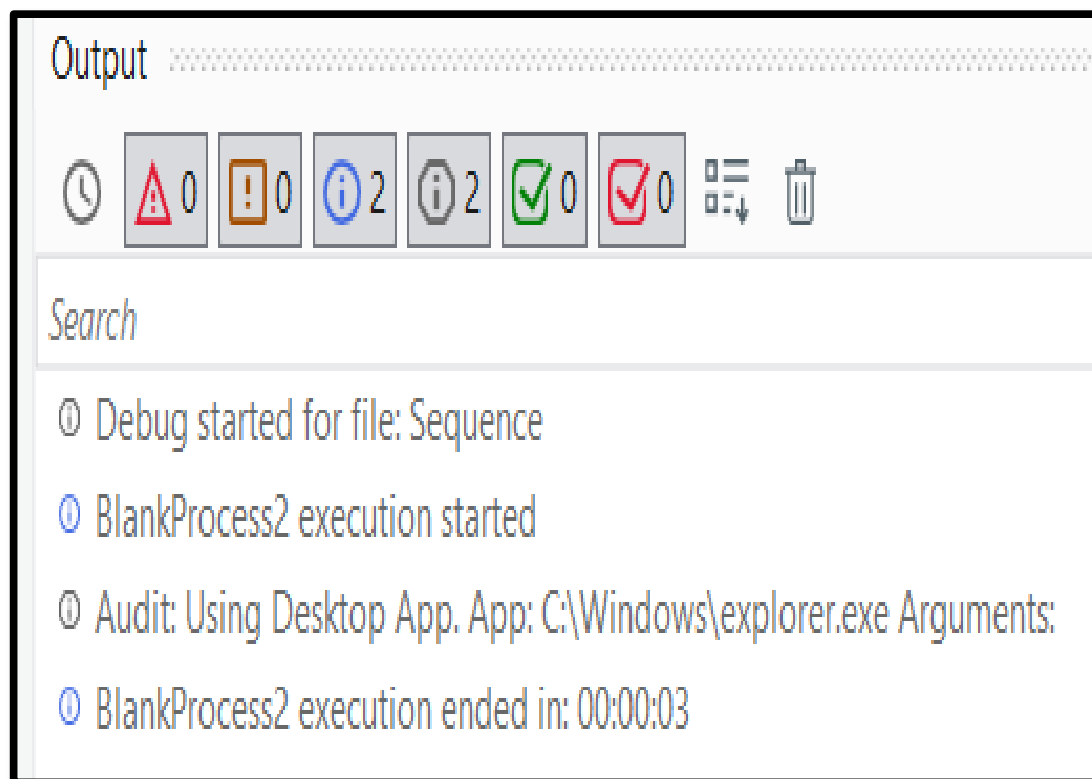
Steps:

1. Open UiPath Studio: Start by launching UiPath Studio on your computer.
2. Create a New Project: Click on "New Project" to create a new automation project.
3. Give it a name and select the location where you want to save it.

4. Drag and Drop Activities: In UiPath Studio, you'll find a list of available activities on the left-hand side. Look for the "Type Into" activity in the Activities panel.
5. Drag the Activity onto the Workflow: Drag and drop the "Type Into" activity onto the workflow canvas.
6. Indicate the Target Element: With the activity on the canvas, you need to indicate the target element on the screen where you want to type. To do this, use the UiExplorer tool or the "Indicate on Screen" option.
7. Configure the Activity: In the properties of the "Type Into" activity, you can specify the text you want to type into the target element. You can enter a static value or use variables to dynamically input data.
8. Run the Workflow: Once you have configured the activity, save the project and click on the "Run" button to execute the workflow.

9. Verify the Action: Observe the automation in action. The "Type Into" activity will type the specified text into the target element, such as an input field.

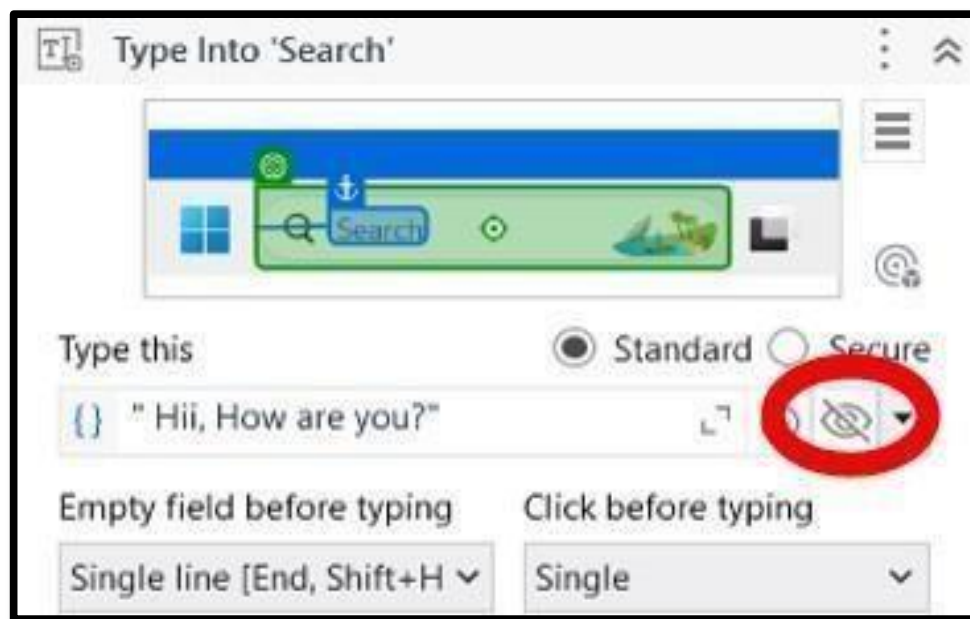




iii. **Type Secure text**

Steps:

1. There's a update in uipath studio earlier we have to use "Type Secure Text" activity for typing secured text such as password.
2. Now u just need to click on eye icon in Type into activity this will change to secured text "*****"



3. Run the above type into practical just make changes to “secured text” by clicking on eye Icon.

b) Demonstrate the following events in UiPath:

i. Element Triggering Event:

ii. Image Triggering Event:

iii. System Triggering Event:

✚ Element Triggering Event:

An element triggering event in UiPath is when a specific element on a webpage or application triggers an action. For example, let's say you want to automate the process of submitting a form on a website. You can use an element triggering event to simulate a click on the "Submit" button. When that button is clicked, it triggers the action of submitting the form. You can use activities like "Click" or "Type Into" to interact with the elements on the screen and perform actions based on those triggers.

Image Triggering Event:

An image triggering event in UiPath is when a specific image on the screen triggers an action. This is useful when you want to automate a process based on the presence or absence of a particular image.

For example, let's say you want to automate the process of checking if a certain icon appears on the screen. You can use an image triggering event to search for that icon and perform actions based on its presence or absence.

The "Click Image" activity is commonly used for image triggering events.

System Triggering Event:

A system triggering event in UiPath is when an external system or application triggers an action in your automation. This can be useful when you want to automate tasks that require interaction with other applications. For example, let's say you want to automate the process of sending an email. You can use a system triggering event to launch your email client and

perform actions like composing and sending the email. The "Start Process" activity is commonly used for system triggering events.

These events allow us to create more dynamic and interactive automations by responding to specific triggers.

i. Element Triggering Event: Steps:

1. Open UiPath Studio and create a new project.
2. Drag and drop a "Click" activity from the Activities panel onto the workflow canvas. This activity simulates a click on an element.
3. Indicate the element on the screen that you want to trigger the event by clicking on it.

4. Drag and drop the activities you want to perform when the element is triggered.

For example, you can add activities to fill out a form or navigate to another page.

5. Save your workflow and run the automation.
6. The specified actions will be performed when the element triggering event occurs.

ii. Image Triggering Event: Steps:

1. Open UiPath Studio and create a new project.
2. Drag and drop a "Click Image" activity from the Activities panel onto the workflow canvas. This activity searches for a specific image on the screen and clicks on it.

3. Configure the properties of the "Click Image" activity by specifying the image you want to search for. You can use the "Indicate on Screen" option to select the image.
4. Drag and drop the activities you want to perform when the image is triggered.

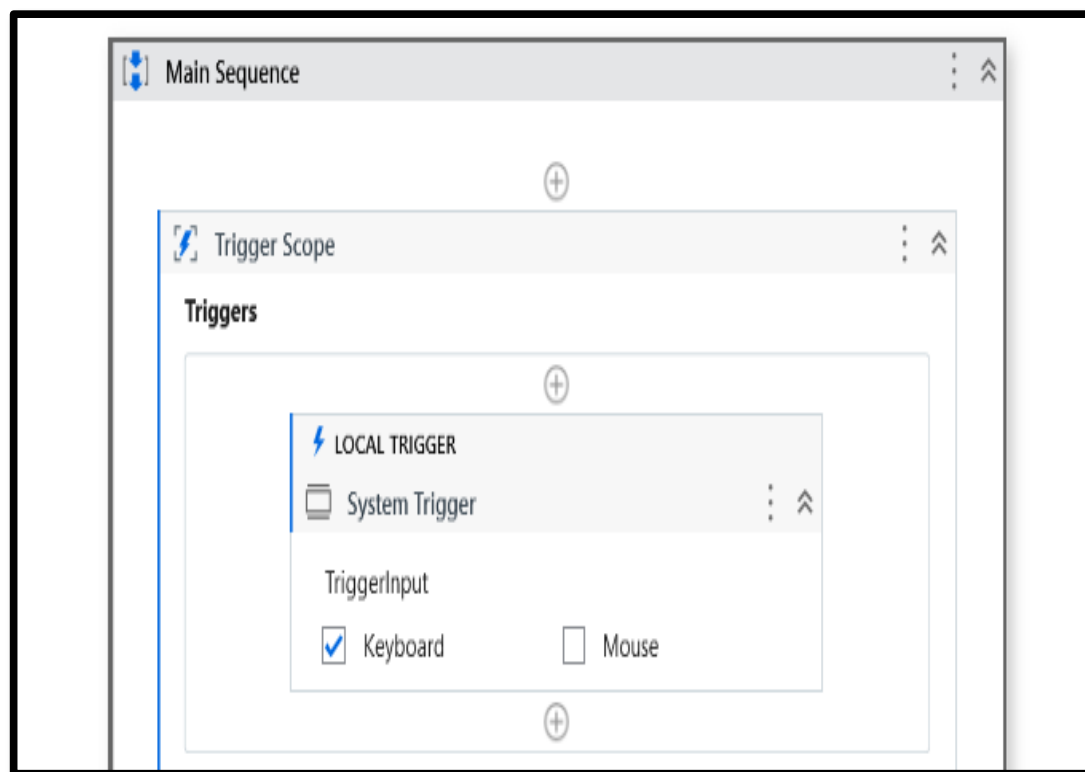
For example, you can add activities to extract data from a specific screen or perform certain actions.
5. Save your workflow and run the automation.
6. The specified actions will be performed when the image triggering event occurs.

iii. System Triggering Event:

Steps:

1. Open UiPath Studio and create a new project.

2. Drag and drop a "Start Process" activity from the Activities panel onto the workflow canvas. This activity launches an external application or system process.
3. Configure the properties of the "Start Process" activity by specifying the file path or executable of the system process you want to trigger.
4. Drag and drop the activities you want to perform when the system triggering event occurs. For example, you can add activities to interact with the launched application or perform certain actions.
5. Save your workflow and run the automation.
6. The specified actions will be performed when the system triggering event occurs.



Actions

EventArgs

args

[↕] Sequence



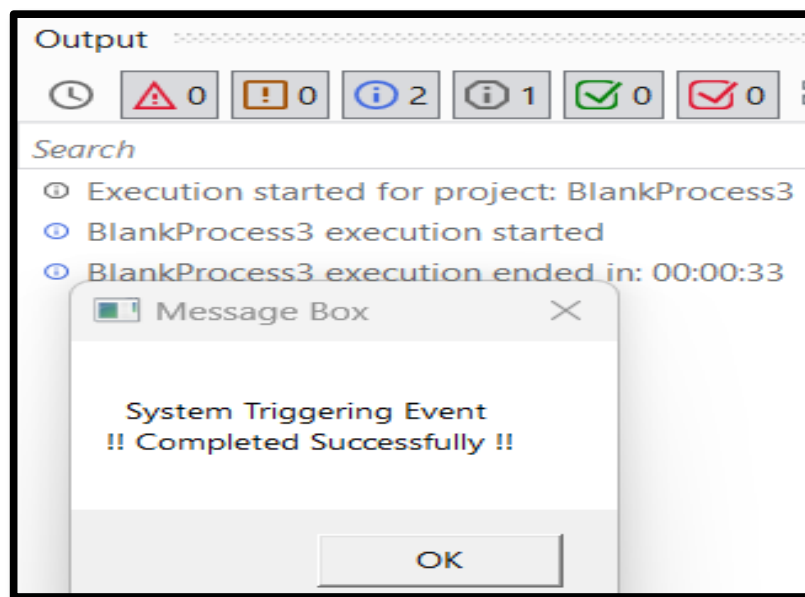
[≡] Message Box



Text *

{ } " System Triggering Event





c) Automate the process of launching an assistant bot on a keyboard event.

- Keyboard events can be used as a trigger to perform certain actions or launch specific processes. In UiPath, you can capture keyboard events using the "Get Key" activity, which allows you to specify the type of key event (such as key down or key up) and the specific key that will trigger the automation. By connecting the "Get Key" activity to other actions or processes, you can automate tasks based on keyboard input.

This practical demonstrates how to automate the process of launching an assistant bot on a keyboard event in UiPath for beginners. Here's a simplified guide:

Steps:

1. Open UiPath Studio and create a new project.
2. Drag and drop a "Get Key" activity from the Activities panel onto the workflow canvas. This activity will allow you to capture keyboard events.
3. Double-click on the "Get Key" activity to open its properties.
4. In the properties panel, select the type of key event you want to trigger the launch of the assistant bot. For example, you can choose "Key Down" or "Key Up".

5. In the "Key" property, specify the key you want to use as the trigger. For example,

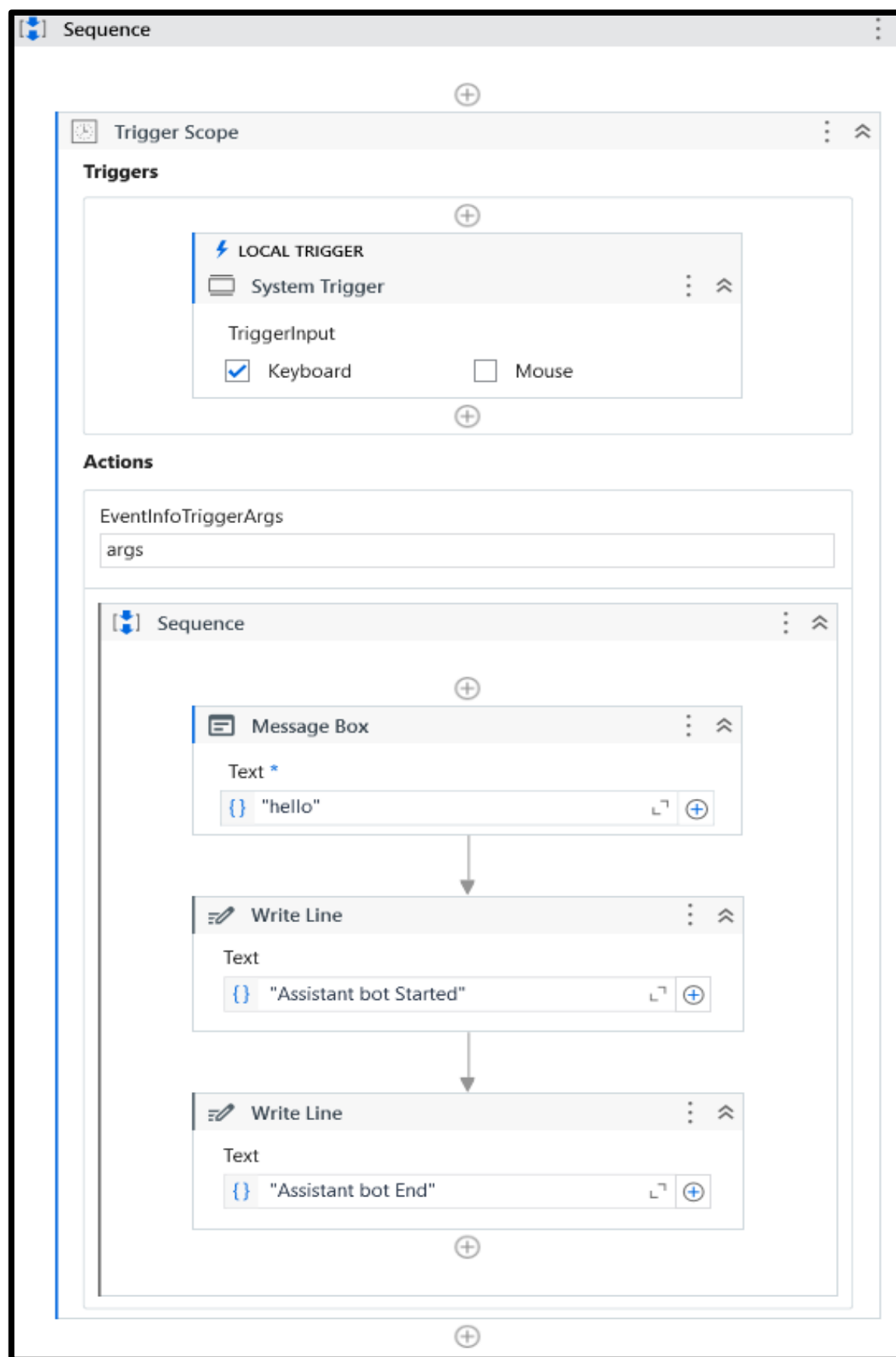
you can enter "Enter" if you want the bot to launch when the Enter key is pressed.
6. Drag and drop a "Start Process" activity from the Activities panel onto the workflow

canvas. This activity will launch the assistant bot.
7. Configure the "Start Process" activity by specifying the file path or executable

of the assistant bot you want to launch.
8. Connect the "Get Key" activity to the "Start Process" activity by dragging a

connection between them.
9. Save your workflow and run the automation.
10. Test the automation by pressing the specified key. The assistant bot

should launch when the key event is triggered.



Output



Search

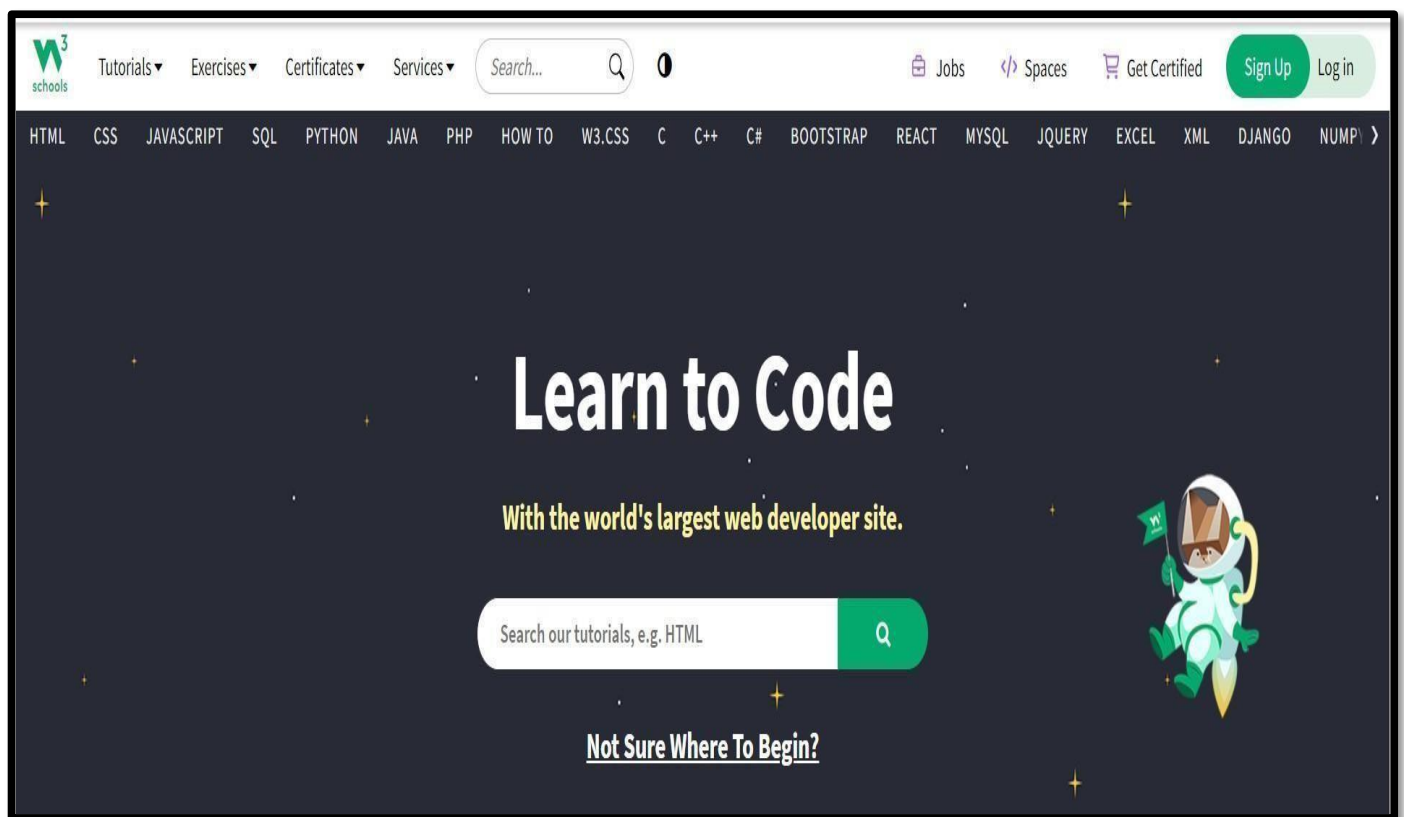
- ⓘ Debug started for file: Main
- ⓘ BlankProcess execution started
- ⓘ Assistant bot Started
- ⓘ Assistant bot End
- ⓘ BlankProcess execution ended in: 00:00:21

Practical No: 7

Screen Scraping and Web Scraping methods

a) Automate the following screen scraping methods using UiPath: Dataset

[UiElement from which the data to be extracted.]:



i. Full Text

- In this practical we'll learn about how to automate screen scraping using the Full Text method using UiPath.

Steps:

1. Open UiPath Studio and create a new project.
2. Drag and drop the "Attach Browser" activity onto the workflow canvas.
3. This activity allows you to specify the web browser you want to scrape data from.
4. Indicate the browser window you want to scrape by using the "Indicate on Screen" option in the properties panel of the "Attach Browser" activity. This will open a selector editor.
5. Use the selector editor to highlight and select the specific browser window you want to scrape data from.

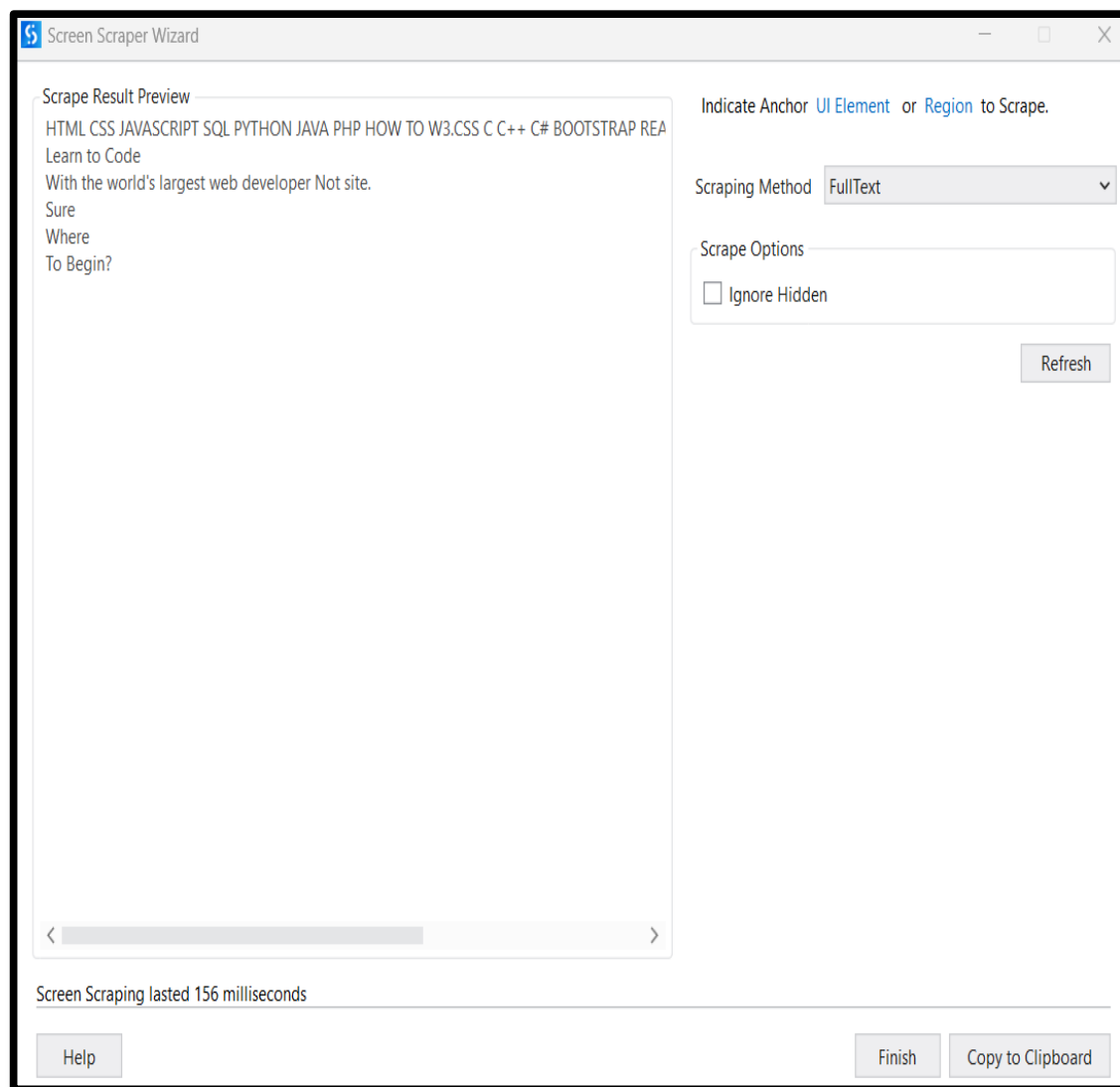
6. The selector is used to identify the target element on the screen.
7. Drag and drop a "Get Full Text" activity inside the "Attach Browser" activity.
8. This activity is used to extract the full text from a specified element on the screen.
9. In the properties panel of the "Get Full Text" activity, indicate the target element by using the "Indicate on Screen" option. This will open a selector editor.
10. Use the selector editor to highlight and select the specific element from which you want to extract the full text.
11. The selector should uniquely identify the target element.
1. Optionally, you can specify the timeout and retry options in the properties panel of the "Get Full Text" activity.

2. These options control how long UiPath waits for the element to appear and how many times it retries before throwing an error.
3. Add an output variable to store the extracted text. In the properties panel of the "Get Full Text" activity,
4. create a new variable of type String and assign it to the "Output" property.
5. You can use the extracted text in subsequent activities.

For example, you can write the text to a file, store it in a variable, or perform further processing on it.

6. Run the workflow to automate the screen scraping using the Full Text method.
7. UiPath will execute the activities and extract the full text from the specified element on the screen.

Output:



ii. Native

- Screen scraping using the Native method in UiPath.

Steps:

1. Open UiPath Studio and create a new project.

2. Drag and drop the "Attach Window" activity onto the workflow canvas.

This activity allows you to specify the application window you want to scrape data from.

3. Indicate the window you want to scrape by using the "Indicate on Screen" option in the properties panel of the "Attach Window" activity. This will open a selector editor.

4. Use the selector editor to highlight and select the specific window you want to scrape data from.

5. The selector is used to identify the target window.

6. Drag and drop a "Get Text" activity inside the "Attach Window" activity.

7. This activity is used to extract the text from a specified element within the window.

8. In the properties panel of the "Get Text" activity, indicate the target element by using the "Indicate on Screen" option. This will open a selector editor.
9. Use the selector editor to highlight and select the specific element from which you want to extract the text.
10. The selector should uniquely identify the target element.
11. Optionally, you can specify the timeout and retry options in the properties panel of the "Get Text" activity.
12. These options control how long UiPath waits for the element to appear and how many times it retries before throwing an error.
13. Add an output variable to store the extracted text. In the properties panel of the "Get Text" activity,
14. create a new variable of type String and assign it to the "Output" property.

15. You can use the extracted text in subsequent activities.

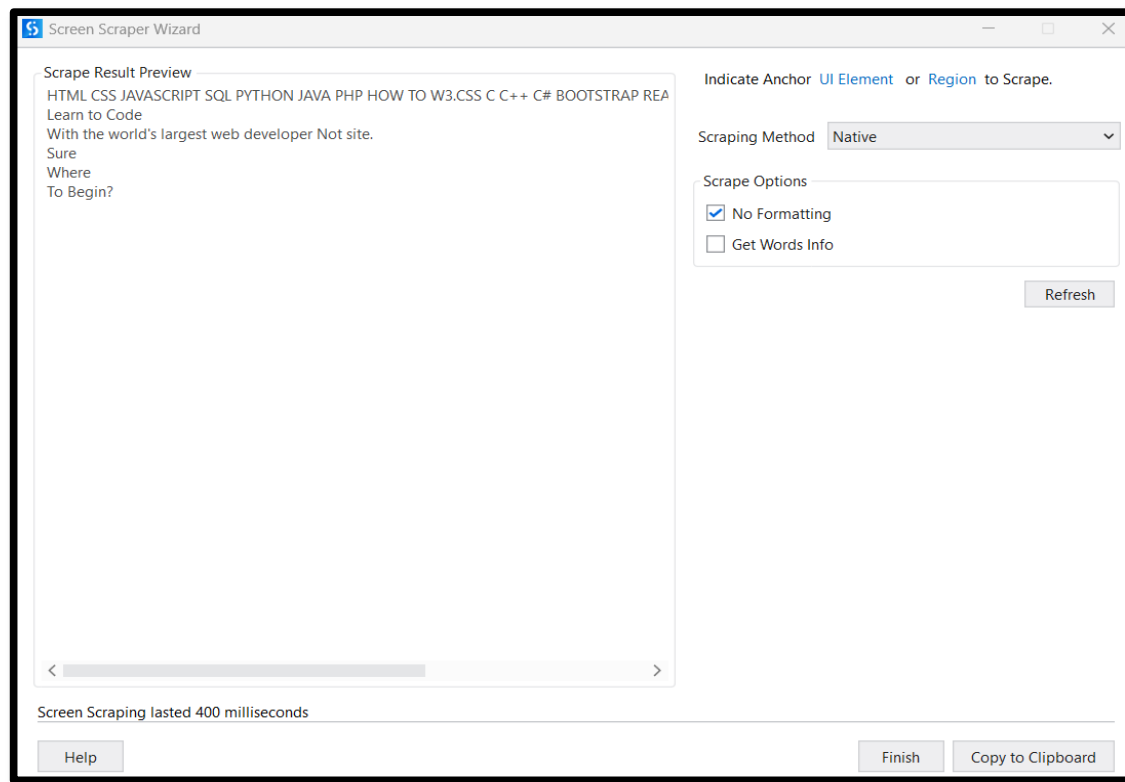
For example, you can write the text to a file, store it in a variable, or perform further processing on it.

16. Run the workflow to automate the screen scraping using the Native method.

17. UiPath will execute the activities and extract the text from the specified element within the window.

Remember to customize the selectors based on the specific application you are working with. Selectors should be robust and unique to ensure accurate scraping.

Output:



iii. OCR

➤ In this practical we will perform Automate screen scraping using OCR (Optical Character Recognition) in UiPath

Steps:

1. Open UiPath Studio and create a new project.

2. Drag and drop the "Attach Window" activity onto the workflow canvas.

This activity allows you to specify the application window you want to scrape data from.

3. Use the "Indicate on Screen" option in the properties panel of the "Attach Window" activity to select the window you want to scrape. This helps UiPath identify the target window.

4. Drag and drop an "OCR Text Recognition" activity inside the "Attach Window" activity. This activity is used to extract text from images or elements that cannot be directly accessed.

5. Use the "Indicate on Screen" option in the properties panel of the "OCR Text Recognition" activity to select the area or element you want to perform OCR on. This helps UiPath identify the target area.

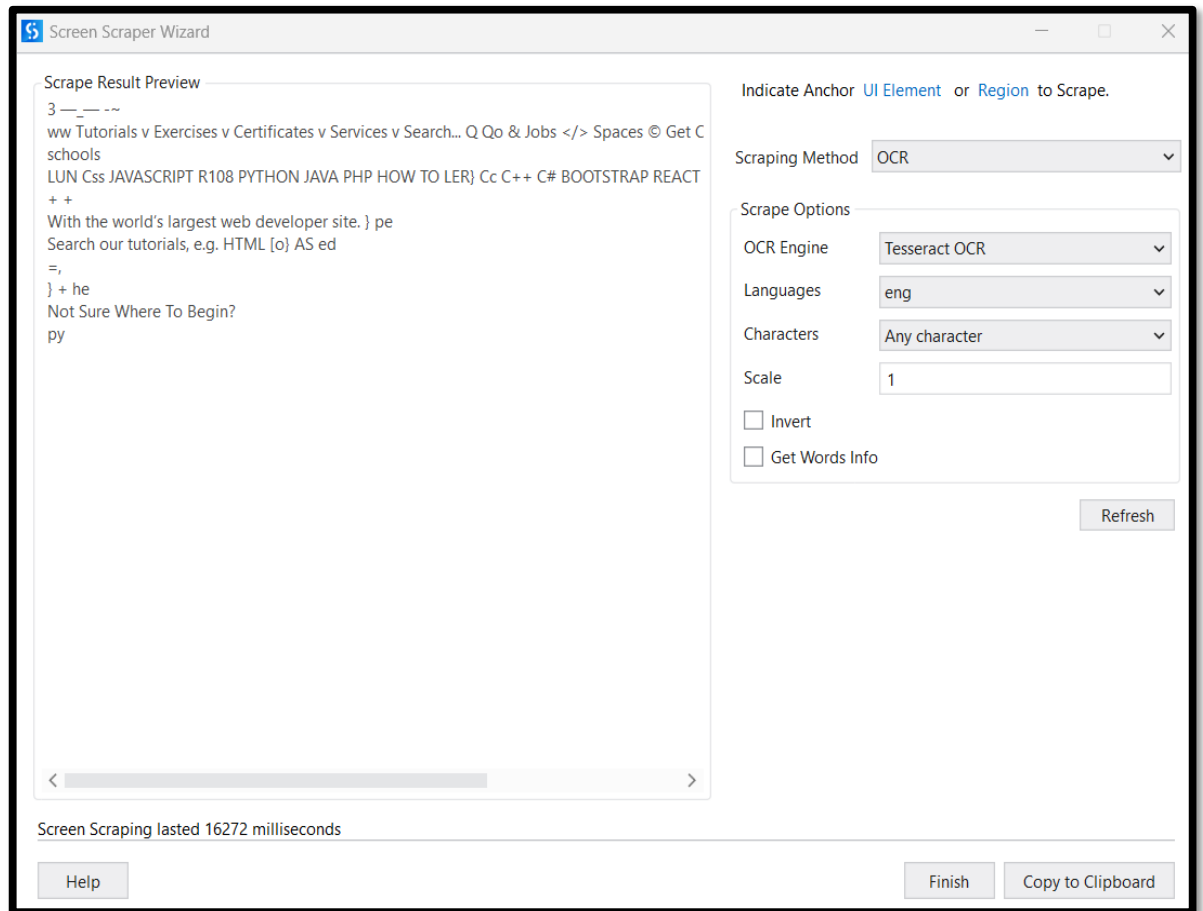
6. Configure the OCR engine by selecting the appropriate OCR provider (such as Microsoft OCR, Google OCR, or Abbyy OCR) in the properties panel of the "OCR Text Recognition" activity.
7. Create a variable of type String to store the extracted text. Assign this variable to the "Output" property of the "OCR Text Recognition" activity.
8. Customize the OCR options in the properties panel of the "OCR Text Recognition" activity if needed. These options include language selection, scale, and OCR engine-specific settings.
9. You can use the extracted text in subsequent activities. For example, you can write the text to a file, store it in a variable, or perform further processing on it.

10. Run the workflow to automate the screen scraping process using OCR.

UiPath will execute the activities and extract the text from the specified area or element using OCR.

Remember that OCR accuracy depends on factors like image quality, font type, and language. Adjust the OCR settings and experiment with different OCR engines to achieve the desired results.

Output:



b) Demonstrate Data Scraping and display values in Message box.

The aim is to extract text and images from a PDF File and store the output in a Message Box.

Follow the below steps to achieve the aim:

- To extract Text only use the Read PDF Text activity and use a message box to display the output.
- To extract Text inside images, use the Read PDF with OCR activity and use a message box to display the output.

Solution

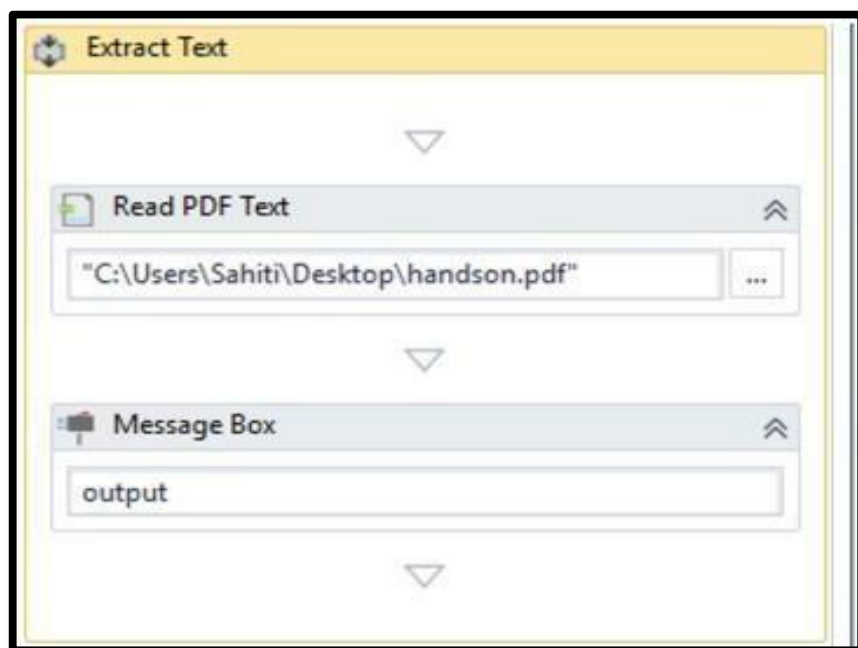
Step 1: Follow the below steps to extract Text only from PDF documents. **Step 1.1:**

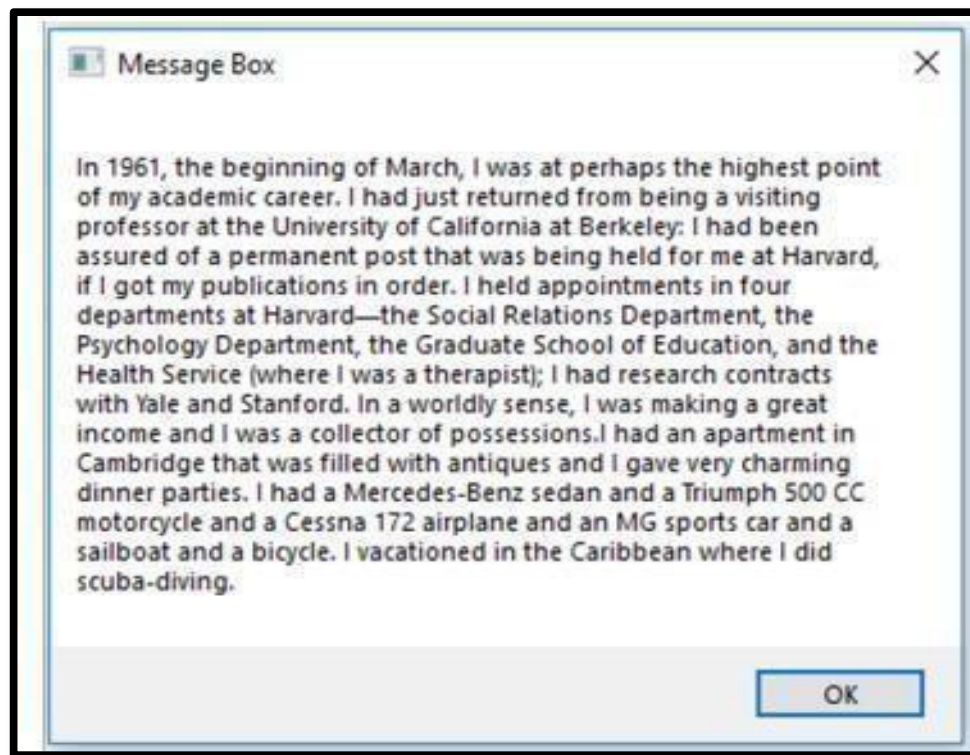
Drag the Read PDF Text Activity. In the activity, mention the path of the PDF Document from which data has to be extracted.

Step 1.2: Now, in the Properties Pane, of the Read PDF Text Activity, mention an output variable to see the output. To set an output variable, press on CTRL + K, and give a name. Here I have mentioned as output.

Step 1.3: After that, drag a message box in the same sequence and mention the output variable in it.

Step 1.4: Execute this sequence by clicking on the Run button. Your complete sequence and the output should look as shown in the below snapshots, respectively.





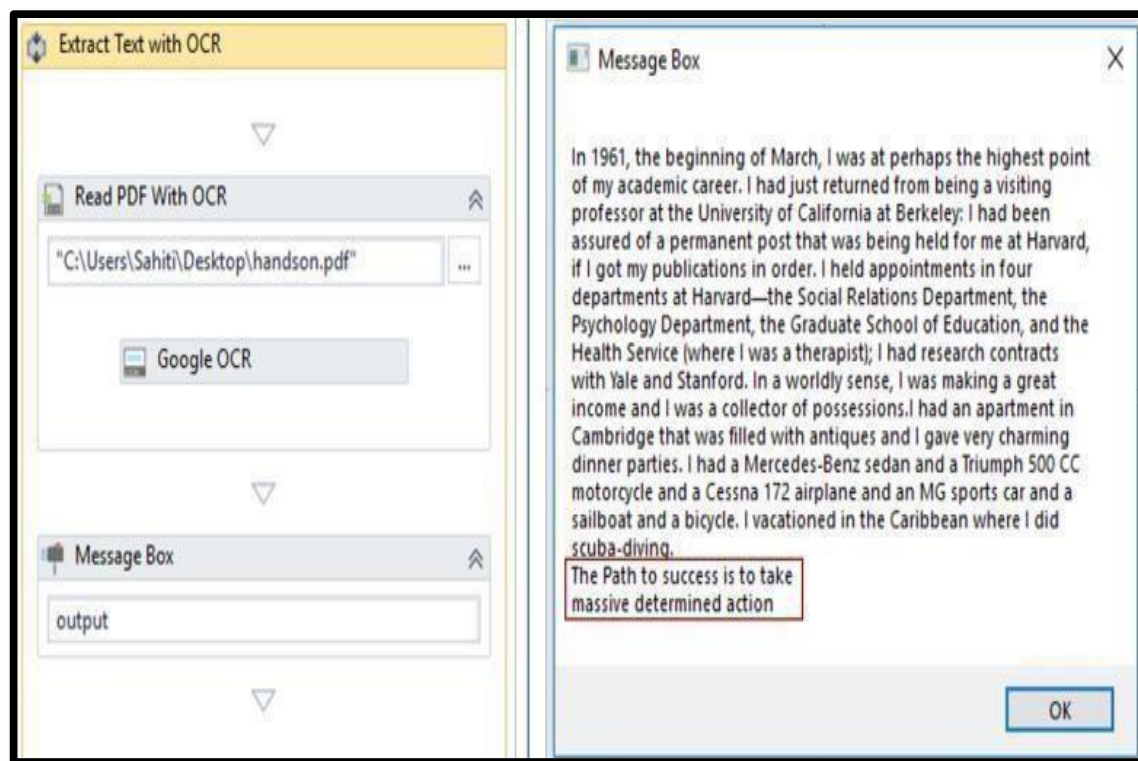
Step 2: Now, similarly if you want to extract the text in the images follow the below steps.

Step 2.1: Drag and drop the Read PDF with OCR Activity. In the activity, mention the path of the PDF Document from which data has to be extracted. **Step 2.2:** Now, search for an OCR Engine, and drag and drop an OCR Engine based on whichever is installed. Here I have used Google OCR Engine.

Step 2.3: Now, in the Properties Pane, of the Read PDF with OCR Activity, mention an output variable to see the output. To set an output variable Press on CTRL + K, and give a name. Here I have mentioned as output.

Step 2.4: After that, drag a message box in the sequence and then mention the output variable in it.

Step 2.5: Execute this sequence by clicking on the Run button. Your complete sequence and the output should look as shown in the below snapshots, respectively



c) Demonstrate Screen Scraping for a pdf, web page and image file.

Follow the below steps to achieve the aim:

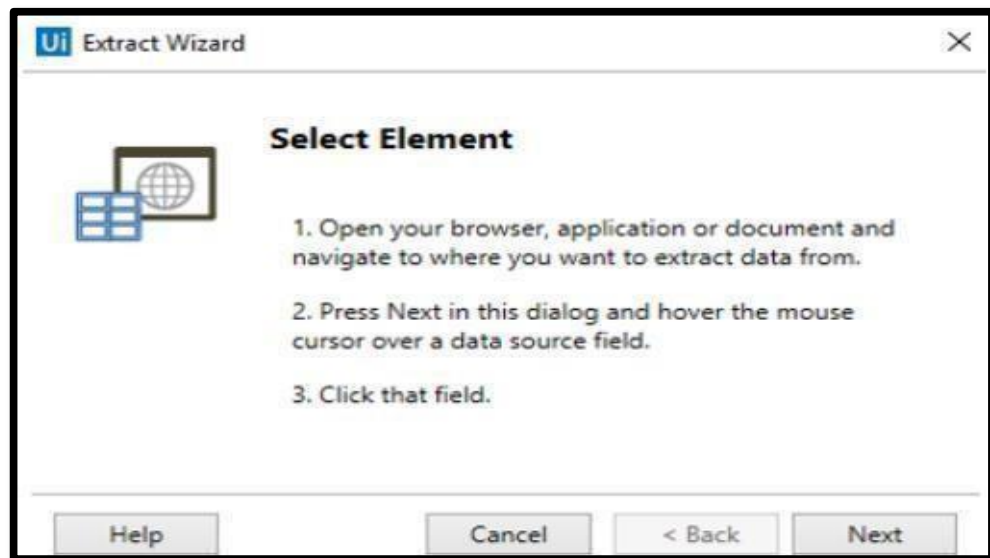
- Use the Data Scraping tool and record the data that you want to extract.
- Extract the correlated values accordingly.
- Use the Write CSV activity and mention the path of the .csv file

Solution

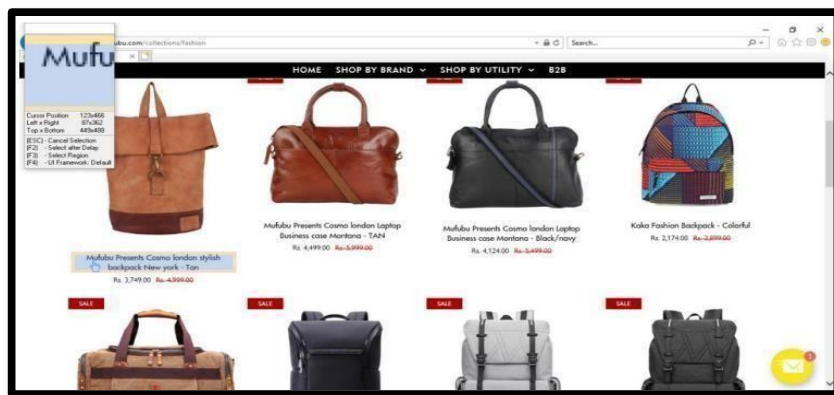
Step 1: Choose the website from where you want to extract data.

“In HOME TAB->Setting Toggle the Modern look for new Project to off” **Step 2:**

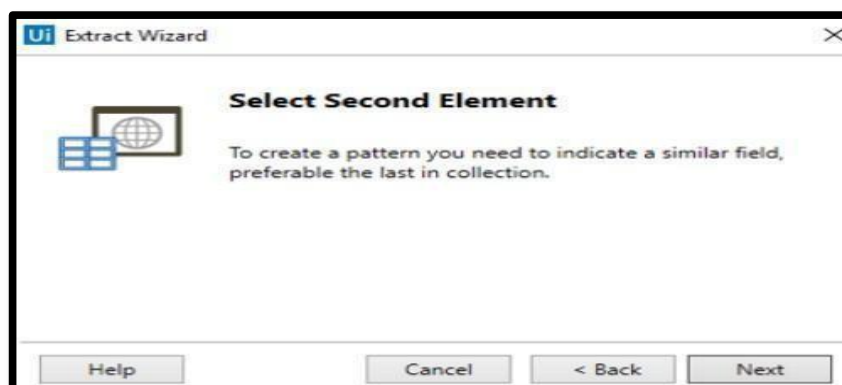
Choose the Data Scraping option from the Ribbon and select the element you want to choose. Press Next in the below Dialog Box.



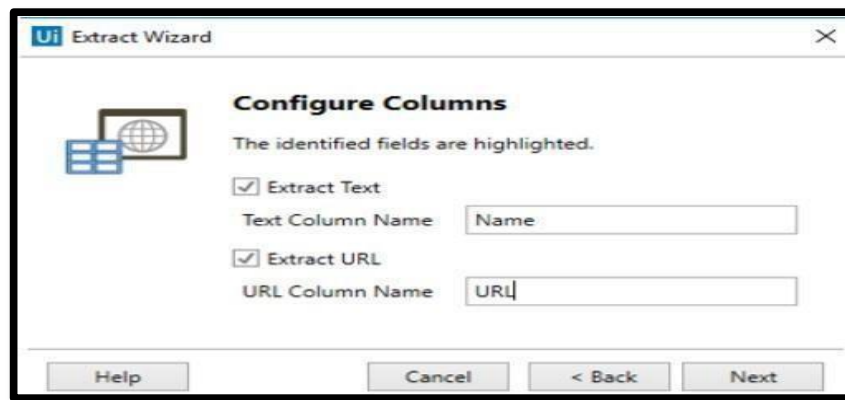
Step 2.1: Hover the mouse over a data source field and then click on the data source field



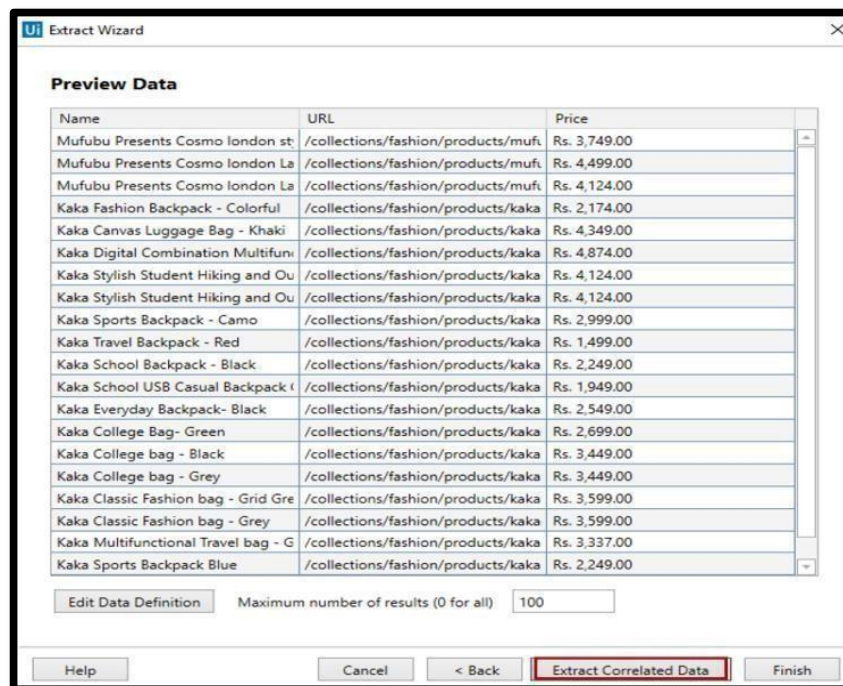
Step 3: After that you will see another Dialog box which will ask you to choose the second element to create a pattern.



Step 4: Once you choose the second element to create a pattern, you will get an option to Configure Columns. In the Dialog Box, that opens up, you can rename the Column name and extract URL also. After that click on Next. Refer Below.



Step 5: You would see the below output. Now, to extract other data sources from the website, click on the Extract Correlated Data option and repeat the above steps.



Step 6: Once you are done extracting all the required data, click on Finish. This action would open a dialog box which will ask you if you wish to span the data across multiple pages. Refer below.



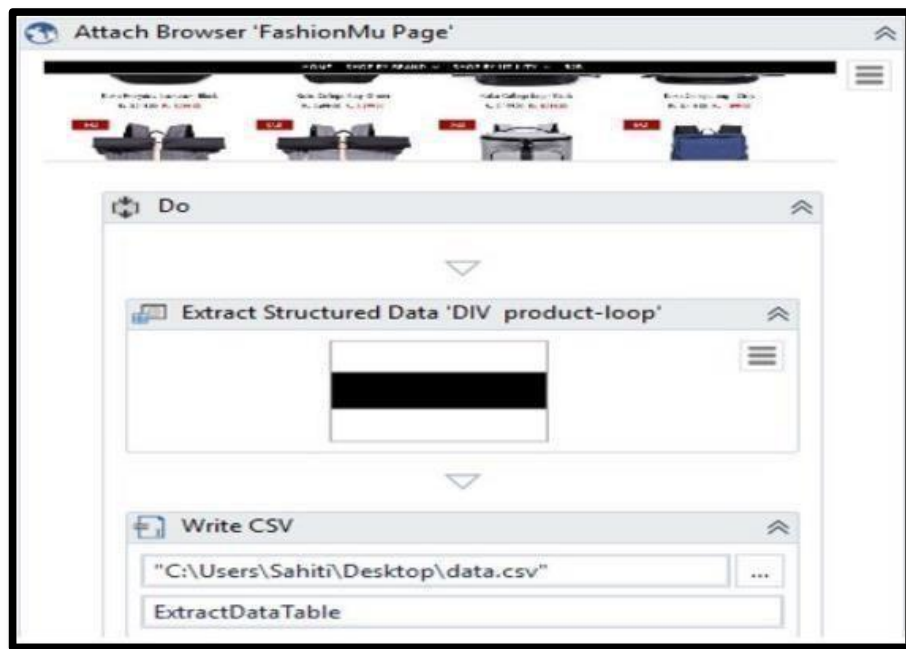
Step 7: To span across multiple pages, choose Yes and hover the mouse over the region which redirects you to the next page. Then you will be redirected to your UiPath Dashboard. Refer below.

Showing items 1-20 of 25.



Step 8: Now to store all the extracted data into a .csv file, drag a Write CSV activity into the Do section of the Data Scraping.

Step 9: In the File Path section of this activity, mention the path of the .csv file where you want to store the extracted data and then mention the ExtractDataTable variable in the DataTable section. Refer below.



NOTE: The ExtractDataTable variable is the output variable that is automatically generated from the Data Scraping Wizard. You can find this variable in the Extract Structured Data activity.

Step 10: Execute this sequence by clicking on the Run button. You would see that the data is getting stored in the .csv file as below.

Name	URL	Price
Mufubu Presents Cosmo london stylish backpack New y	/collections/fashion/products/mufubu-	Rs. 3,749.00
Mufubu Presents Cosmo london Laptop Business case N	/collections/fashion/products/mufubu-	Rs. 4,499.00
Mufubu Presents Cosmo london Laptop Business case N	/collections/fashion/products/mufubu-	Rs. 4,124.00
Kaka Fashion Backpack - Colorful	/collections/fashion/products/kaka-fas	Rs. 2,174.00
Kaka Canvas Luggage Bag - Khaki	/collections/fashion/products/kaka-car	Rs. 4,349.00
Kaka Digital Combination Multifunctional Anti Theft Ba	/collections/fashion/products/kaka-wa	Rs. 4,874.00
Kaka Stylish Student Hiking and Outdoor Student Backp	/collections/fashion/products/kaka-sty	Rs. 4,124.00
Kaka Stylish Student Hiking and Outdoor Student Backp	/collections/fashion/products/kaka-sty	Rs. 4,124.00
Kaka Sports Backpack - Camo	/collections/fashion/products/kaka-spc	Rs. 2,999.00
Kaka Travel Backpack - Red	/collections/fashion/products/kaka-tra	Rs. 1,499.00
Kaka School Backpack - Black	/collections/fashion/products/kaka-sch	Rs. 2,249.00
Kaka School USB Casual Backpack Colour Blue	/collections/fashion/products/kaka-lap	Rs. 1,949.00
Kaka Everyday Backpack- Black	/collections/fashion/products/kaka-ev	Rs. 2,549.00
Kaka College Bag- Green	/collections/fashion/products/kaka-stu	Rs. 2,699.00
Kaka College bag - Black	/collections/fashion/products/kaka-fas	Rs. 3,449.00
Kaka College bag - Grey	/collections/fashion/products/kaka-fas	Rs. 3,449.00
Kaka Classic Fashion bag - Grid Grey	/collections/fashion/products/kaka-cla	Rs. 3,599.00
Kaka Classic Fashion bag - Grey	/collections/fashion/products/kaka-cla	Rs. 3,599.00
Kaka Multifunctional Travel bag - Grey	/collections/fashion/products/kaka-mu	Rs. 3,337.00
Kaka Sports Backpack Blue	/collections/fashion/products/kaka-spc	Rs. 2,249.00
Kaka College USB bag - Blue	/collections/fashion/products/kaka-sty	Rs. 3,599.00

Practical No: 8

PDF Automation and Exception Handling

a) Read PDF With OCR Steps:

1. Drag and drop the Read PDF with OCR Activity. In the activity, mention the path of the PDF Document from which data has to be extracted.
2. Now, search for an OCR Engine, and drag and drop an OCR Engine based on whichever is installed. Here I have used Google OCR Engine.
3. Now, in the Properties Pane, of the Read PDF with OCR Activity, mention an output variable to see the output. To set an output variable

Press on CTRL + K, and give a name. Here I have mentioned as output.

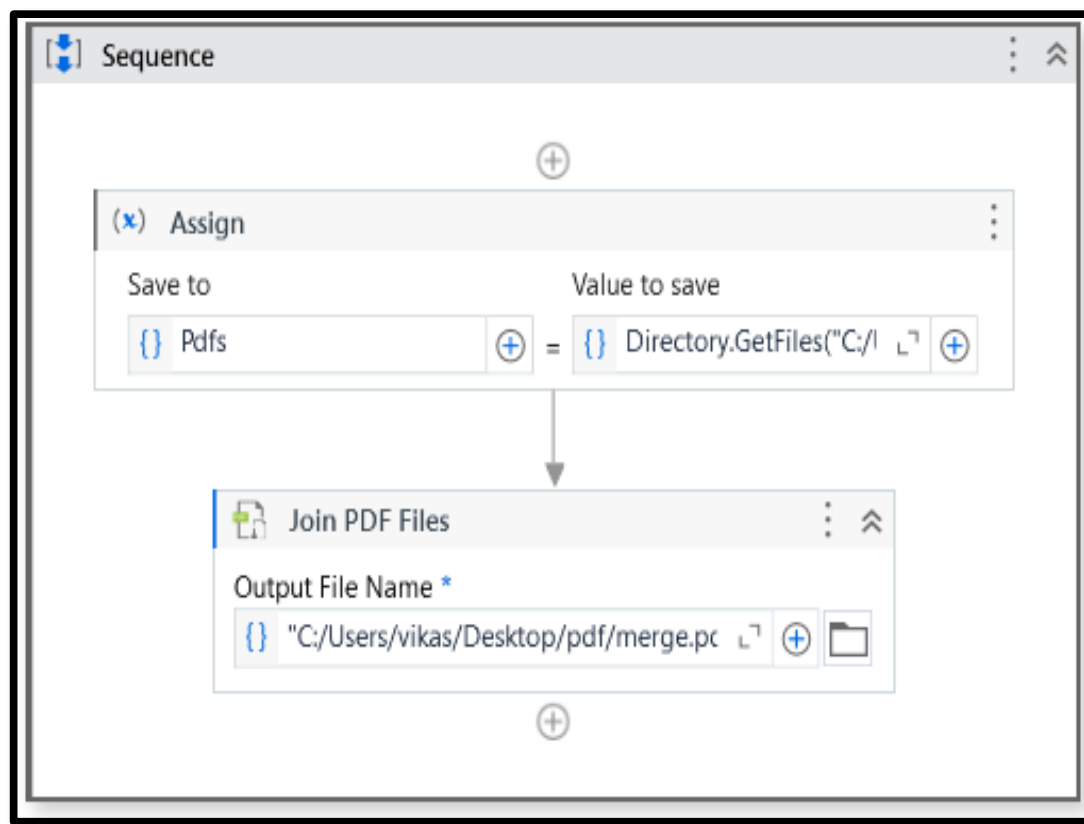
4. After that, drag a message box in the sequence and then mention the output variable in it.

5. Execute this sequence by clicking on the Run button. Your complete sequence and the output should look as shown in the below snapshots, respectively.

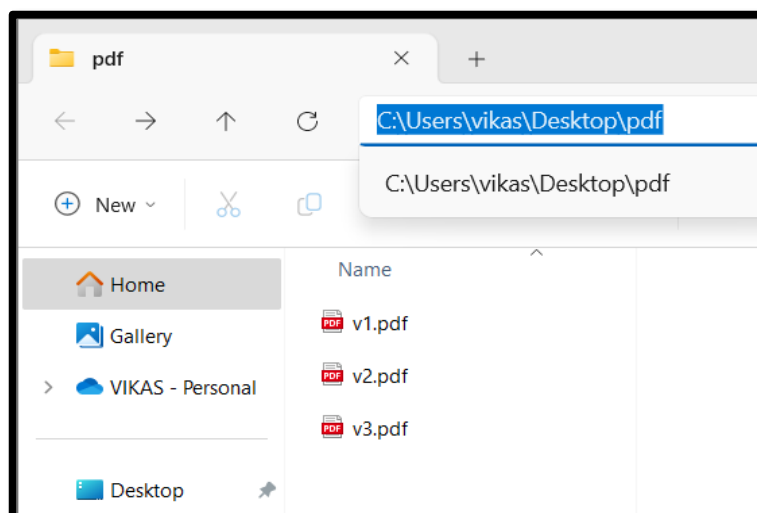


b) Merge PDF's into one Steps:

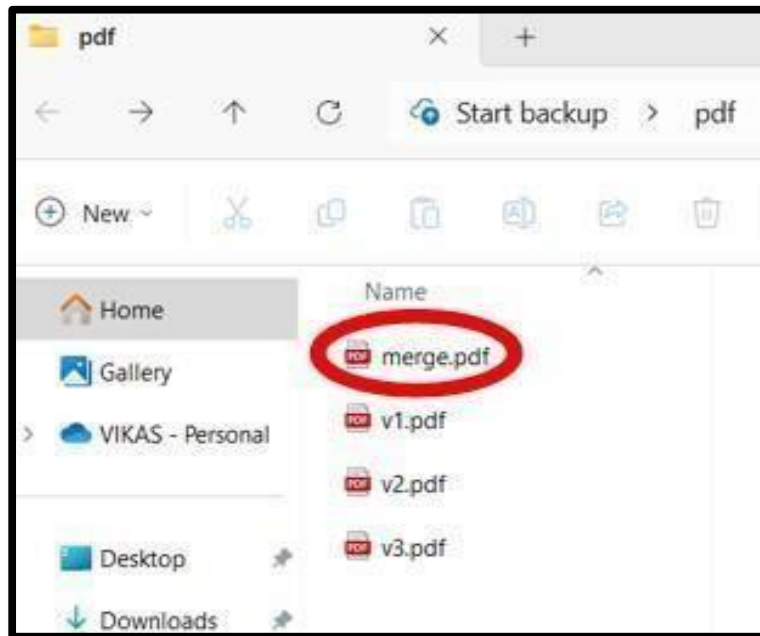
1. Open UiPath Studio & create a new project by clicking on "New Project" in the start page.
2. Drag and drop sequence on workflow. then search Assign activity and drop it to sequence
3. Create a variable pdf of type system.pdf [] now in assign activity under “save to” field type pdfs variable & in “Value to Save” field type Directory.GetFiles(“Type the path of your pdfs”).
4. Drag and drop “Join Pdf Files” Activity below Assign Activity.
5. Under “Output File Name” field type file path along with file name inside double inverted comma.in my case "C:/Users/vikas/Desktop/pdf/merge.pdf"
6. Save your project and run it to merge the PDFs into one.



Before:



After:



c) Get PDF Total Page count Using Regex

- This practical demonstrates step-by-step explanation of how to get the total page count of a PDF using regex in UiPath. Here's a simplified explanation:

Steps:

1. Open UiPath Studio and create a new project.
2. Drag and drop the "Read PDF Text" activity from the activities panel onto the workflow. This activity allows you to extract the text content from a PDF file.
3. In the properties panel of the "Read PDF Text" activity, specify the path of the PDF file you want to extract the text from.
4. Create a new variable of type String to store the extracted text. Let's name it "pdfText".
5. Connect the output of the "Read PDF Text" activity to the "pdfText" variable using an assign activity. This assigns the extracted text to the variable.
6. Drag and drop the "Matches" activity from the activities panel onto the workflow.

This activity allows you to find all matches of a regex pattern in a given text.

7. In the properties panel of the "Matches" activity, specify the following:

a. Input: pdfText

b. Pattern: Enter your regex pattern here. For example, if the page numbers are in the format "Page X of Y", the regex pattern could be "Page \d+ of (\d+)".

8. Create a new variable of type List<Match> to store the matches. Let's name it "pageMatches".

9. Connect the output of the "Matches" activity to the "pageMatches" variable using an assign activity. This assigns the matches to the variable.

10. Create a new variable of type Int32 to store the total page count. Let's name it "totalPageCount".

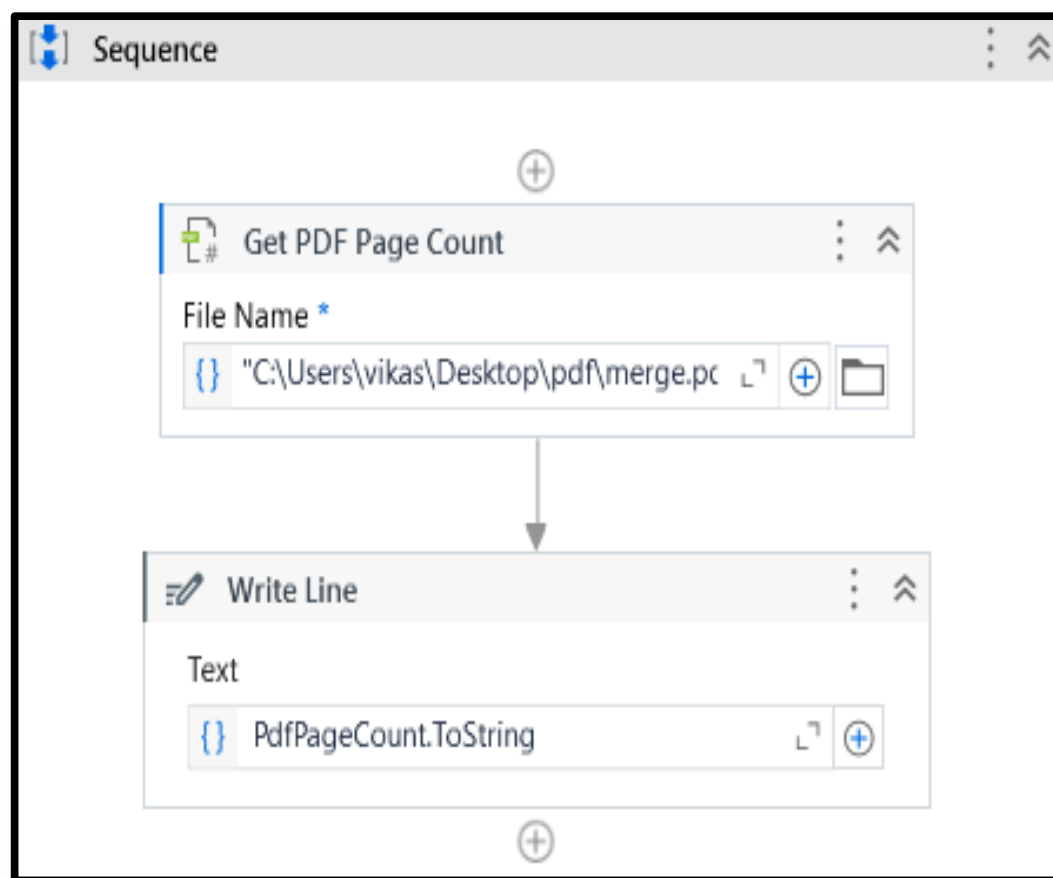
11. Use an assign activity to assign the total page count by extracting the last match from the "pageMatches" variable and converting it to an integer. For

example, you can use "totalPageCount =

Convert.ToInt32(pageMatches(pageMatches.Count-1).Groups(1).Value)".

12. Finally, you can display the total page count using a message box activity or by storing it in a variable for further use in your automation.

13. Remember to adjust the regex pattern based on the specific format of the page numbers in your PDF. Also, make sure to handle any exceptions or errors that may occur during the process.



Output

⌚ ⚠️ 0 ⚠️ 0 ⓘ 2 ⓘ 2 ✅ 0 ✅ 0 ⚙️

Search

- ⌚ Debug started for file: Main
- ⌚ BlankProcess execution started
- ⌚ 3
- ⌚ BlankProcess execution ended in: 00:00:08

d) Demonstrate Exception Handling using UiPath

➤ In this practical we'll learn about how to demonstrate exception handling in UiPath.

Here's a simplified explanation:

Steps:

1. Open UiPath Studio and create a new project.
2. Drag and drop the activities you want to include in your workflow.
3. Enclose the activities that may throw exceptions within a "Try Catch" activity.
4. This allows you to catch and handle any exceptions that occur during the execution of those activities.
5. Inside the "Try" block of the "Try Catch" activity, place the activities that you want to execute.

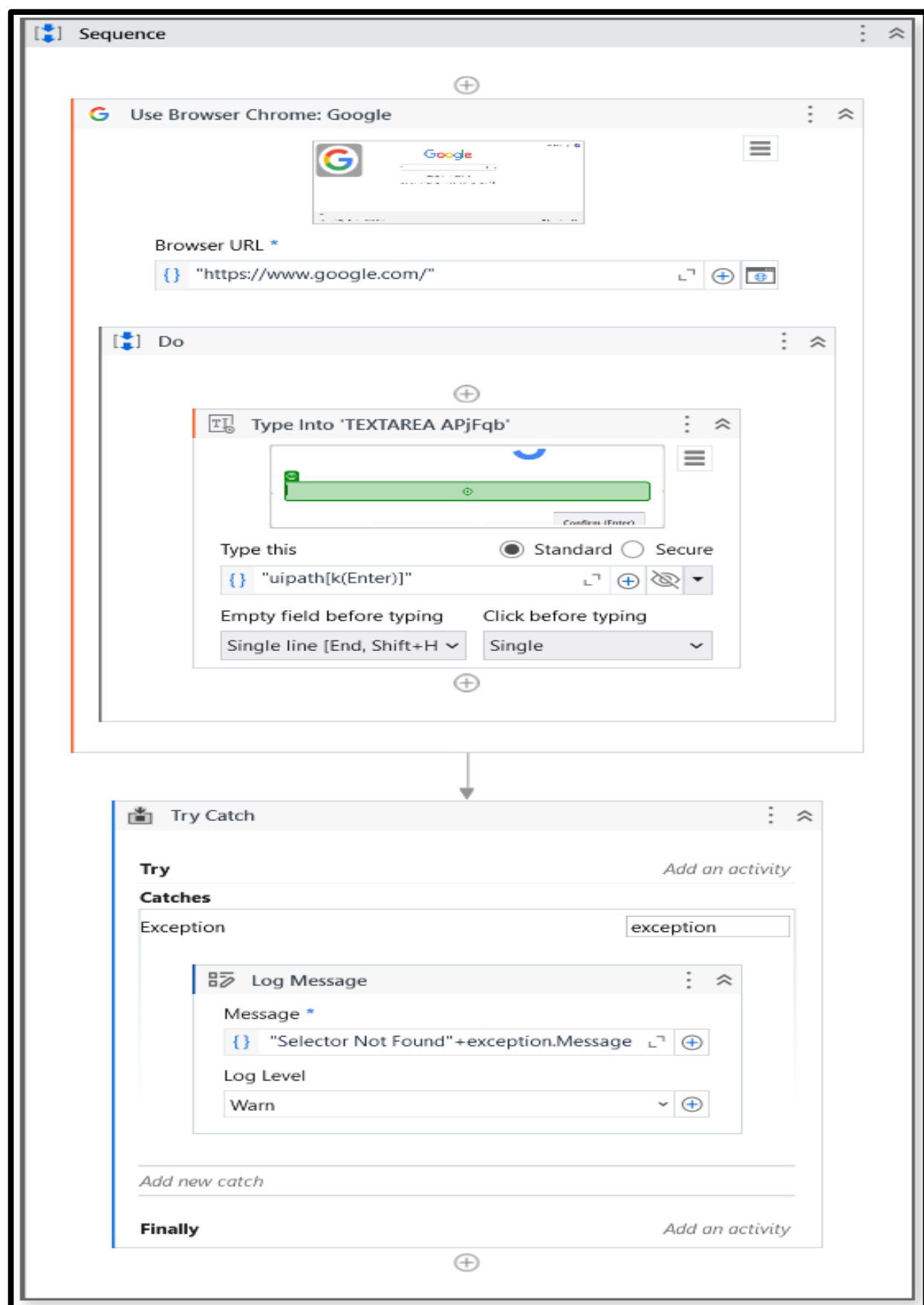
6. Inside the "Catch" block of the "Try Catch" activity, place the activities that you want to execute when an exception occurs.
7. Drag and drop an "Assign" activity inside the "Catch" block to create a variable that will store the exception message.
8. Let's name it "errorMessage" of type String.
9. In the properties panel of the "Assign" activity, assign the value "exception.Message" to the "errorMessage" variable. This retrieves the error message from the exception.
10. Add activities within the "Catch" block to handle the exception. For example, you can use a "Log Message" activity to log the error message, a "Send Email" activity to send an email notification, or a "Message Box" activity to display an error message to the user.

11. If needed, you can also include a "Finally" block within the "Try Catch" activity.
12. The activities inside the "Finally" block will always execute, regardless of whether an exception occurs or not.
13. This is useful for cleanup tasks or activities that should always be performed.
14. Run the workflow to test the exception handling.
15. You can intentionally introduce errors in the activities within the "Try" block to see how the exception is caught and handled in the "Catch" block.

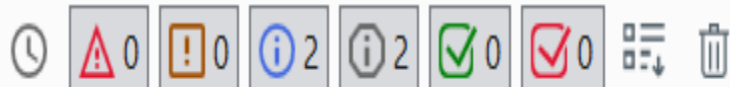
Remember to customize the exception handling based on your specific requirements and the types of exceptions you expect to encounter.

You can use different types of exceptions, such as "System.Exception"

for general exceptions or specific exceptions like "System.IO.IOException" for input/output errors.



Output



Search

- ⌚ Execution started for file: Main
- ⌚ BlankProcess execution started
- ⌚ Audit: Using Web App. Browser: Chrome URL: <https://www.google.com/>
- ⌚ BlankProcess execution ended in: 00:00:10

Practical No: 9 Email Automation

Configure Email using UiPath

a) Read Emails

Theory: Emails are widely used in automation for the purpose of communication between users and robots.

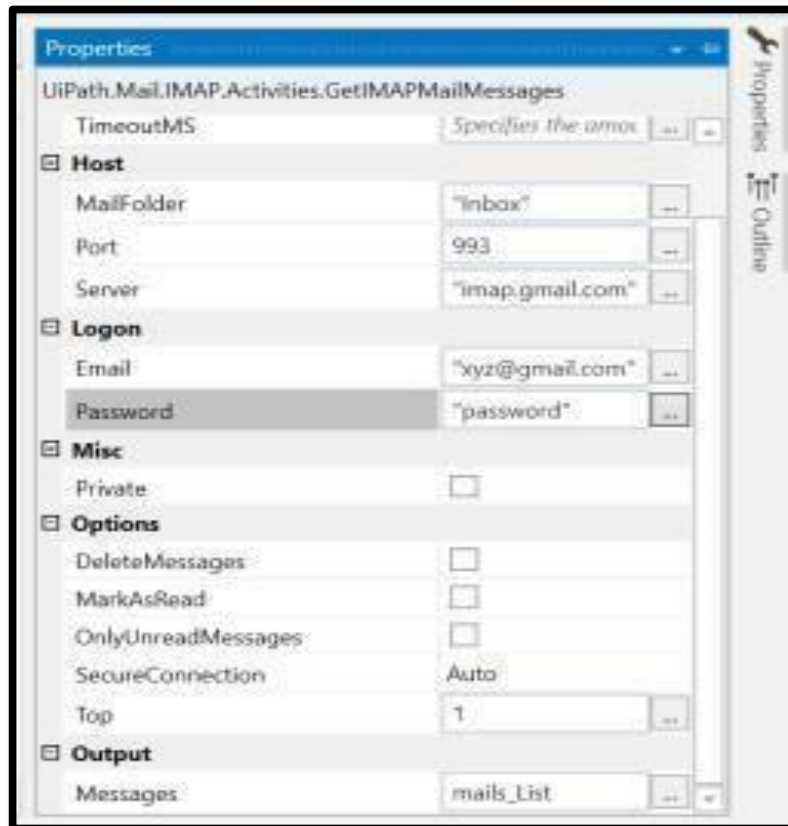
Sometimes emails act as the input to a few processes!!!

Let us see how to retrieve mail's Body, Subject and Sender details using UiPath activities.

Let us implement a workflow which will read the emails from Gmail server and retrieve the emails.

Step 1: Drag “**Get IMAP Mail Messages**” activity into the designer panel and supply the required

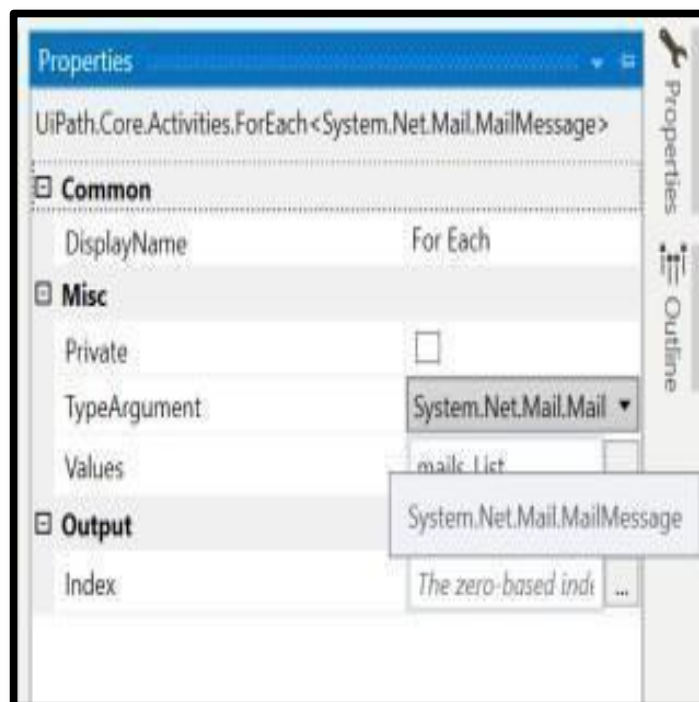
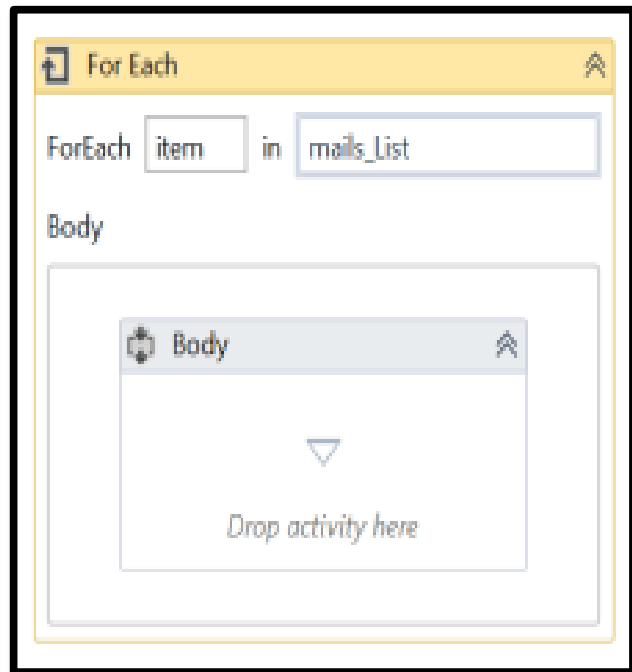
parameters to it as shown below.



Step 2: Drag **“For Each”** activity into the designer panel and pass the

above-created mail List to it.

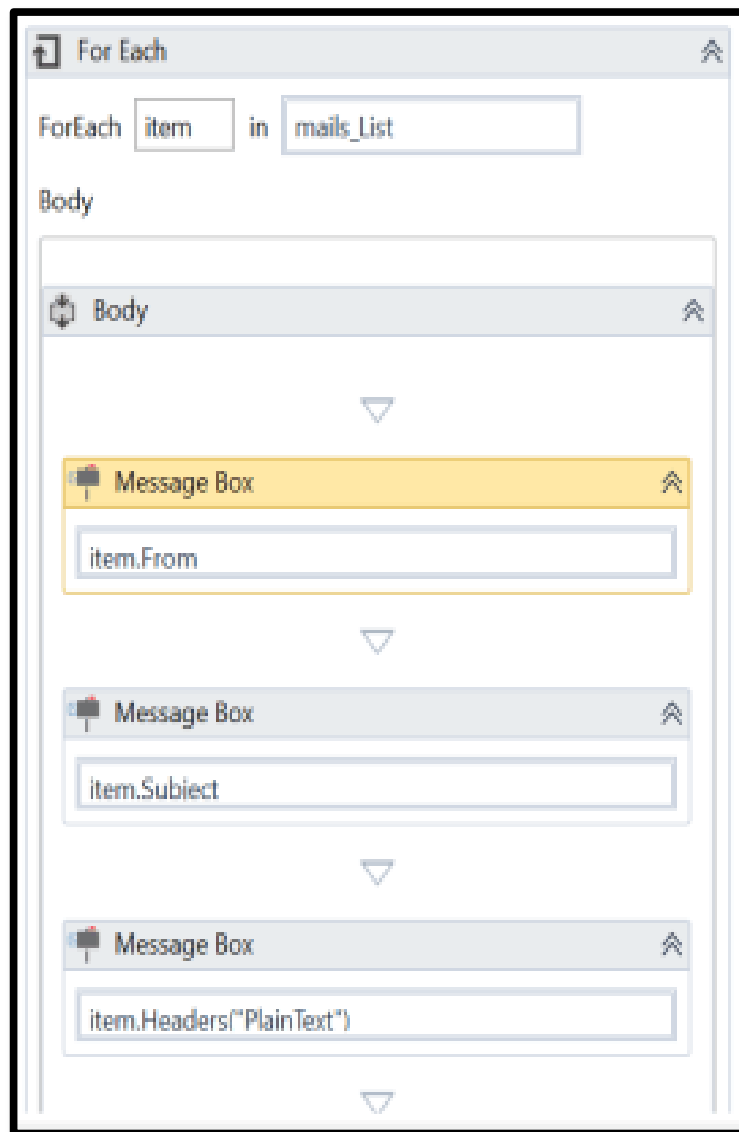
And change the **“TypeArgument”** in the properties panel to `System.Net.Mail.MailMessage`



Step 3: Drag “**Message Box**” activities into the designer panel to display the Sender

details,

Subject and Body of the received mail.



Step 3: Now, execute the project!!!

Mail Sender:



Mail Subject:



Mail Body:



b) Send Email with Attachment

Theory:

➤ In this practical, we are going to learn how we can automate sending emails

using UiPath Studio. This project is a basic application of Robotic Process

Automation (RPA). The user just needs to provide their login credentials and

the user email they want to send the email and that is it, rest of the work will be done by

the bot that we are going to create using the steps mentioned below –

Note: *For this automation to work you need to enable access from Less Secure apps on your Gmail account.*

Step-1: Create a new process in UiPath Studio by clicking on the ***Process Tab***.

Step-2: Set the name of the process, give a brief description, and click on

Create. The UiPath studio will automatically load and add all the dependencies of the project.

The start screen panel of ui-path along with each and every dependencies will be opened.

Step-3: Now in the activities panel search for ***Sequence*** activity. Drag and drop it in the designer window.

Step-4: Now in the sequence add an ***Input dialog box*** using the search activity panel.

Step-5: Now click on the ***Variables*** tab to create a variable of string type that will store the personal email id entered by the user.

In the properties panel of the Input Dialog activity add the variable in the

Output area.

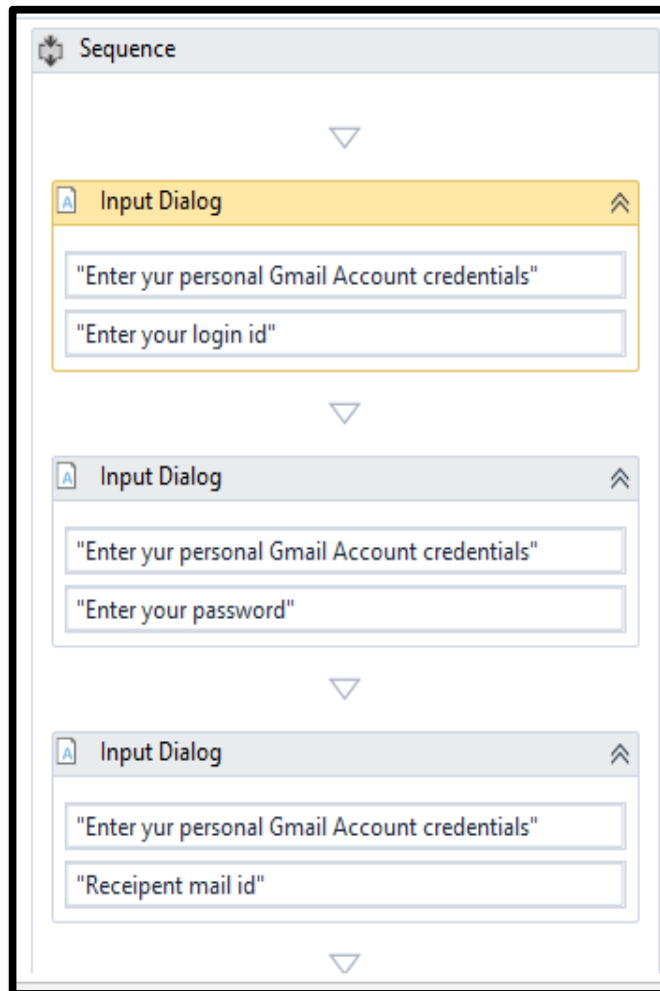
Step: Create two more string variables, one for storing the password of the user and the second for storing the recipient email id.

Step-6: Now add another ***Input dialog box*** and pass the variable created earlier to store the password in the ***Output Area*** of the properties panel.

Click the checkbox ***IsPassword*** for this activity.

Step-7: Finally add another ***Input dialog box*** for taking the recipient email id from the user.

Step: Pass the variable created earlier in the **Output Area**.



Step-8: Now in the **Activity Panel** search for **Send SMTP Mail Message**

activity. Drag and drop it to the sequence.

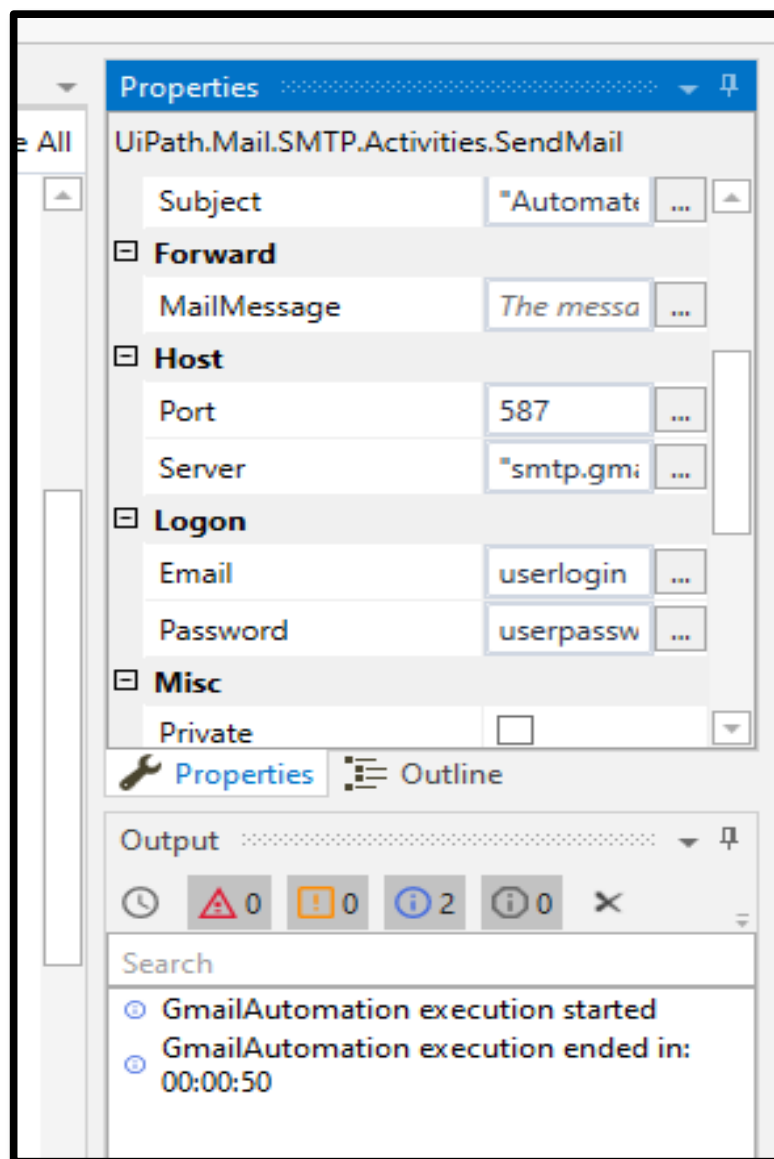
Step-9: In the **To** field pass the variable that stores the recipient email id. Specify the **subject** and **body** of the mail.

The screenshot displays a workflow editor with three steps arranged vertically, separated by downward-pointing triangles. The first step is an 'Input Dialog' with two input fields: 'Enter yur personal Gmail Account credentials' and 'Enter your password'. The second step is another 'Input Dialog' with two input fields: 'Enter yur personal Gmail Account credentials' and 'Receipent mail id'. The third step is 'Send SMTP Mail Message', which is highlighted with a yellow border. It contains three input fields: 'To' with the value 'receiverlogin', 'Subject' with the value 'Automated Mail', and 'Body' with the value 'This mail is sent by an automated bot.'. Below these fields is a blue link labeled 'Attach Files'.

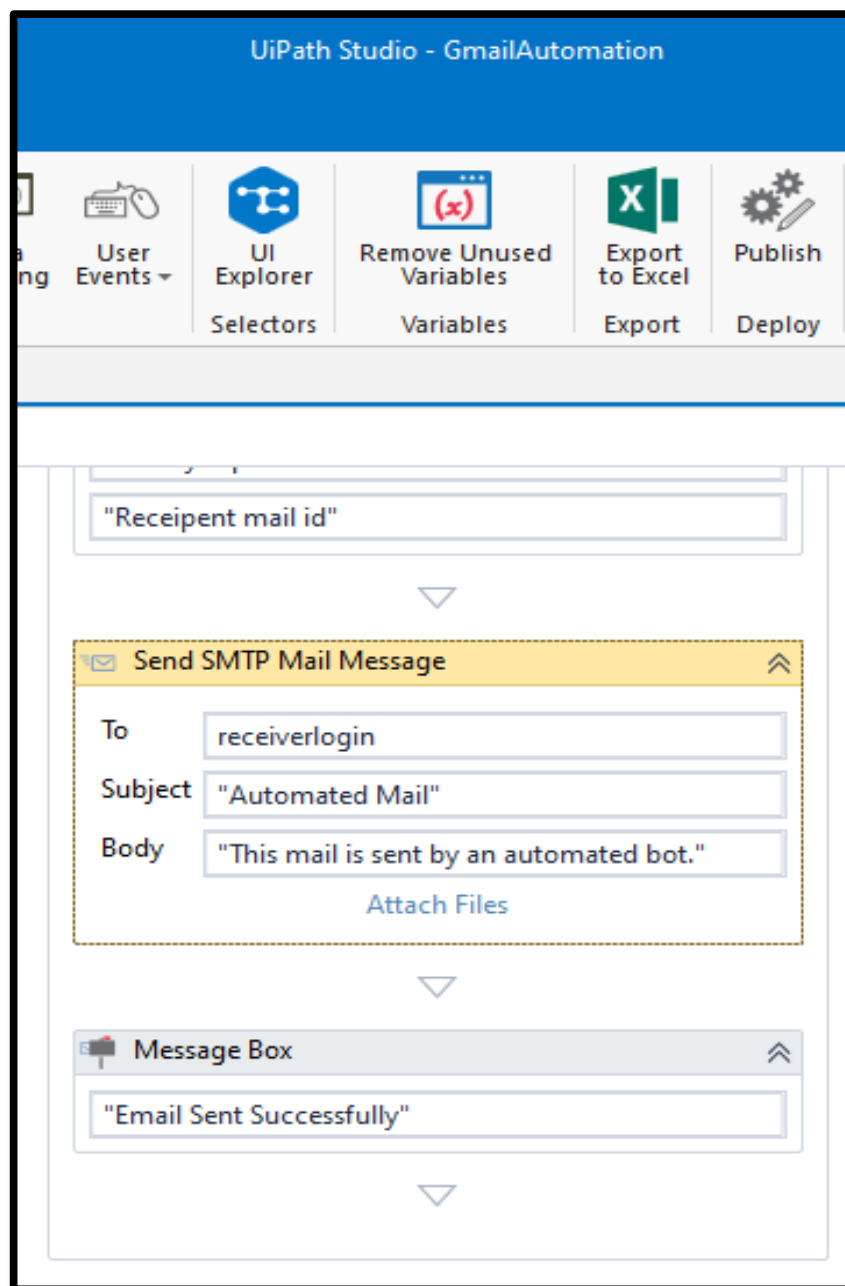
Step-10: In the ***Properties panel***, set *Port*

no. to **587** and *Server* to **smtp.gmail.com**. Pass the variables that store user credentials

in the email and password field respectively.



Step-11: Finally add a **Message Box** activity. Specify the message you want to display as a notification that the email was sent successfully.



Step-12: Save the process using the **Save** button in the design panel and then click on

Run. Your bot is ready for sending automated emails.

c) Save Email Attachments

➤ **Theory:**

The aim is to save the attachments of the top 30 emails which have a keyword in the subject line. The attachments will be stored in a particular folder.

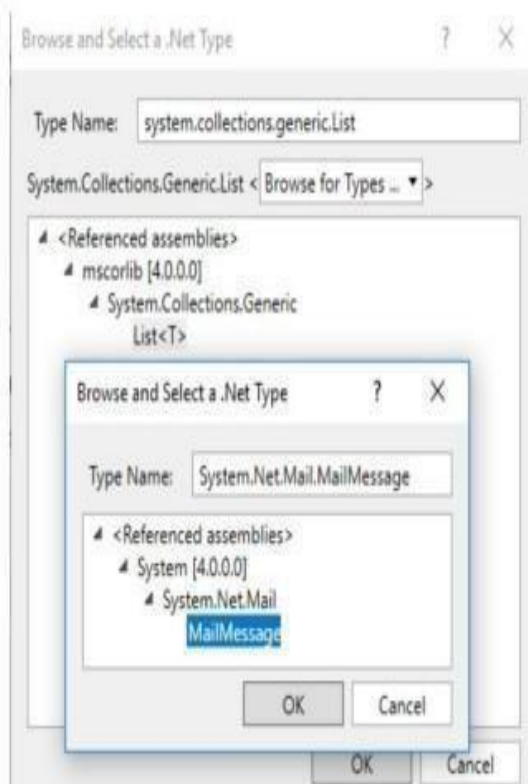
Steps To Automate Follow the below steps to achieve the aim:

- ❖ Assign the Email address to a variable and mention the password in the Get Password activity.
- ❖ Use the Get IMAP mail messages activity and mention the folder, port number and server.
- ❖ For each mail mention the keyword that you want to consider and then use an If-else loop.
- ❖ In the If section, use the Save Attachments activity and mention the destination directory.

Step 1: Create variables such as Email, Password, and GetMailMessages of String,

String and List respectively.

First type 'System.Collections.Generic.List' then under 'Browse for type' section, select 'System.Net.Mail.MailMessage'. Then click OK. You'll get the datatype of List



Name	Variable type	Scope	Default
Email	String	Email Automation	Enter a VB expression
Password	String	Email Automation	Enter a VB expression
GetMailMessages	List<MailMessage>	Email Automation	Enter a VB expression

Step 2: Drag the Assign activity and assign the To section of the activity to Email

variable and value to the email address from which you want the details to be read.

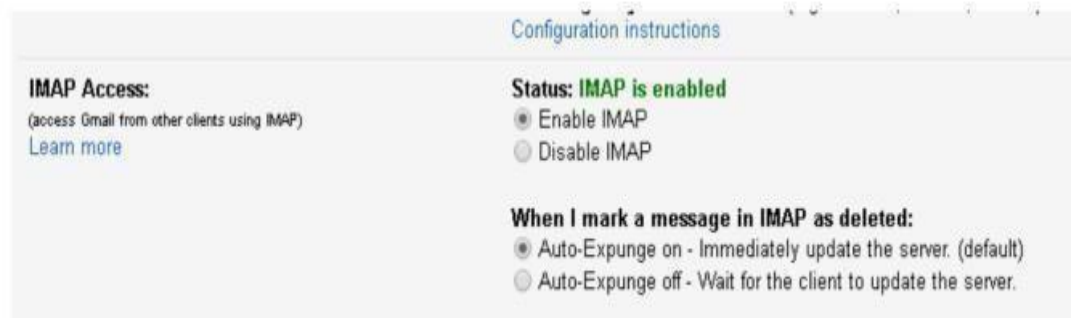
Step 3: Drag the Get Password activity, and mention Password for your email ID in the

Password section of the Properties pane. Also, mention the Password variable in the

Result pane.

Kindly check with these pls

- Username and password of the user mentioned in the property panel of IMAP mail activity
- In ur gmail Accounts settings Turn on “Allow Secure apps” and try the workflow.
- also make sure that IMPA is enables



Step 4: Drag the Get IMAP Mail Message activity and mention the following details in

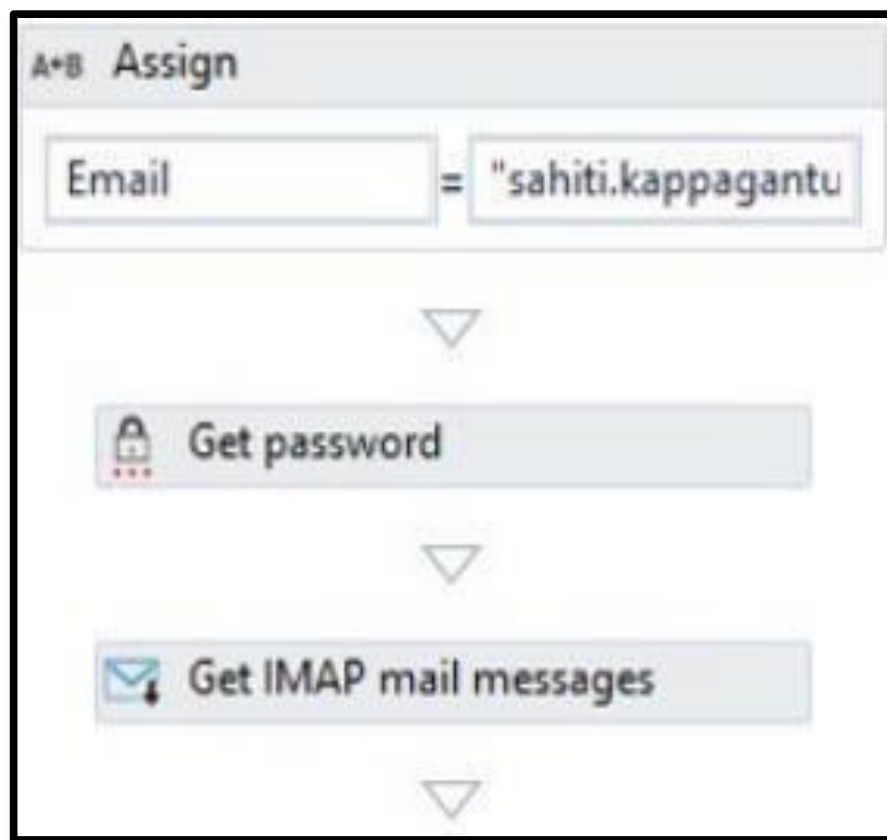
the Properties pane.

In the section of:

- MailFolder – “Inbox”
- Port – 993
- Server – “imap.gmail.com”
- Email – Email variable
- Password – Password variable
- Top – 30
- Messages – GetMailMessages Refer to the
snapshot below.

DisplayName	Get IMAP mail messages
TimeoutMS	<i>Specifies the amount</i> ...
Host	
MailFolder	"Inbox" ...
Port	993 ...
Server	"imap.gmail.com" ...
Logon	
Email	Email ...
Password	Password ...
Misc	
Private	☐
Options	
DeleteMessages	☐
MarkAsRead	☐
OnlyUnreadMessages	☐
SecureConnection	Auto
Top	30 ...
Output	
Messages	GetMailMessages ...

Your execution should look like below till the above step.



Step 5: Drag the For Each activity, and then mention mail in the item section and the GetMailMessages in the variable expression section.

Step 6: Now, in the Body section of this activity, drag the If activity. In this activity, you have to specify the condition to save the attachments if the subject line contains a keyword. To do so, follow the below steps.

Step 6.1: In the Condition section of the If activity, mention

`mail.Subject.Contains("example")`, where 'example' is the keyword that has to be considered.

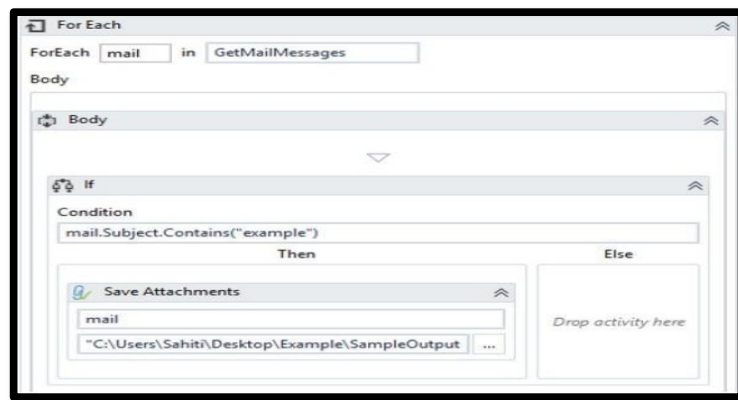
Step 6.2: Go to the properties section of For Each activity and in the Type Argument

mention `System.Net.Mail.Message`. Refer to the snapshot below.



Step 6.3: In the Then section of the If-activity, drag the Save Attachments activity. In

this activity, mention the mail variable in the mail message area and the path of the folder where all the attachments have to be saved. Refer below.



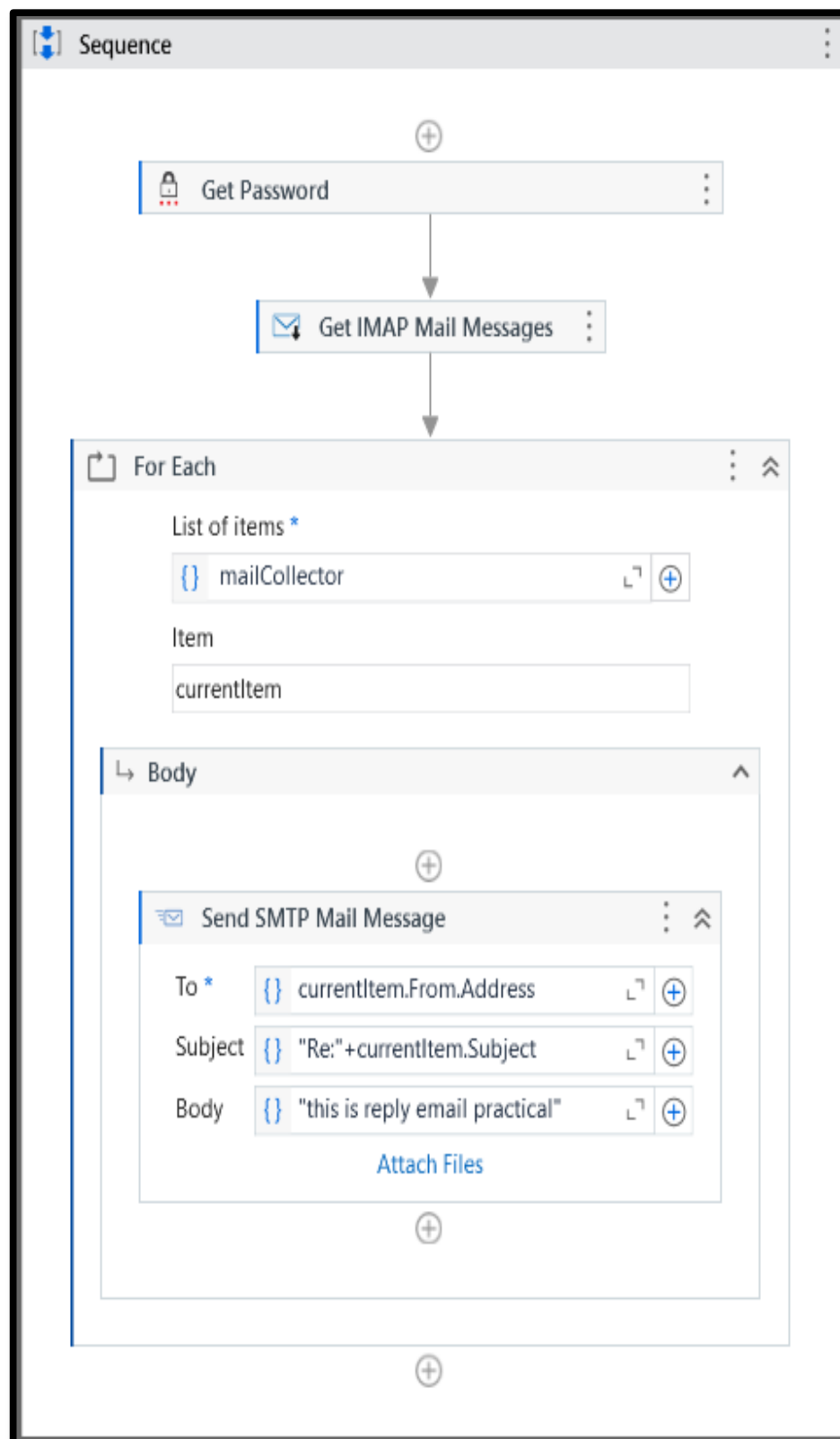
Step 7: Execute this sequence by clicking on the Run button. You would see that all the emails having the keyword example, in the subject line, would be read and the attachments would be stored in the mentioned folder.

d) Reply to Email Steps:

1. First, open UiPath Studio on your computer. create a new project by clicking on "New Project" in the start page.
2. Give your project a name and select the location where you want to save it. Click on "Create" to proceed.
3. In the project, you'll see a sequence of activities. Drag and drop the "Open Browser" activity from the Activities panel to the sequence.
4. Configure the "Open Browser" activity by providing the URL of your email provider's website and selecting the browser you want to use.
5. Next, drag and drop the "Type Into" activity to the sequence. This activity will allow you to enter the email address, subject, and message body.

6. Configure the "Type Into" activity by indicating the email address field, subject field, and message body field on the email compose page.
7. Use variables or arguments to dynamically pass the email address, subject, and message body to the "Type Into" activity.
8. After entering the email details, drag and drop the "Click" activity to the sequence.

This activity will simulate clicking the send button.
9. Configure the "Click" activity by indicating the send button on the email compose page. Finally, save and run your UiPath project to send the reply email.



Practical No: 10

Orchestrator management and mini project

a) Deploy bots to Orchestrator

&

b) Run jobs from Orchestrator

Theory:

 To deploy and run Robots to Orchestrator, we need to configure the machine with

Orchestrator. To do this, we first have to create a provisioning Robot from the

Orchestrator URL: <https://platform.uipath.com>

Creating a provision Robot from the Orchestrator:

The user needs; permissions to register the new Robot and they must have the

following information:

- The name of the machine and the key required to connect to Orchestrator.

They can be found through the **Control Panel | System**, for the key, log on to the

Orchestrator URL, click on the **ROBOTS** page, then click on the + button. A pop-up window appears where your key is visible.

Security | System and User | Settings | Deployment, which provides us the Robot API Key.

- The username and password of the specified machine to access it.
- To create the provision Robot, go to the Orchestrator. Click on the Robot option on the left side of the Orchestrator page.
- After displaying the Robot page, click on the + button. It will display a small window to create a provision Robot:



- After the window is displayed, fill in all the mandatory information that is required to connect the Robotdmachine name, Robot name, username/domain name, password, type, and description, as given in the following screenshot:

CONFIGURATION

RUNTIME

Key *

1a354725-847d-4e8b-92c3-898574f0f9ee

Machine *

Name *

Domain\Username *

Password

Type

NonProduction

Description

☐ Create another

CANCEL

PROVISION

- In the **Machine *** field section, enter the **machine** name that is required to connect to Orchestrator.
- From the **Name *** field enter the desired name of the Robot.

- Now in the **Domain\Username *** field, type the name that will be used to log in to the defined machine. Here, we need to define the format **Domain/Username *** if the user is in the domain. It is compulsory to choose a short domain name.
- **Password** field is optional; it can be skipped.
- We can choose the type of Robot from the drop-down list.
- **The description** field is also optional. You can give a short summary describing your Robot.
- Now copy the Robot **Key *** and paste the key in UiPath Robot while configuring your Robot.
- Click the **PROVISION** button and the Robot will be displayed on the Robot page.

Connecting a Robot to Orchestrator

When we deploy a Robot to Orchestrator, we must have the machine name and the key for each Robot. To keep the value of these fields, we can create another Robot from the **Provisional Robot** windows.

If we want to create a new Robot, we have to take authentication from administrator.

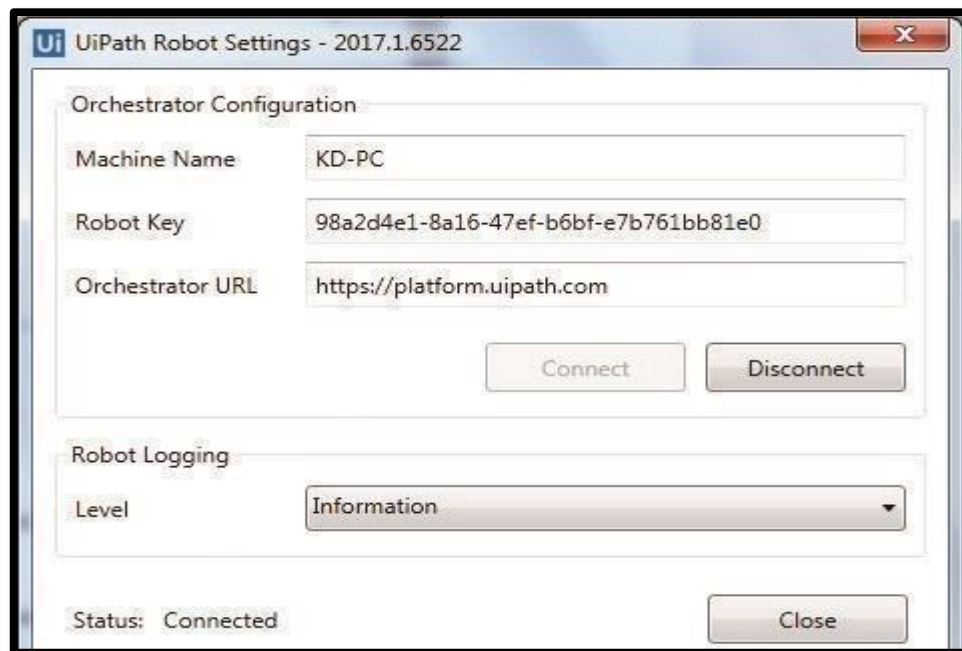
By default, the administrator has the right to register a new Robot.

To connect the Robot machine to Orchestrator, follow the given steps:

1. From the system taskbar, click on UiPath Robot. The **Robots** windows will be displayed:



2. Now go to **Options** and choose **Settings...**; the Robot Settings will be displayed:

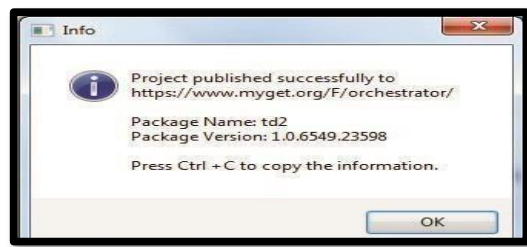


3. In the **Robot Key** field, paste the received key of provision Robot to Orchestrator.
4. In the **Orchestrator URL** field, enter the address of the Orchestrator.
5. Now click the **Connect** button. After clicking the **Connect** button, the Robot will connect to the Orchestrator.

❖ **Deploy the Robot to Orchestrator**

To deploy our Robot, first of all, it must be connected to Orchestrator. Ensure that our bot is connected to Orchestrator then follow the given steps to deploy it:

1. First of all, install UiPath on the machine.
2. Provision the Robot machine and take the Robot key from Orchestrator.
3. After receiving the key go to the Robot configuration panel and enter the key here.
4. Also, you need to enter the Robot key into the configuration URL, which can be found from the admin section of Orchestrator.
5. Publish the project with publish utility from UiPath. When it is published successfully, it will display the information shown in the following screenshot:



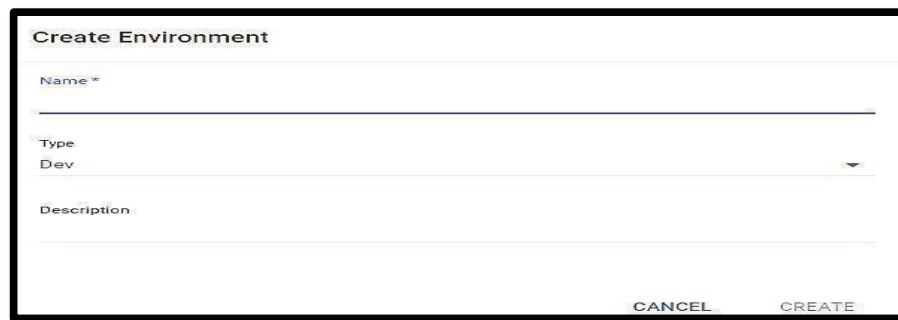
6. The project has been published in the Orchestrator.

7. To create the environment, go to the home page, click on the **ROBOTS**

option, and click on the **Environments** Tab. Then click on the + button:



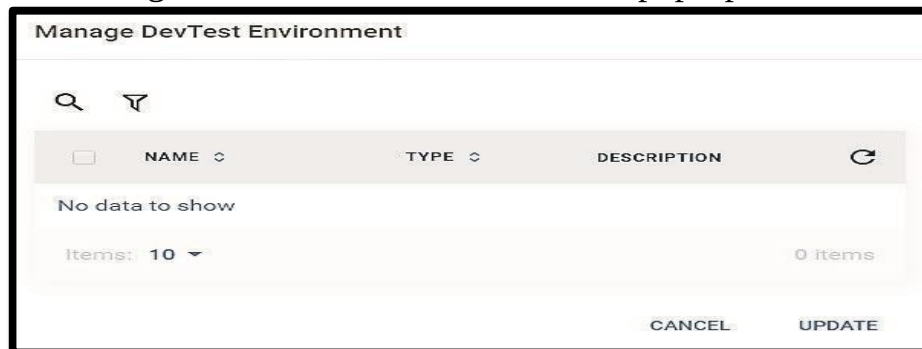
8. Once the details are filled in, click on **Create**:



A screenshot of a 'Create Environment' form. It has a title bar 'Create Environment'. Below it are three input fields: 'Name *' (required), 'Type' (with a dropdown arrow), and 'Description'. At the bottom right are two buttons: 'CANCEL' and 'CREATE'.

9. After creating the environment, a small window will appear as shown in the following screenshot, where we can manage the Robot within the environment:

10. After clicking on the + button, a will window pop-up where we can choose



A screenshot of a 'Manage DevTest Environment' window. It has a title bar 'Manage DevTest Environment'. Below the title bar are search and filter icons. There is a table with columns: NAME, TYPE, and DESCRIPTION. The table is currently empty, showing 'No data to show'. Below the table is a pagination bar showing 'Items: 10' and '0 items'. At the bottom right are two buttons: 'CANCEL' and 'UPDATE'.

the Published package, as shown in the following screenshot, and then click on the **CREATE** button:



11. After clicking the **Deploy Process** button, a window popup will appear where we can choose the published package, as shown in the following screenshot, and then click on the **CREATE** button.

OR

12. Packages can be manually uploaded from the local directory after clicking the **View Packages** option and then clicking the **Upload** button as given in the following screenshot: **PROCESS | View Packages | Upload Packages:**

Deploy Process

Package Name
No process selected.

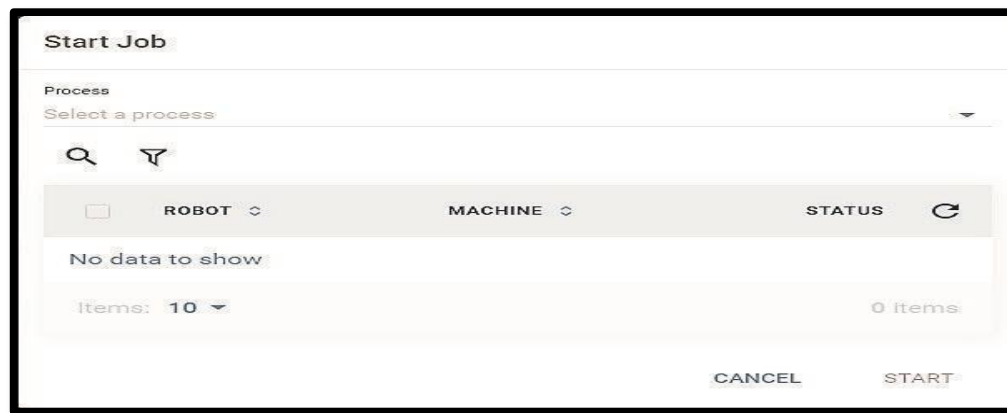
Package Version
No version selected.

Environment
DevTest

Description

CANCEL CREATE

13. Now the package has been deployed to the Orchestrator and is ready to be executed through the web.
14. Next, click the **JOBS** option for execution and click on the **Start** icon as shown:



15. After clicking the **Start Job** button, the Robot will execute over
Orchestrator.

c) Queue Introduction:

i. Add items to Queue.

Theory: To demonstrate how to add items into a queue, let us create an automation that processes entries contained in a .xlsx file. Let's say you represent a banking institution and you have a deposits' report of 50 clients. You want to process the data of each client that previously deposited an On- Us Check smaller than \$5000, using a third-party financial application. The report contains the name of the client, the On-Us Check value, and two other additional values, let's say Cash In and Not On-Us Check. We create a workflow in which the QueueItem variable stores, by turn, the information of each client. Each entry read by the QueueItem variable is then stored in a queue item and added to the designated queue in Orchestrator. The result of

this whole process is a queue which contains each clients' specific information. Before proceeding to the actual workflow, you might want to take a look at the clients' report [here](#).

We are going to work with the "July Reports" sheet. Without further do:

1. Create a new sequence.
2. Add an Use Excel File activity inside the sequence, and define the location of the workbook in the Excel File pane of the activity.
3. Inside the Do section, add a Read Range activity to read the desired range of values from the report and store them in a DataTable variable. In our case, type "A1-D51" or simply leave the default value, "", which means the entire spreadsheet is read.

Read Range Value : `Excel.Sheet("July Report").Range("A1:D51")` In Save To Press

Ctrl+k and Name the variable as DataTable1

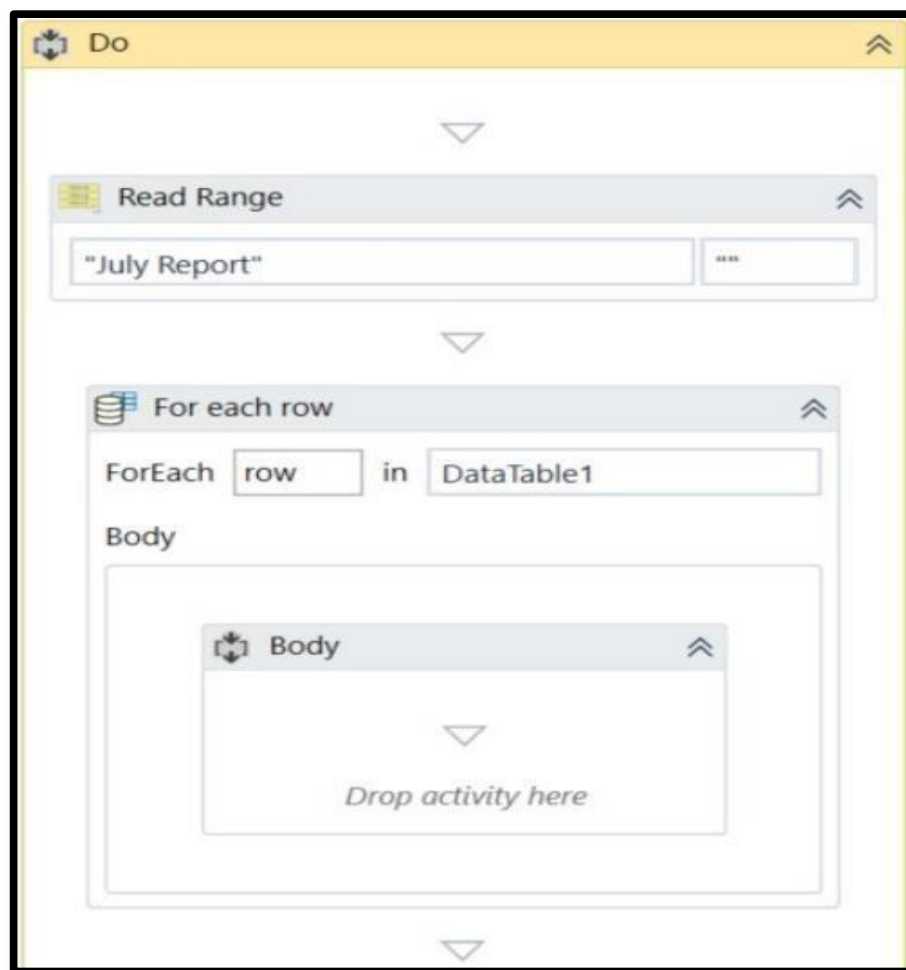
4. In the Data Table field, press Ctrl + K to create a new DataTable variable which stores the previously read data range. Name it DataTable1 or similar. The Variables pane should also display the newly created variable as follows:



The screenshot shows a table with four columns: Name, Variable type, Scope, and Default. The first row contains the text 'DataTable1', 'DataTable', 'Do', and 'Enter a VB expression'.

Name	Variable type	Scope	Default
DataTable1	DataTable	Do	Enter a VB expression

5. Next, add a For Each Row activity to the Designer panel and fill in the blank fields with “row” and the name of the DataTable variable, DataTable1, respectively. This activity performs the action defined in the Body section for each row in the data table. The Do section should look similar to the following screenshot so far:



6. In the Body section of the For Each Row activity, add an Add Queue Item activity.

This adds an item to a queue (defined below at steps 8 and 12), for each row in the spreadsheet.

7. In the Properties pane, in the QueueName field, type the name of the queue you want your items to be stored in, let's say ReportQueue.

8. In the Input section, click the button corresponding to the ItemInformation field to open the ItemInformation window. Create 4 arguments, each corresponding to a column from the report. Define them as follows:

❖ Name, keeping the default values for Direction and Type, and with Value set to `row(0).ToString`. This retrieves the value from the column with the counter 0 for each row in the workbook, in our case the name of the client.

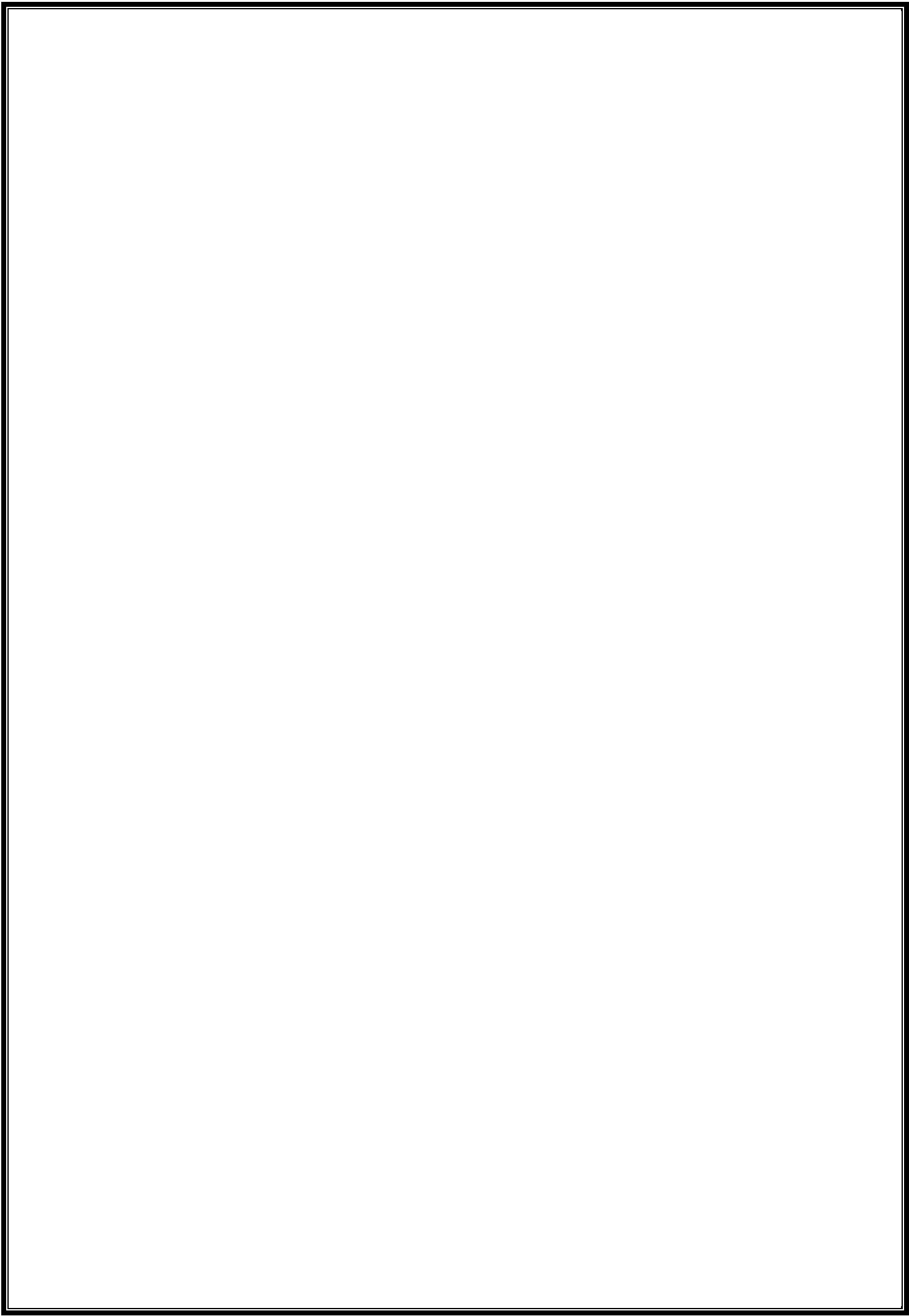
❖ CashIn, keeping the default values for Direction and Type, and with Value set to `row(1).ToString`. This retrieves the value from the column with the counter 1 for each row in the workbook, in our case the Cash In value.

- ❖ OnUsCheck, keeping the default values for Direction and Type, and with Value set to row(2).ToString. This retrieves the value from the column with the counter 2 for each row in the workbook, in our case the On-Us Check value.
- ❖ NotOnUsCheck, keeping the default values for Direction and Type, and with Value set to row(3).ToString. This retrieves the value from the column with the counter 3 for each row in the workbook, in our case the Not On-Us Check value.

9. The ItemInformation window should look similar to the following screenshot:



10. Connect Studio's Robot to Orchestrator. For details on how to do it, click here.
 11. Navigate to <https://platform.uipath.com>, log in using your credentials, and then create a queue, ReportQueue, to store the items. Make sure that, whatever name you used above (step 8), you use the same here.
 12. Go back to Studio. At this point your workflow should be error-free and functional, so the next step is to make it available to Orchestrator. In this regard click Design > Publish.
 13. To run the process, click Design > Run
- Note: Take into account that, after being processed, queue items become transactions, and are further handled as such.*
14. Navigate to <https://platform.uipath.com> again and display the transactions.



ii. Get Queue item from Orchestrator

Theory: To demonstrate how to further process transactions from a queue, we are going to create a second workflow which retrieves from the appropriate queue, the transactions with an On-Us Check value smaller than \$5000. For each transaction that meets the condition, the values are collected and introduced into a third-party dummy application. The other ones are labeled as business exceptions.

1. Go to Design > New > Flowchart to create a new file in the project initiated above.
2. Add an Open Application activity to open the third-party dummy application.

Connect it to the Start node.
3. Fill in the required details (Display Name, FileName, Selector) on the Properties panel, and configure the activity to log in with the

corresponding credentials. Use admin for the Username field, and password for the Password field.

4. In the FileName field, fill in the path of the application.

5. In the Selector field, using Selector Editor, define a selector for

UIDemo.

6. Add a Should Stop activity to the Designer panel and connect it to the Open

Application activity.

7. Add a Flow Decision activity to the Designer panel and connect it to the Should Stop

activity. These last two activities check if you stopped the job while it was still

running. This gives you the possibility to further configure the workflow to perform

various safety routines after the stop is triggered. See below an example of such a

routine:

- ❖ Add a Log Message activity and connect it to the True branch of the previously added flow decision.

- ❖ In the Properties panel, in the Message field, type “Stop process required” or something similar to be displayed if the stop is triggered.

- ❖ Add a Close Application activity and connect it to the Log Message activity previously added.

- ❖ In the Properties panel, in the DisplayName field, after Close application, type UIDemo.exe UIDemo.

8. Add a Get Transaction Item activity to the False branch of the flow decision added previously (step 7). This activity retrieves the transaction items from the queue and stores them, by turn, with each iteration of the loop, in a QueueItem variable.

9. In the Properties pane, in the QueueName field, type the name of the queue which stores your items. In our case, as defined in the Uploading Items into a Queue section, the name is "ReportQueue".
10. In the TransactionItem field, press Ctrl+K to create the QueueItem variable which stores each transaction item. Name it "TransItem" or similar.
11. Add a Flow Decision activity to the Designer panel and connect it to the Get Transaction Item activity.
12. In the Properties pane, in the Condition field, type TransItem isNot Nothing. This activity checks the value of each item to determine if the end of the queue has been reached.

13. Connect the False branch of the flow decision to the Log Message

activity created in step 7. This ensures the job stops when the end of the queue is reached.

14. Add another Flow Decision activity to the Designer panel and connect it to the True branch of the flow decision created in step 11.

15. In the Properties pane, in the Condition field, type

`cint(TransItem.SpecificContent("OnUSCheck").ToString) < 5000`, where:

❖ `cint` converts a variable of type string to one of type integer.

❖ `TransItem` is the `QueueItem` variable (step 18) used to store the items.

❖ `SpecificContent("OnUSCheck")` retrieves the content of the On US Check argument in each item.

❖ .ToString converts the content of the QueueItem variable into a variable of type string.

❖ < 5000 is the condition to be checked against.

The above expression converts the OnUSCheck value in each QueueItem variable into a string variable, which is then further converted into an integer. The value thus obtained is checked for compliance with the condition.

16. Add a Set Transaction Status activity to the Designer panel, and connect it to the False branch of the previous flow decision (step 13).

17. In the Properties pane, make sure the Status field is set to Failed, and ErrorType to Business. This way, if the value of On US Check is bigger or equal to 5000, the transaction is marked as failed with a business exception.

18. In the TransactionItem field add the TransItem variable.
19. In the Reason field, type "OnUSCheck amount is bigger than 5000" or similar, to display for a failed transaction.
20. Connect the Set Transaction Status activity to the Should Stop activity (step 6), to create a loop and process the next transaction in the queue.
21. Add a Sequence activity to the Designer panel, and connect it to the True branch of the previous flow decision (step 13). Inside the sequence, add a Try Catch activity and configure it such that any potential errors are correctly labeled as application errors when processing the items.
22. Open the UIDemo application, and log in with the username admin and the password password.

23. Perform the following series of activities for each of the Cash In, On Us

Check and Not On Us Check fields in the UIDemo application:

- Into the previous Try Catch activity, add a Type Into activity.
- Click Indicate on screen and select from the UIDemo interface,

the blank field corresponding to Cash In.

- In the Properties pane, in the Text field, type

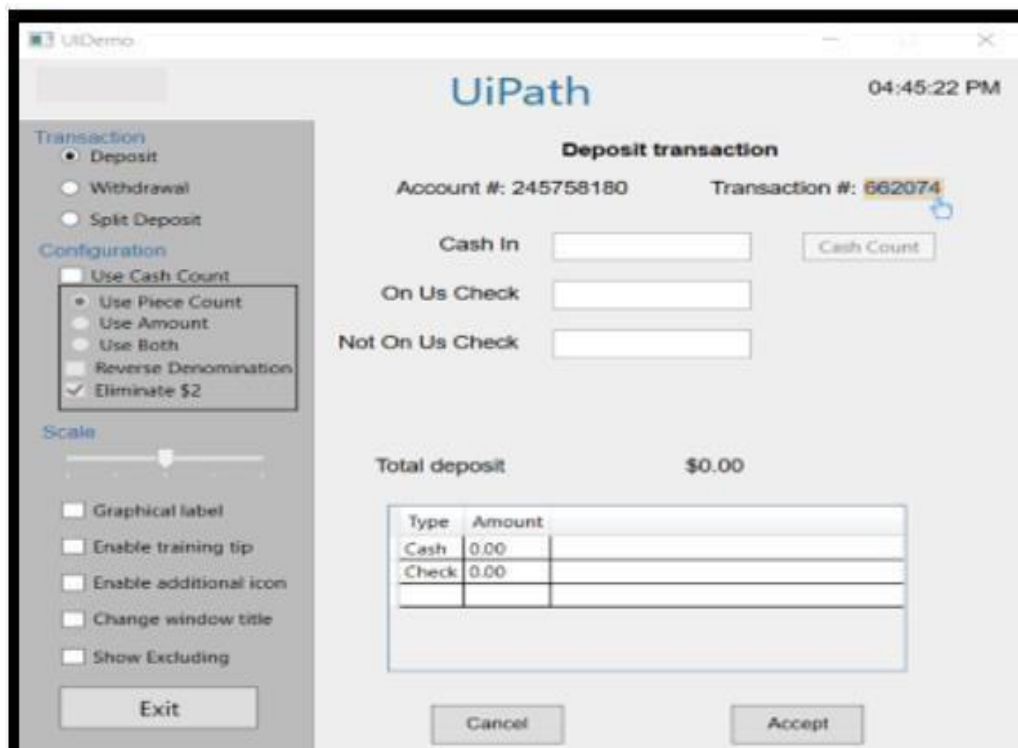
`transItem.SpecificContent("CashIn").ToString`. Each termin the expression is explained above (step 15).

This expression converts the value in each QueueItem variable into a string variable.

- Make sure that the Simulate Type option is selected.

24. On the Design tab, in the Wizards section, click Screen Scraping and point

at the transaction number in the UIDemo application.



25. From the Screen Scraper Wizard, choose a scraping method and click Finish. This inserts a Screen Scraping activity in your workflow. Make sure to position it after the previous Type Into activities, and set the Text field to a new variable, say TransactionValue. This retrieves the transaction number for each queue item that meets the initial condition.

26. Add a Set Transaction Status activity after the Screen Scraping activity.
27. In the Properties pane, make sure the Status field is set to Successful, and the ErrorType is set to Application.
28. This way, if the set of values is successfully introduced in the application, the status of the transaction is set to Successful, or else labeled as an application exception.
29. In the Output window, define a new argument and add the previously created variable, TransactionValue (step 25) as value.
30. In the TransactionItem field, add the TransItem variable.
31. Add a Click activity to the Sequence activity after the Type Into activities.

32. Click Indicate on screen and select the Accept button from the UIDemo

interface. Make sure the Simulate Type option is selected.

33. The Try Catch activity should look similar to the following

screenshot:

Try catch

Try

Sequence

Type into 'editable text: cashintb'

transItem.SpecificContent("CashIn").ToString

Type into 'editable text: onustb'

transItem.SpecificContent("OnUsCheck").ToString

Type into 'editable text: notonustb'

transItem.SpecificContent("NotOnUsCheck").ToString

Screen Scraping

Attach Window 'UiDemo Hwndwrapp'

UiPath

11/26/2018 03:56:10 PM

Deposit transaction

Do

Get Full Text 'text: 658238'

Transaction #: 658238

Set transaction status

Click 'push button: Accept'

Catches

Exception

Add new catch

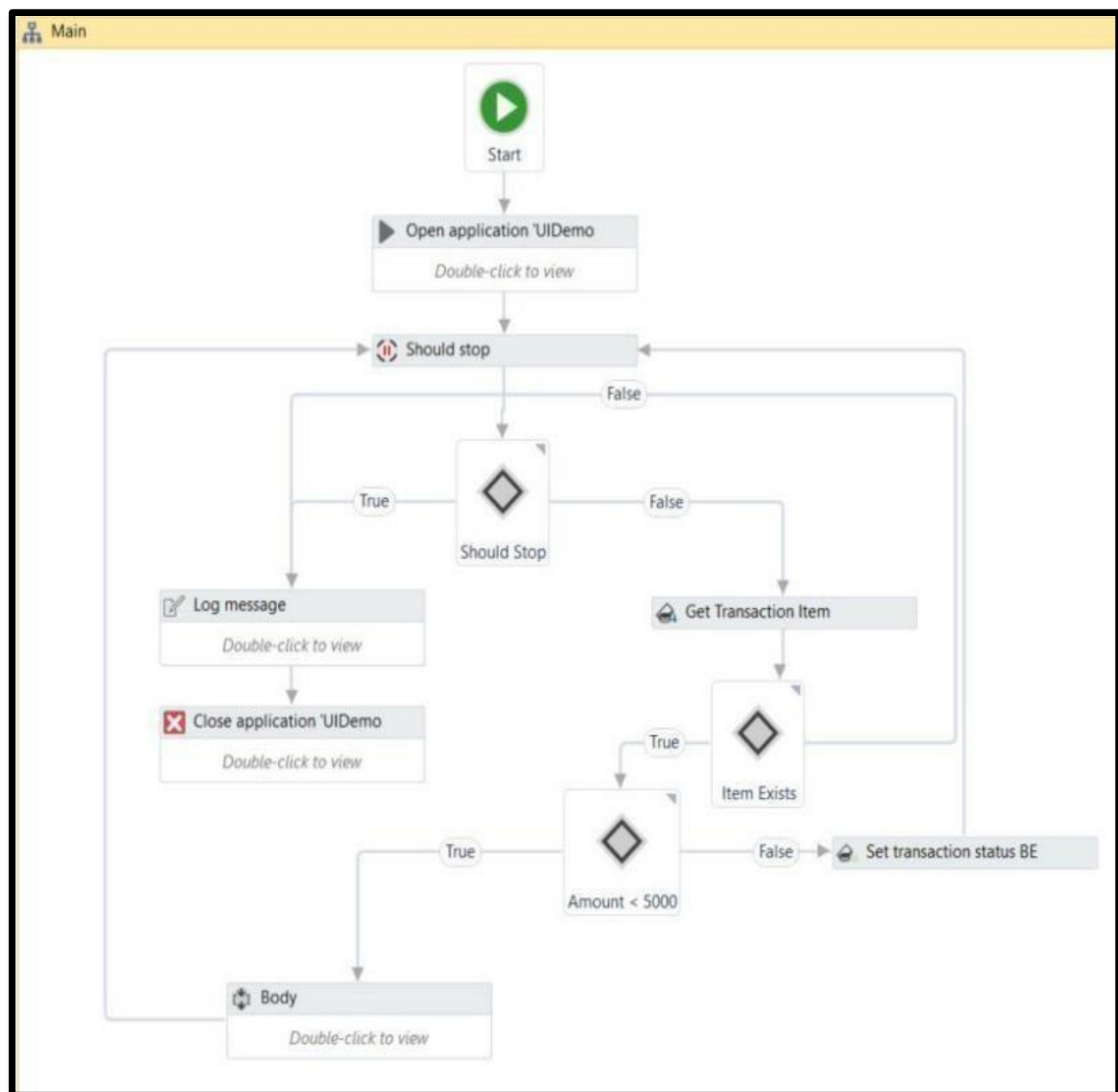
Set transaction status

Finally

Add an activity

34. Return to the flowchart and connect the previous Sequence activity (step 21) to the Should Stop activity (step 6) to create a loop and process the next transaction in the queue.

35. The flowchart should look similar to the following screenshot:



36. To run the process, click Design > Run.

37. After the execution has ended, you may navigate to

<https://platform.uipath.com> and display the processed transactions

with the corresponding processing details:

- Successful Transaction

DETAILS

COMMENTS

HISTORY

Transaction: 97e1470f-3f51-4325-a400-ed74a8db9dc7

▼ Specific Data: Object
CashIn: 350
OnUSCheck: 875
NotOnUsCheck: 210
Name: Scarlett Hunter

▼ Output Data: Object
transNumber: 637114

STATUS	REVISION	DEADLINE	RETRY NO.	POSTPONE	STARTED ↕	ENDED ↕	ROBOT ↕	EXCEPTION
Succe...	None		0		an hour ago	an hour ago	Gustave	

- Failed Transaction

DETAILS

COMMENTS

HISTORY

Transaction: 028f653b-38f8-4773-a4bc-3a2bea5282bc

▼ Specific Data: Object

CashIn: 2140

OnUSCheck: 5350

NotOnUsCheck: 1105

Name: Molly Adam

Output Data: undefined

Exception

- Reason: OnUSCheck amount is bigger than 5000

STATUS	REVISION	DEADLINE	RETRY NO.	POSTPONE	STARTED ↕	ENDED ↕	ROBOT ↕	EXCEPTION
Failed	None		0		an hour ago	an hour ago	Gustave	Business

CLOSE

d) Build UiPath Chatbot using Google dialogflow

- A chatbot is an artificial intelligence (AI) software that can simulate a conversation (or a chat) with an end user in natural language through messaging applications like – websites, mobile apps or through the telephone.

There are 2 important steps which will get carried out at the core of chatbot

They are:

- ❖ User Request
- ❖ Response from chatbot

Here we go



As you can see in above picture, a Chatbot returns a response based on input from a user. This process may look simple; in practice, things are quite complex.

How to create UiPath Chatbot?

Following are the Steps to create the UiPath Chatbot.

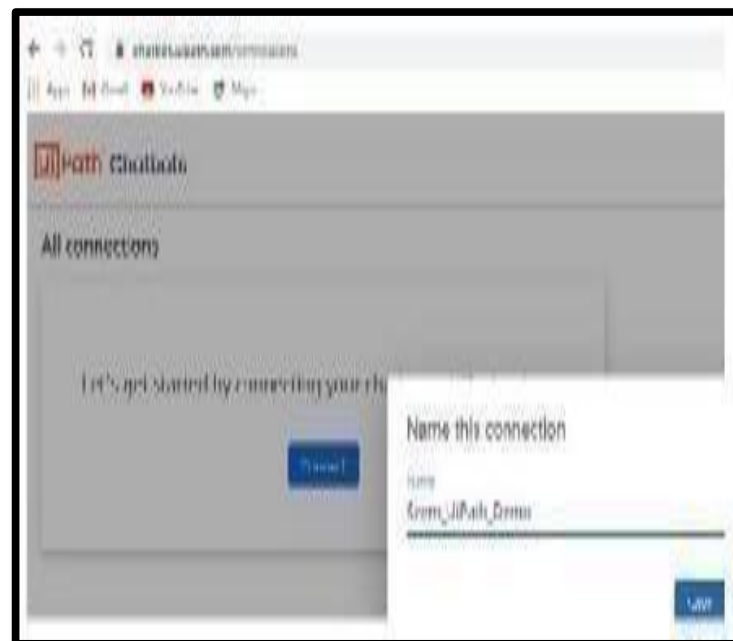
It requires multiple components tide together to work in coordinated ways.

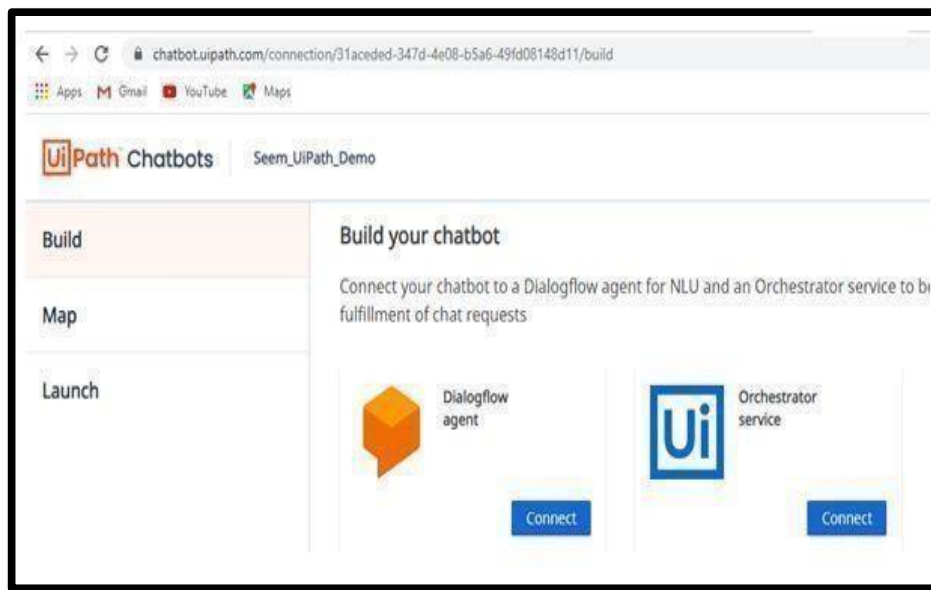
Pre-requisites

- Google Console — <https://console.cloud.google.com/>
- uipath chatbot — <https://chatbot.uipath.com/>

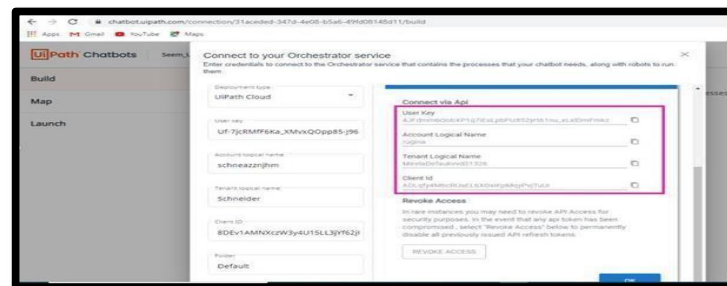
- Dialogflow — <https://dialogflow.cloud.google.com/>
- UiPath Orchestrator — <https://cloud.uipath.com/>

1. Create the connection in chatbot.uipath.com as below:



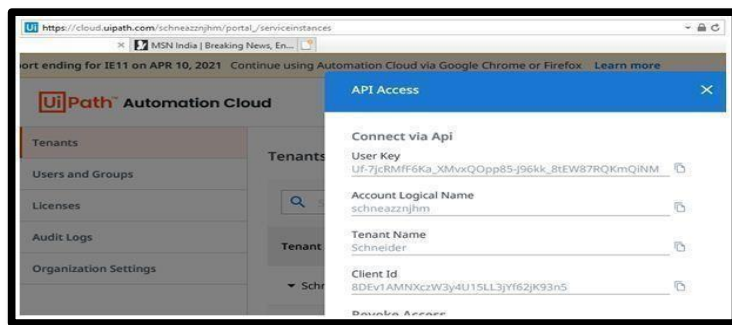


2. Now Connect to the **“Orchestrator Service”** from Orchestrator. Click on the **“Connect”** button of **“Orchestrator Service”**. You will get the below screen.

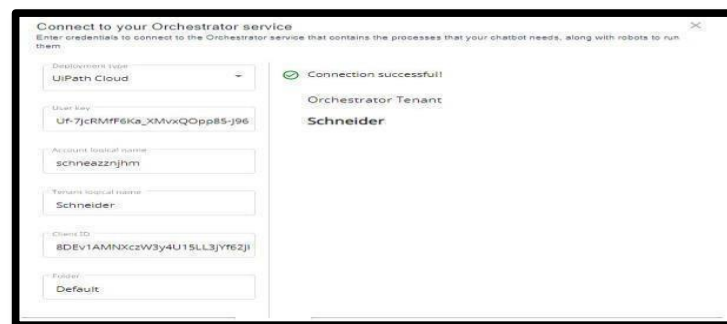


3. Enter all the fields in the above form from Orchestrator –

<https://cloud.uipath.com/> as shown in the below screen

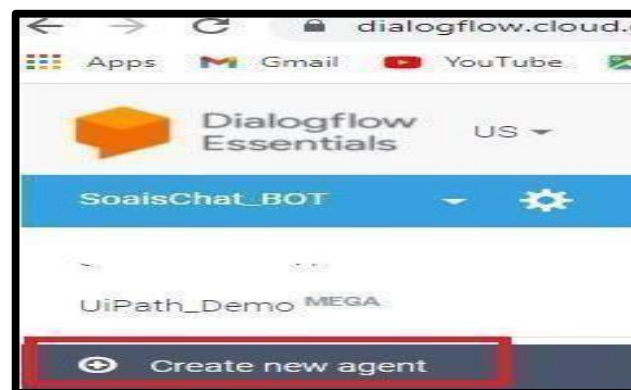


4. Copy all the above field sand fill it in to the following screen



5. Click on **connect** and now the **connection** is established with the orchestrator.

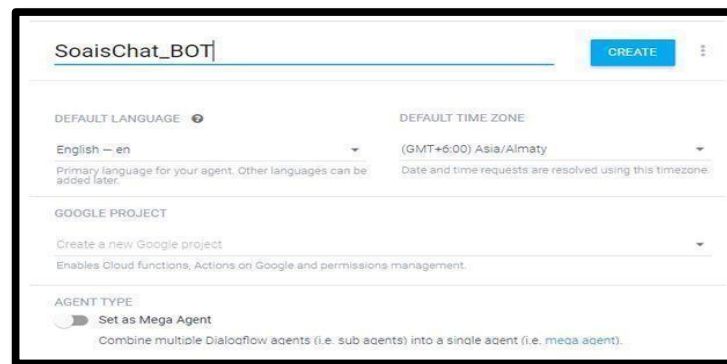
Creating the **Dialogflow** agent



6. It will open the below form enter the “**Agent name**”, select the Default time

zone,

Language – English

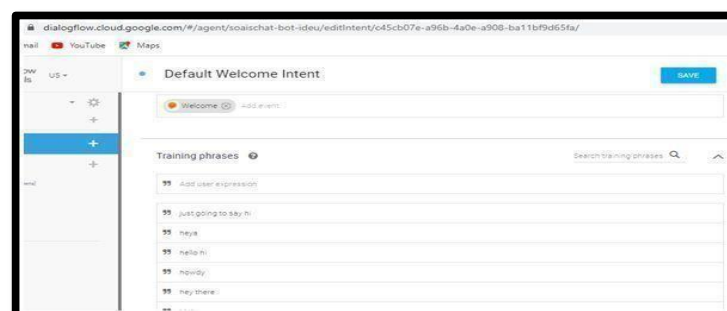


The screenshot shows the 'CREATE' form in the Google Dialogflow console. At the top, the agent name 'SoaisChat_BOT' is entered in a text field, followed by a blue 'CREATE' button. Below this, there are three main sections: 1. 'DEFAULT LANGUAGE' with a dropdown menu set to 'English - en' and a note: 'Primary language for your agent. Other languages can be added later.' 2. 'DEFAULT TIME ZONE' with a dropdown menu set to '(GMT+6:00) Asia/Almaty' and a note: 'Date and time requests are resolved using this timezone.' 3. 'GOOGLE PROJECT' with a dropdown menu set to 'Create a new Google project' and a note: 'Enables Cloud functions, Actions on Google and permissions management.' At the bottom, there is an 'AGENT TYPE' section with a toggle switch labeled 'Set as Mega Agent' and a note: 'Combine multiple Dialogflow agents (i.e. sub agents) into a single agent (i.e. mega agent).'

7. Creating the intents – intents are nothing but the conversations

between user and the chatbot

Add the training phrases in to the intents



The screenshot shows the 'Training phrases' section in the Google Dialogflow console for the 'Default Welcome Intent'. The interface includes a search bar labeled 'Search training phrases' and a list of training phrases. The phrases are: 'Add user expression', 'just going to say hi', 'hey!', 'hello hi', 'hey hi', and 'hey there!'. Each phrase is preceded by a small icon of a speech bubble. The list is scrollable, and there are plus and minus icons on the left side of the list.

And the response which you want to give to the end user from the bot as below

The screenshot shows the 'Default Welcome Intent' configuration in the Dialogflow console. At the top right is a blue 'SAVE' button. Below it is a table with columns: 'REQUIRED', 'PARAMETER NAME', 'ENTITY', 'VALUE', and 'IS LIST'. The table contains one row with placeholder text: 'Enter name', 'Enter entity', 'Enter value', and an unchecked checkbox. Below the table is a '+ New parameter' link. Further down is the 'Responses' section, which includes a 'DEFAULT' tab and a list of 'Text Response' items. The list contains four items:

REQUIRED	PARAMETER NAME	ENTITY	VALUE	IS LIST
☐	Enter name	Enter entity	Enter value	☐

+ New parameter

Responses

DEFAULT +

Text Response

- 1 Hi! How are you doing?
- 2 Hello! How can I help you?
- 3 Good day! What can I do for you today?
- 4 Goodnight! How can I assist?

Google Console

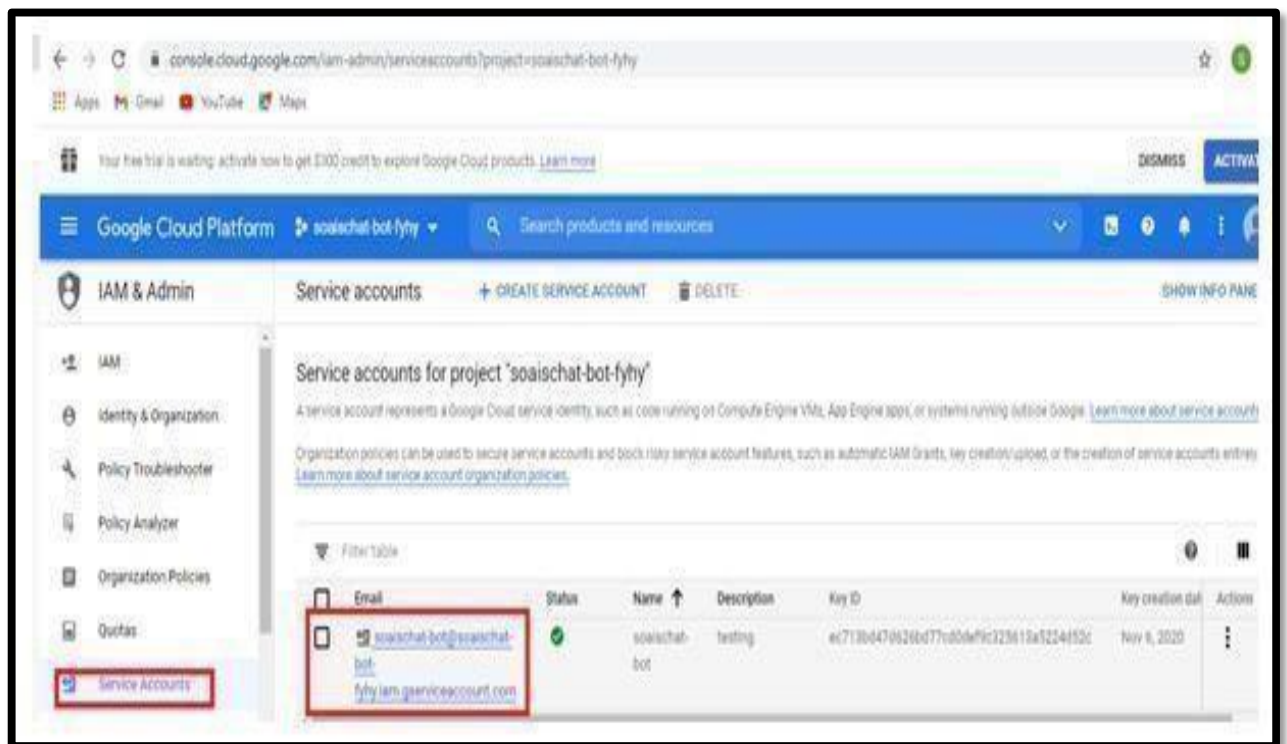
Google Console is used to get the – “Service Account” and the Primary key for

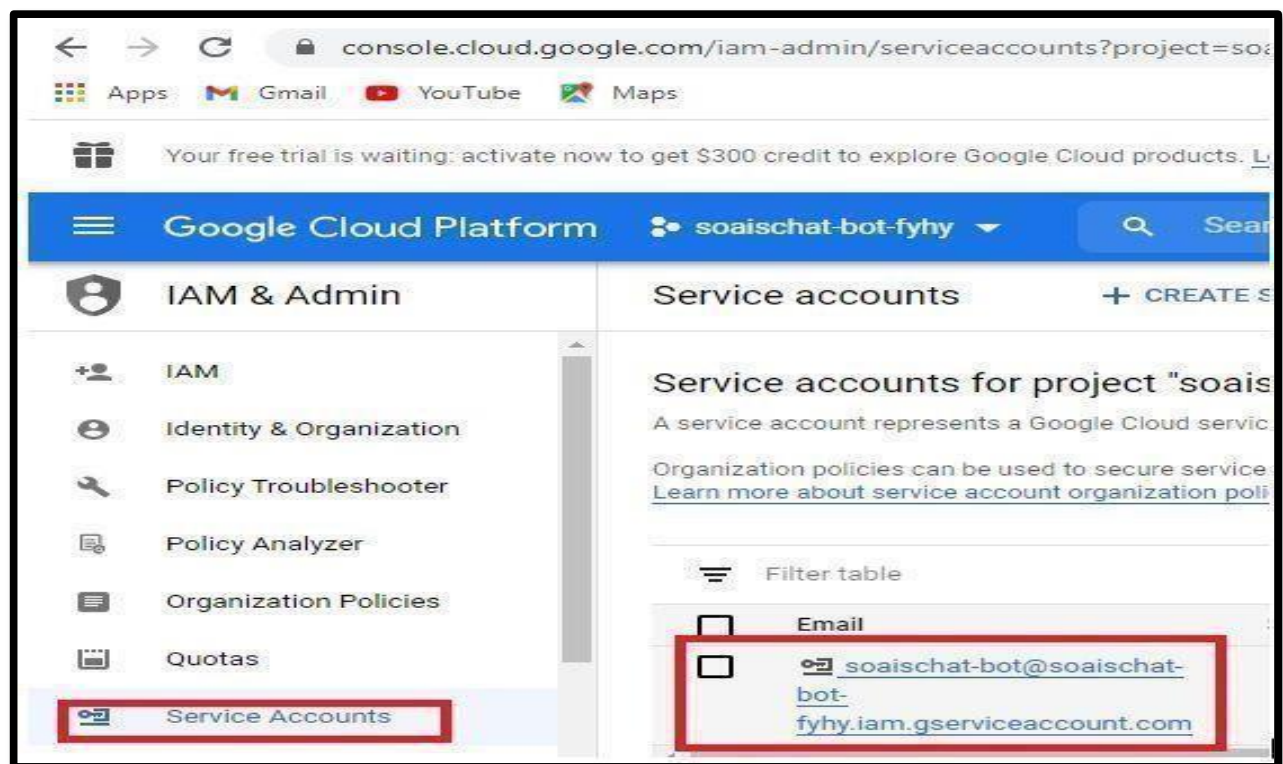
connecting the –Dialogflow agent present in the Chatbot.uipath.com

Now we are done with the “Dialogflow”.

Next step **Google console**

8. Open the google console – <https://console.cloud.google.com/>





NOTE: The service account is automatically generated from the Google

Dialogflow, which is connected to the Google Console internally. Now getting the

“Primary Key”



Email	Status	Name	Description	Key ID	Key creation date	Actions
soaischat-bot@soaischat-bot-fyhyiam.gs-serviceaccount.com	✓	soaischat-bot	testing	ec713bd47d626bd77cd0def9c325613a5224d52c	Nov 6, 2020	Edit Disable Create key Delete

Create private key for "soaischat-bot"

Downloads a file that contains the private key. Store the file securely because this key can't be recovered if lost.

Key type

☒ JSON
Recommended

☐ P12
For backward compatibility with code using the P12 format

CANCEL CREATE

9. Get the Jason file and enter the –**Primary key**. You can find the Primary key

as below

Connect to your Dialogflow agent
Enter credentials to connect to a Dialogflow agent that provides NLU capabilities for your chatbot.

Project ID
soaischat-bot-fyhy

Service account
soaischat-bot@soaischat-bot-fyh

Private key
-----BEGIN PRIVATE KEY-----
\\nMIEvQIBADANBgkqhkiG9w
0BAQEFAASCBCkwggSjAgEAA
oIBAQDAyHrKEBJZ40kj
ZrsqhtRDn12GAfTXp7Rp7hpC
wh+N2B4U0mAsjzwc6HB/Dco
AgqAFIRuGHemqslYr
f9t2N10VxCCpZCepsu199HI20
R0GtphpjaWIEj8Bas6sAytX8X8
flajbu4MAKUAM
98njmTD3D/5usOY6TQAoCeZ
AfnGfnZXHcsmMdGgD9des9tx
WkZ9tWS58Q5X7z/ji
2x4jf4GoIK+o67LtLxPw6kPhtx
dDDLjsASAJKrn2m7NIX/IF0vY
8H9jkPO8my/Sq

✓ Connection successful!

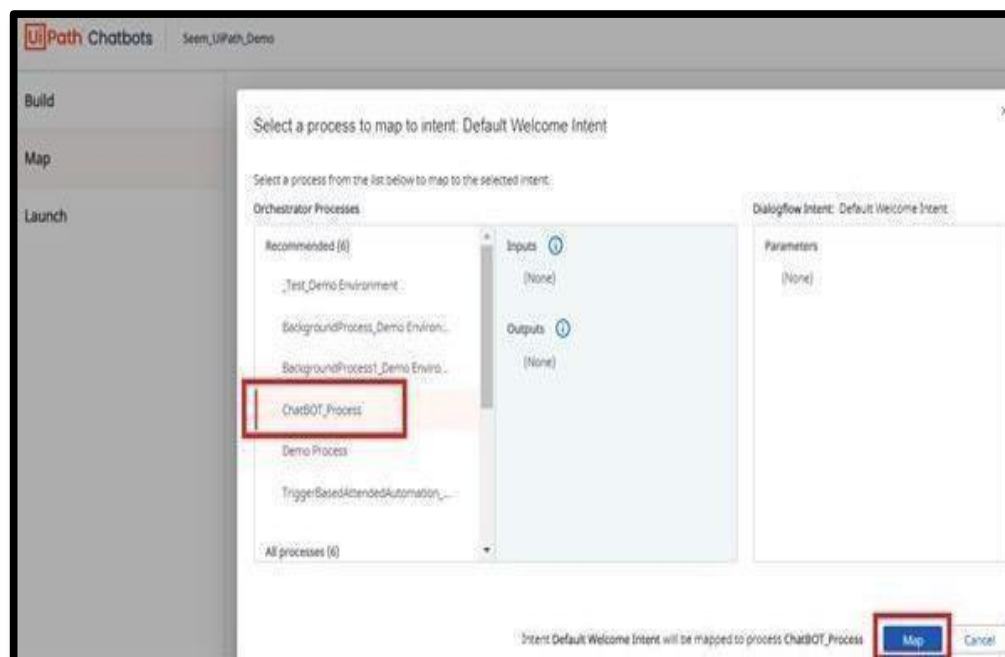
Dialogflow Agent
SoaisChat_BOT

So now we are done with connecting to the dialogflow in the Chatbot.uipath.com

Once you are connected to the “Orchestrator Service”,

in **chatbot.uipath.com**, click on the “Map” and you can see the list of process which are present in the Orchestrator you have connected.

Select the “chatbot_process” from the list of robots. Finally click on “Map”.

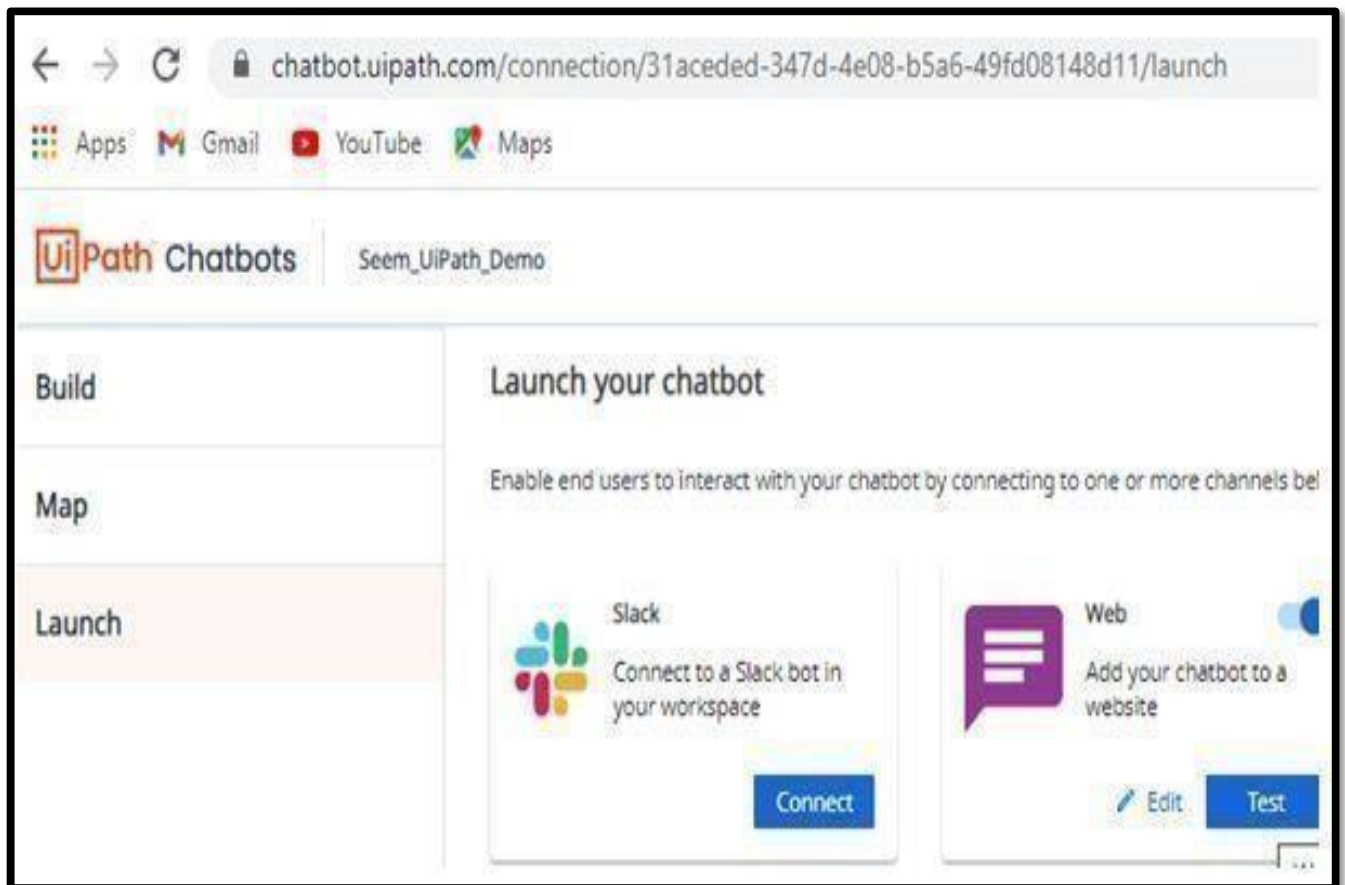


Now we can see the mapped robot

Map intents to processes			
Map processes to run from the connected Orchestrator service to one or more intents listed below from the connected Dialogflow agent			
All intents			
Intent	Status	Process	Date Mapped
Default Fallback Intent	Unmapped		
Default Welcome Intent	Mapped	ChatBOT_Process	November 15, 2020
soas	Unmapped		

12. Now click on the “**Launch**”

Click on “Test” as shown below:



Once the “Test” is clicked the webpage is opened So now user

can chat with the chat bot

And our “chatbot_process” Robot has simple “Hi” message which is shown

finally.

Output:

